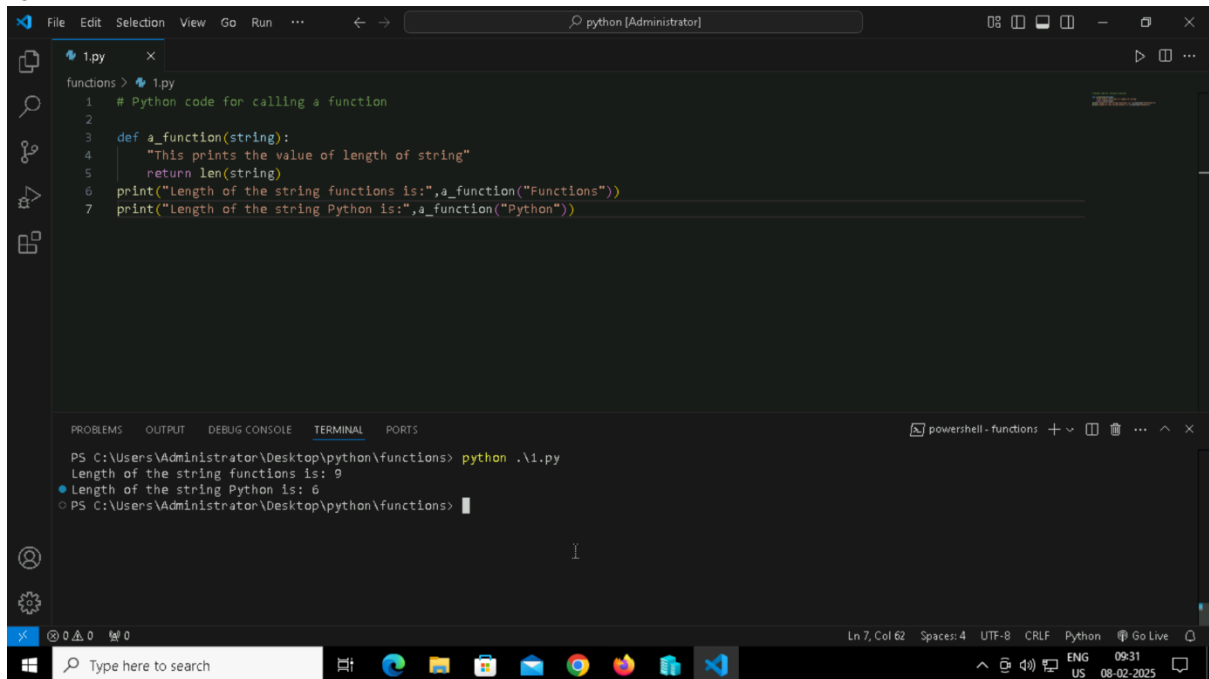


Python Functions

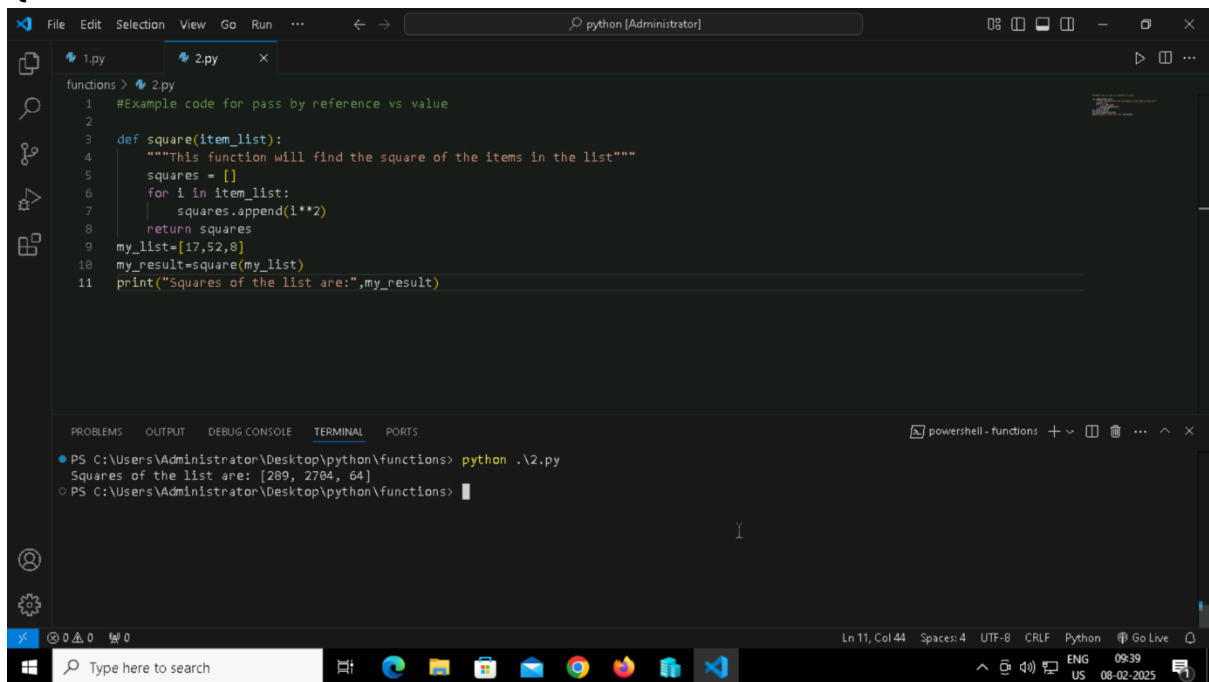
Q.



The screenshot shows the Visual Studio Code editor with a file named `1.py` open. The code defines a function `a_function` that takes a string and returns its length. It then prints the length of the string "Functions" and the string "Python". The terminal output shows the execution of `python .\1.py`, resulting in the following output:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\1.py
Length of the string functions is: 9
Length of the string Python is: 6
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.



The screenshot shows the Visual Studio Code editor with a file named `2.py` open. The code defines a function `square` that takes a list and returns a new list containing the squares of the elements. It then prints the squares of the list `[17, 52, 8]`. The terminal output shows the execution of `python .\2.py`, resulting in the following output:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\2.py
Squares of the list are: [289, 2704, 64]
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.

The screenshot shows the Visual Studio Code editor with a file named `3.py` open. The code defines a function `function(n1, n2=20)` that prints the values of `n1` and `n2`. It then calls the function twice: first with a single argument `30`, and then with two arguments `50` and `30`.

```
1 # Python default arguments
2 def function(n1,n2=20):
3     print("Number 1 is:",n1)
4     print("Number 2 is:",n2)
5 print("Passing only one argument")
6 function(30)
7
8 print("Passing two arguments")
9 function(50,30)
```

The terminal output shows the execution of the script:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\3.py
Passing only one argument
Number 1 is: 30
Number 2 is: 20
Passing two arguments
Number 1 is: 50
Number 2 is: 30
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.

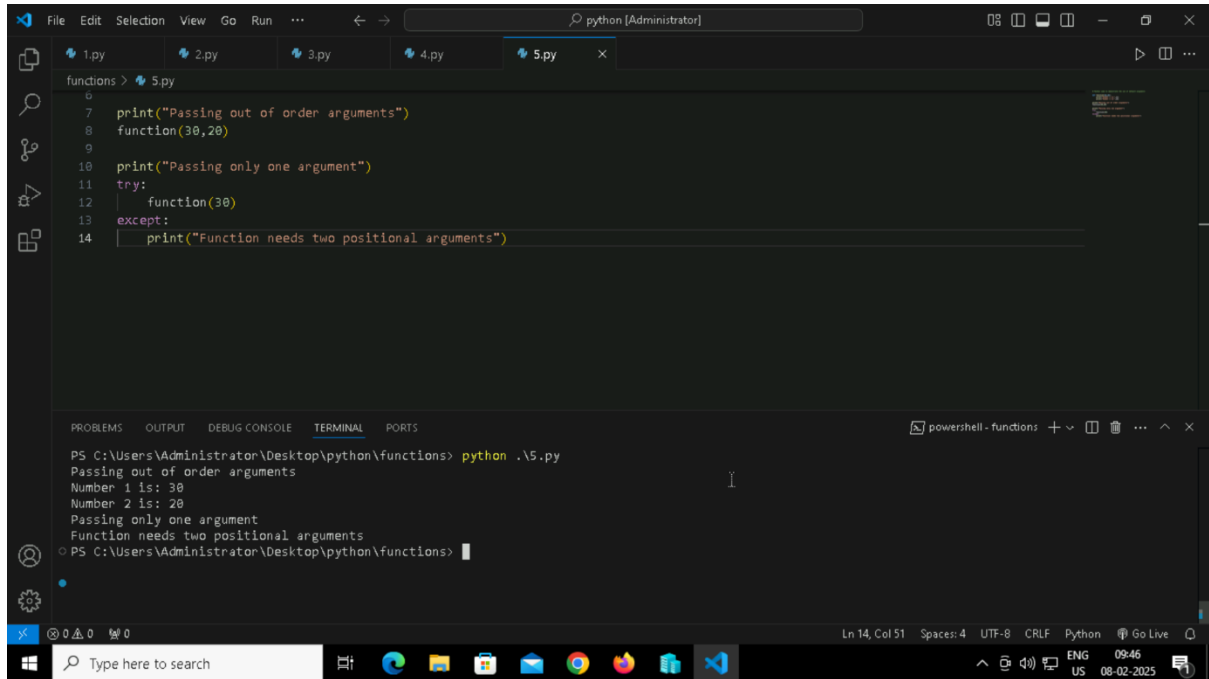
The screenshot shows the Visual Studio Code editor with a file named `4.py` open. The code defines a function `function(n1, n2)` that prints the values of `n1` and `n2`. It then calls the function twice: first with two arguments `50` and `30` without using keyword arguments, and then with two arguments `n2=50` and `n1=30` using keyword arguments.

```
1 #Python code to demonstrate the use of keyword arguments
2
3 def function(n1,n2):
4     print("Number 1 is:",n1)
5     print("Number 2 is:",n2)
6
7 print("Without using keyword")
8 function(50,30)
9
10 print("With using keyword")
11 function(n2=50,n1=30)
```

The terminal output shows the execution of the script:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\4.py
Without using keyword
Number 1 is: 50
Number 2 is: 30
With using keyword
Number 1 is: 30
Number 2 is: 50
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.



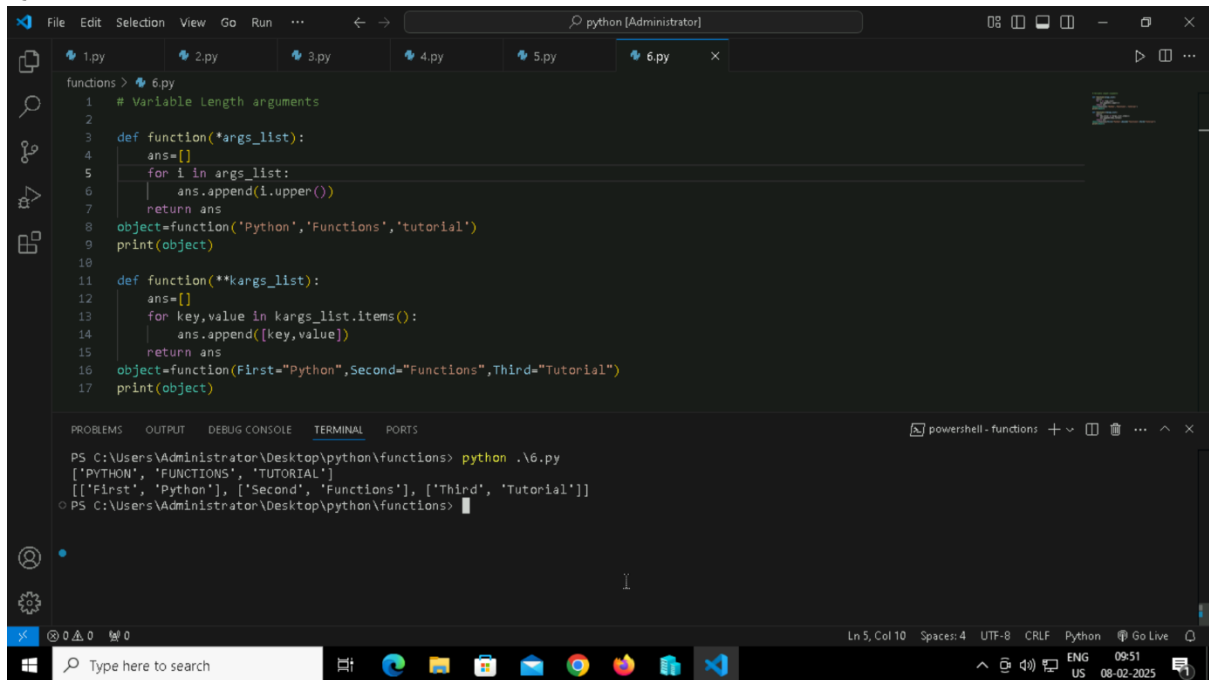
The screenshot shows the Visual Studio Code editor with a file named `5.py` open. The code in the editor is as follows:

```
functions > 5.py
6
7 print("Passing out of order arguments")
8 function(30,20)
9
10 print("Passing only one argument")
11 try:
12     function(30)
13 except:
14     print("Function needs two positional arguments")
```

The terminal window at the bottom shows the output of running `python .\5.py`:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\5.py
Passing out of order arguments
Number 1 is: 30
Number 2 is: 20
Passing only one argument
Function needs two positional arguments
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.



The screenshot shows the Visual Studio Code editor with a file named `6.py` open. The code in the editor is as follows:

```
functions > 6.py
1 # Variable Length arguments
2
3 def function(*args_list):
4     ans=[]
5     for i in args_list:
6         ans.append(i.upper())
7     return ans
8 object=function('Python','Functions','tutorial')
9 print(object)
10
11 def function(**kargs_list):
12     ans=[]
13     for key,value in kargs_list.items():
14         ans.append([key,value])
15     return ans
16 object=function(First="Python",Second="Functions",Third="Tutorial")
17 print(object)
```

The terminal window at the bottom shows the output of running `python .\6.py`:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\6.py
['PYTHON', 'FUNCTIONS', 'TUTORIAL']
[['First', 'Python'], ['Second', 'Functions'], ['Third', 'Tutorial']]
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.

The screenshot shows the Visual Studio Code editor with a file named `7.py` open. The code defines a function `square` and prints its output for `52` with and without a `return` statement. The terminal shows the output of running `python .\7.py`.

```
functions > 7.py
1 # Use of return statements
2 def square(num):
3     return num**2
4
5 print("With return statement")
6 print(square(52))
7
8 def square(num):
9     num**2
10
11 print("Without return statement")
12 print(square(52))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Administrator\Desktop\python\functions> python .\7.py
With return statement
2704
Without return statement
None
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.

The screenshot shows the Visual Studio Code editor with a file named `8.py` open. The code defines an anonymous function using `lambda` and prints its output for two different arguments. The terminal shows the output of running `python .\8.py`.

```
functions > 8.py
1 # Anonymous functions, i.e, lambda functions
2
3 lambda_=lambda argument1,argument2:argument1+argument2
4
5 print("Value of the function is:",lambda_(20,30))
6 print("Value of the function is:",lambda_(40,50))
7
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Administrator\Desktop\python\functions> python .\8.py
Value of the function is: 50
Value of the function is: 90
PS C:\Users\Administrator\Desktop\python\functions>
```


Q.

The screenshot shows the Visual Studio Code editor with a Python file named 9.py. The code defines a function `number()` that prints the value of a variable `num` inside the function and then prints the value of `num` outside the function. The terminal shows the output of running the script, which is "Value of num inside the function: 50" and "Value of num outside the function: 10".

```
functions > 9.py
1 # Scope and lifetime of variables
2 def number():
3     num=50
4     print("Value of num inside the function:",num)
5 num=10
6 number()
7 print("Value of num outside the function",num)
```

```
PS C:\Users\Administrator\Desktop\python\functions> python .\9.py
Value of num inside the function: 50
Value of num outside the function: 10
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.

The screenshot shows the Visual Studio Code editor with a Python file named 10.py. The code defines a function `word()` that prints the value of a variable `x` inside the function and then prints the value of `x` outside the function. The terminal shows the output of running the script, which is "Python functions tutorial" and "5".

```
functions > 10.py
1 # Python nested functions
2 def word():
3     string='Python functions tutorial'
4     x=5
5     def number():
6         print(string)
7         print(x)
8     number()
9 word()
```

```
PS C:\Users\Administrator\Desktop\python\functions> python .\10.py
Python functions tutorial
5
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.

The screenshot shows the Visual Studio Code editor with a Python file named 11.py. The code defines an example of the abs() function. The terminal shows the output of running the script.

```
functions > 11.py
1 # abs() function example
2 integer=-20
3 print('Absolute value of -20 is:',abs(integer))
4
5 floating=-20.83
6 print('Absolute value of -20.83 is:',abs(floating))
```

```
PS C:\Users\Administrator\Desktop\python\functions> python .\11.py
Absolute value of -20 is: 20
Absolute value of -20.83 is: 20.83
PS C:\Users\Administrator\Desktop\python\functions>
```

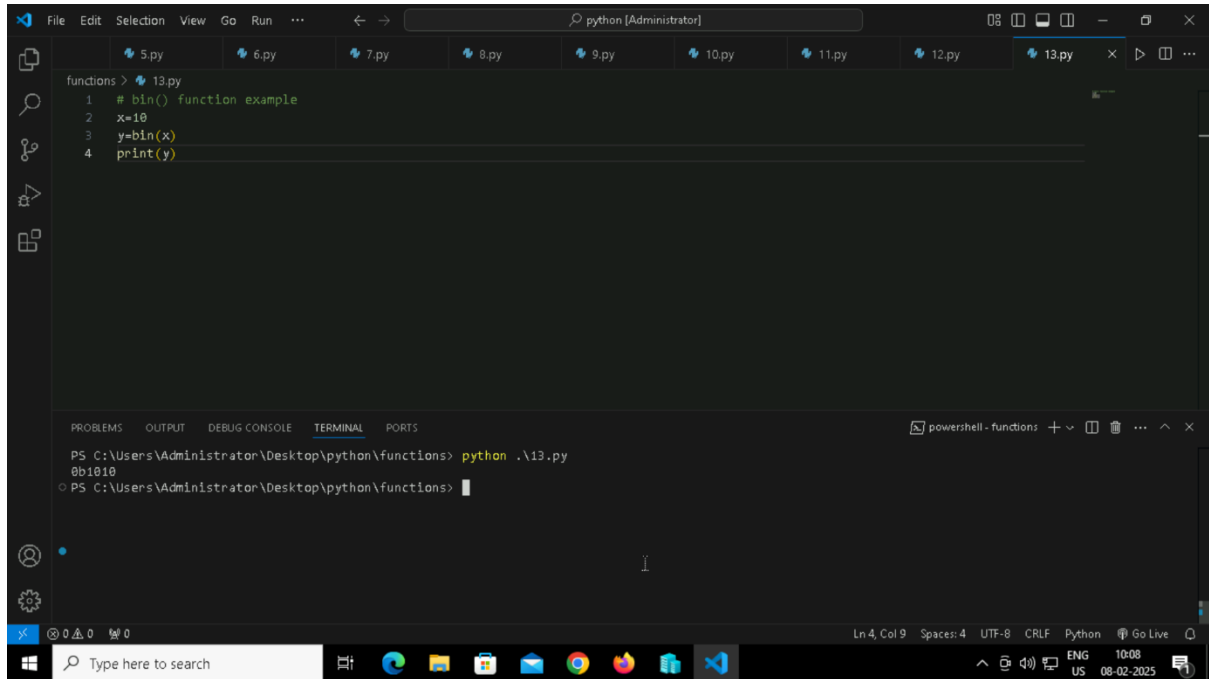
Q.

The screenshot shows the Visual Studio Code editor with a Python file named 12.py. The code defines an example of the all() function. The terminal shows the output of running the script.

```
functions > 12.py
1 # Python all() function example
2
3 k=[1,3,4,6]
4 print(all(k))
5
6 k=[0,False]
7 print(all(k))
8
9 k=[1,3,7,0]
10 print(all(k))
11
12 k=[]
13 print(all(k))
```

```
PS C:\Users\Administrator\Desktop\python\functions> python .\12.py
True
False
False
True
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.



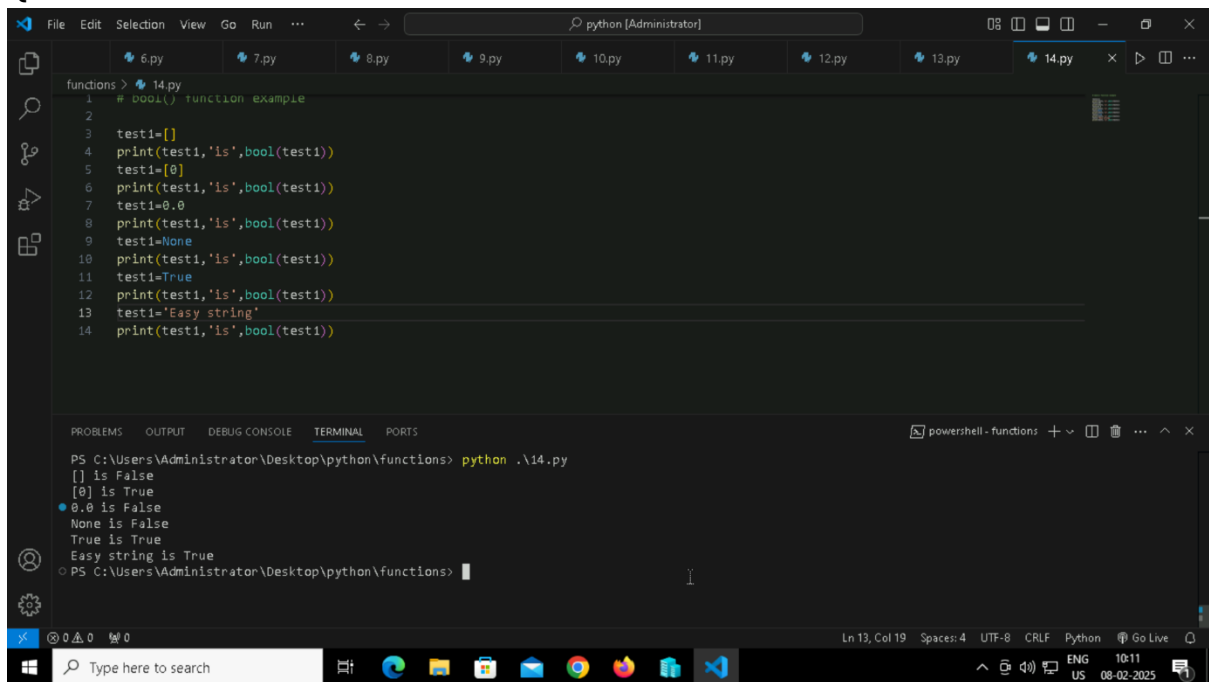
The screenshot shows the Visual Studio Code editor with a file explorer on the left displaying a directory named 'functions' containing files 5.py through 13.py. The editor window shows the content of 13.py:

```
functions > 13.py
1 # bin() function example
2 x=10
3 y=bin(x)
4 print(y)
```

The terminal at the bottom shows the command `python .\13.py` being executed, resulting in the output `0b1010`.

```
PS C:\Users\Administrator\Desktop\python\functions> python .\13.py
0b1010
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.



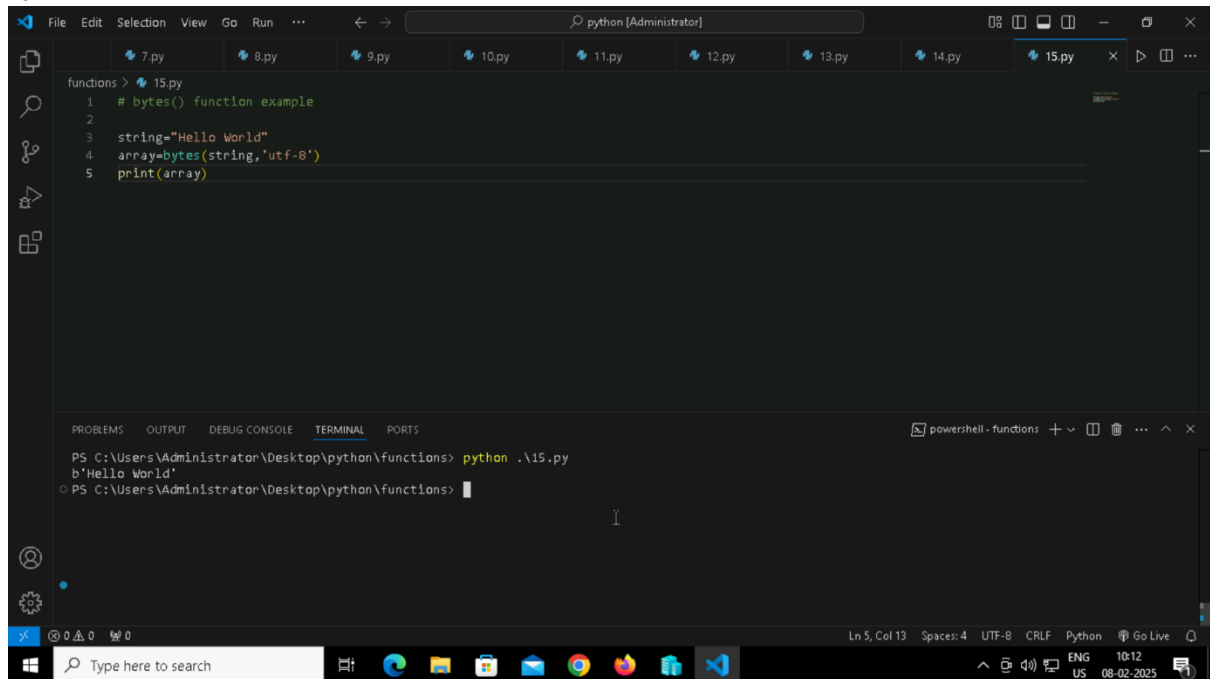
The screenshot shows the Visual Studio Code editor with a file explorer on the left displaying a directory named 'functions' containing files 6.py through 14.py. The editor window shows the content of 14.py:

```
functions > 14.py
1 # bool() function example
2
3 test1=[]
4 print(test1,'is',bool(test1))
5 test1=[0]
6 print(test1,'is',bool(test1))
7 test1=0.0
8 print(test1,'is',bool(test1))
9 test1=None
10 print(test1,'is',bool(test1))
11 test1=True
12 print(test1,'is',bool(test1))
13 test1='Easy string'
14 print(test1,'is',bool(test1))
```

The terminal at the bottom shows the command `python .\14.py` being executed, resulting in the following output:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\14.py
[] is False
[0] is True
0.0 is False
None is False
True is True
Easy string is True
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.



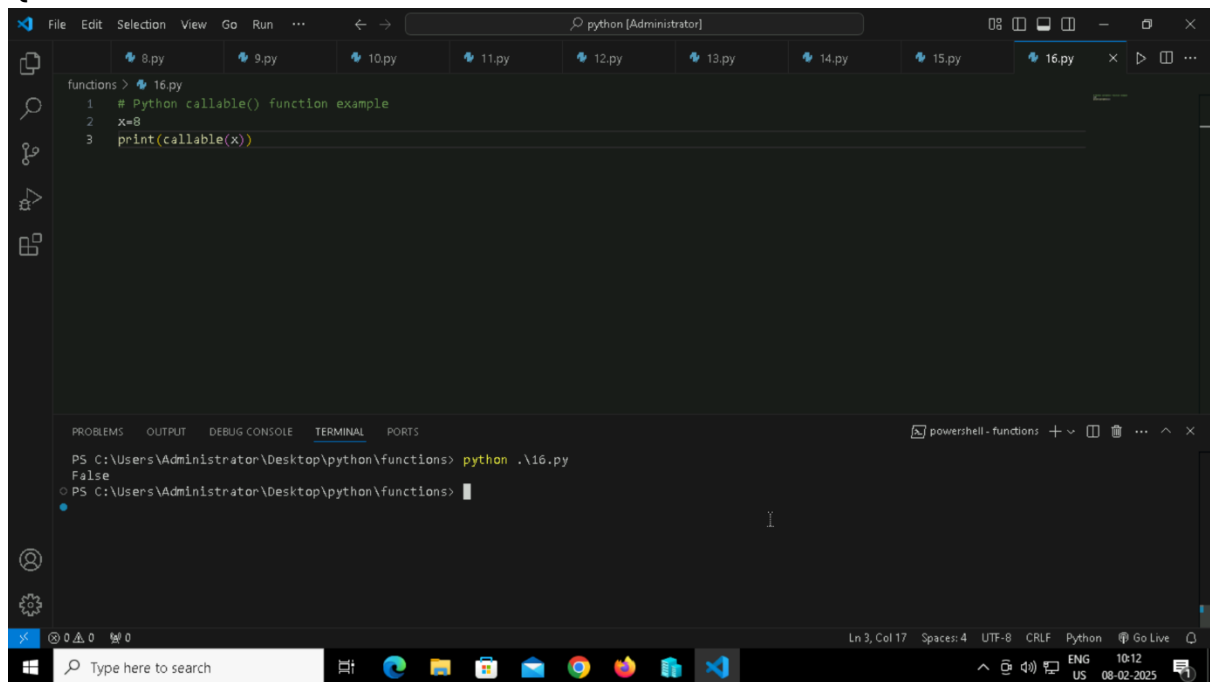
The screenshot shows the Visual Studio Code editor with a file named `15.py` open. The code in the editor is as follows:

```
functions > 15.py
1 # bytes() function example
2
3 string="Hello World"
4 array=bytes(string,'utf-8')
5 print(array)
```

The terminal window at the bottom shows the command `python .\15.py` being executed, resulting in the output `b'Hello World'`.

```
PS C:\Users\Administrator\Desktop\python\functions> python .\15.py
b'Hello World'
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.



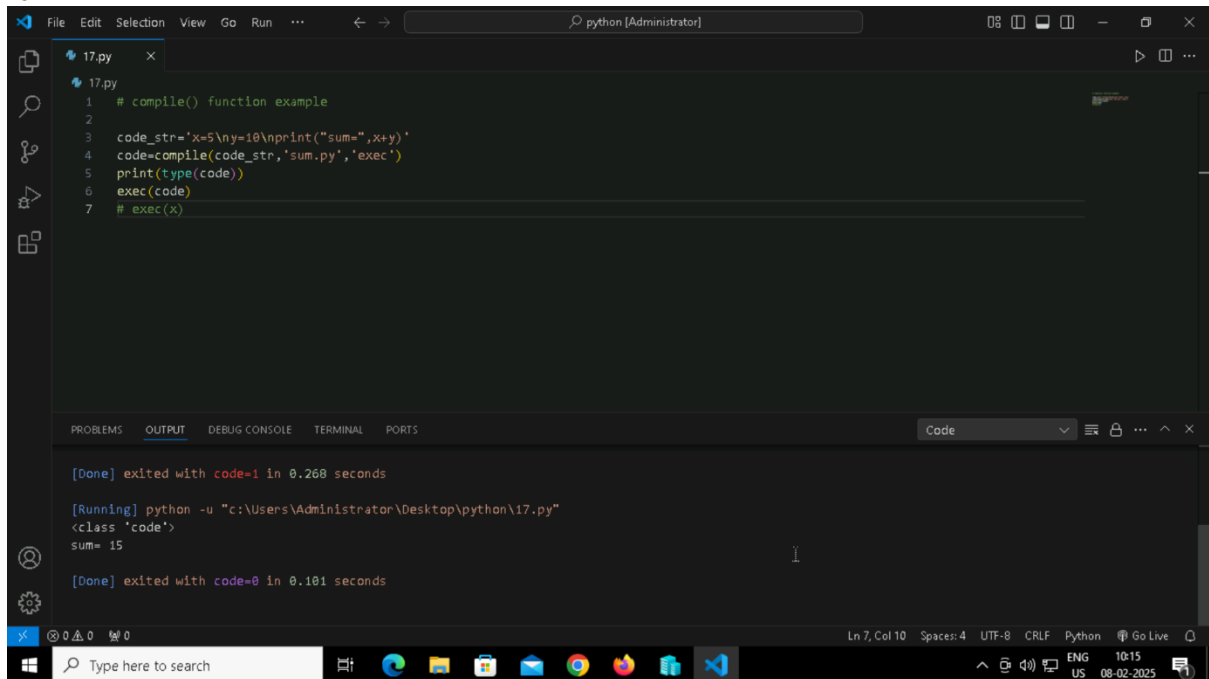
The screenshot shows the Visual Studio Code editor with a file named `16.py` open. The code in the editor is as follows:

```
functions > 16.py
1 # Python callable() function example
2 x=8
3 print(callable(x))
```

The terminal window at the bottom shows the command `python .\16.py` being executed, resulting in the output `False`.

```
PS C:\Users\Administrator\Desktop\python\functions> python .\16.py
False
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.



The screenshot shows the Visual Studio Code editor with a file named `17.py` open. The code in the editor is as follows:

```
1 # compile() function example
2
3 code_str='x=5\ny=10\nprint("sum=",x+y)'
4 code=compile(code_str,'sum.py','exec')
5 print(type(code))
6 exec(code)
7 # exec(x)
```

The terminal output shows the execution of the script:

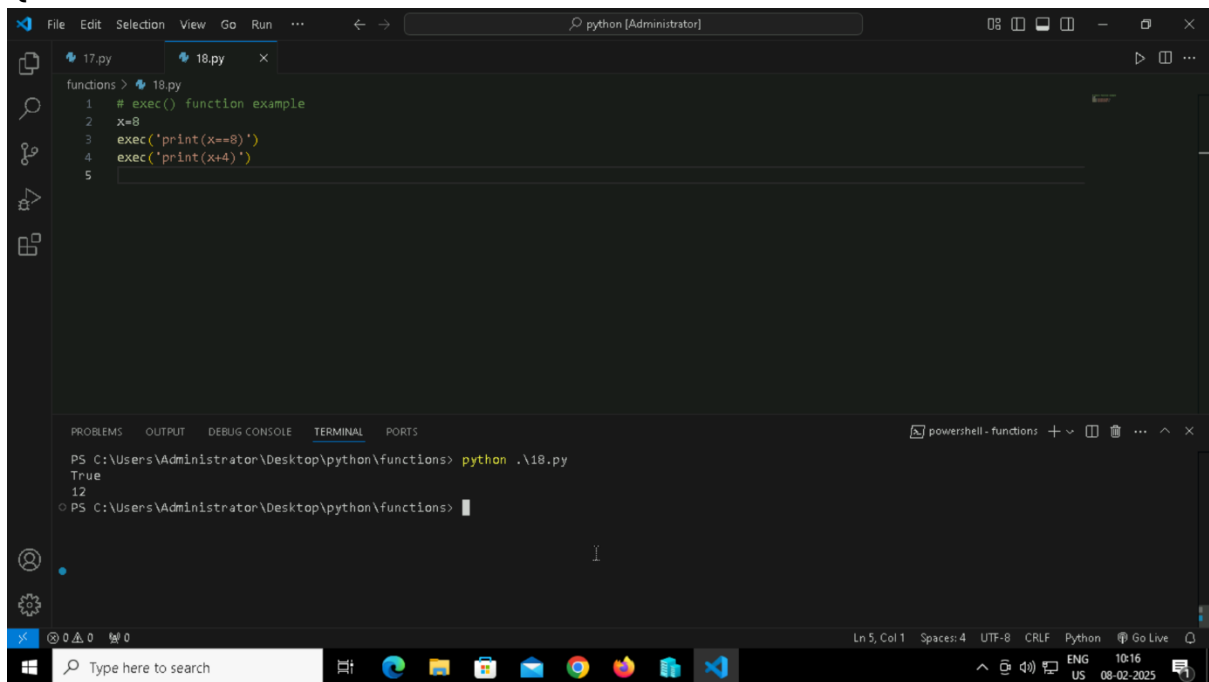
```
[Done] exited with code=1 in 0.268 seconds

[Running] python -u "c:\Users\Administrator\Desktop\python\17.py"
<class 'code'>
sum= 15

[Done] exited with code=0 in 0.101 seconds
```

The status bar at the bottom indicates the cursor is at line 7, column 10, with 4 spaces, in UTF-8 encoding, CRLF line endings, and Python language mode.

Q.



The screenshot shows the Visual Studio Code editor with a file named `18.py` open. The code in the editor is as follows:

```
1 # exec() function example
2 x=8
3 exec('print(x==8)')
4 exec('print(x+4)')
5
```

The terminal output shows the execution of the script:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\18.py
True
12
PS C:\Users\Administrator\Desktop\python\functions>
```

The status bar at the bottom indicates the cursor is at line 5, column 1, with 4 spaces, in UTF-8 encoding, CRLF line endings, and PowerShell language mode.

Q.

The screenshot shows the Visual Studio Code editor with a file named `19.py` open. The code in the editor is as follows:

```
functions > 19.py
1 # python sum() function example
2 s=sum([1,2,4])
3 print(s)
4
5 s=sum([1,2,4],10)
6 print(s)
```

The terminal window at the bottom shows the command `python .\19.py` being executed, with the output:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\19.py
7
17
PS C:\Users\Administrator\Desktop\python\functions>
```

The status bar at the bottom indicates the file is at line 6, column 9, using UTF-8 encoding with CRLF line endings. The system tray shows the date as 08-02-2025 and time as 10:17.

Q.

The screenshot shows the Visual Studio Code editor with a file named `20.py` open. The code in the editor is as follows:

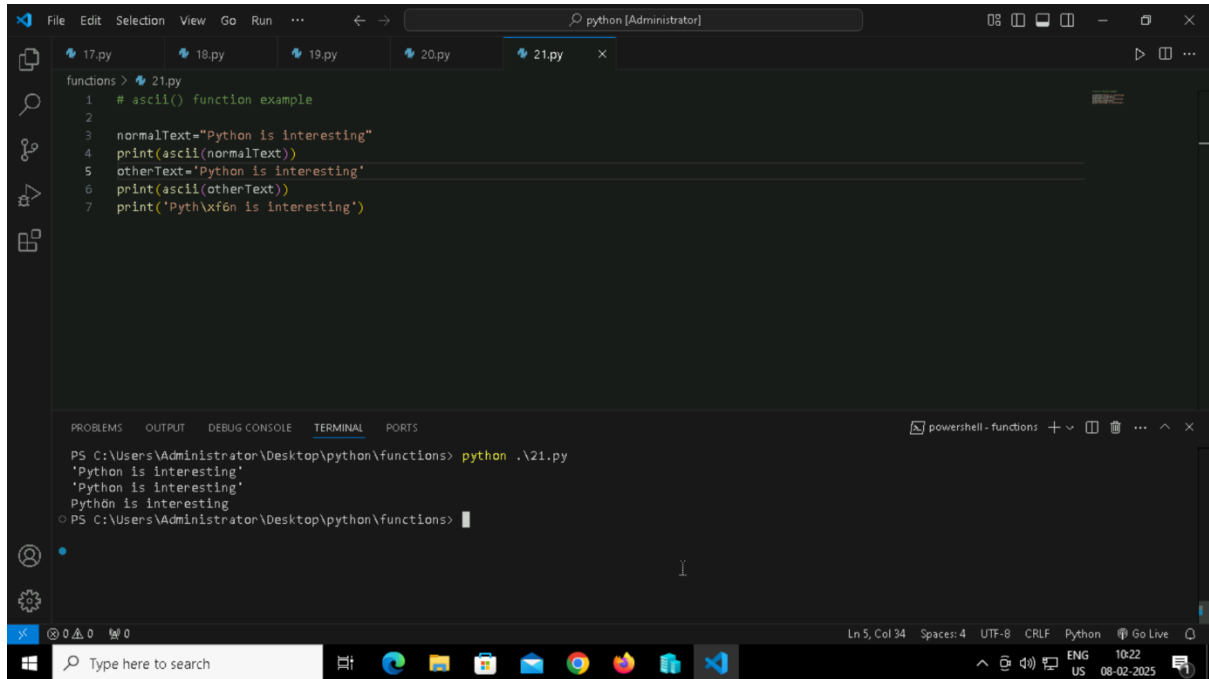
```
functions > 20.py
1 # any() function example
2
3 l=[4,3,2,0]
4 print(any(l))
5
6 l=[0,False]
7 print(any(l))
8
9 l=[0,False,5]
10 print(any(l))
11
12 l=[]
13 print(any(l))
```

The terminal window at the bottom shows the command `python .\20.py` being executed, with the output:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\20.py
True
False
True
False
PS C:\Users\Administrator\Desktop\python\functions>
```

The status bar at the bottom indicates the file is at line 13, column 14, using UTF-8 encoding with CRLF line endings. The system tray shows the date as 08-02-2025 and time as 10:19.

Q.



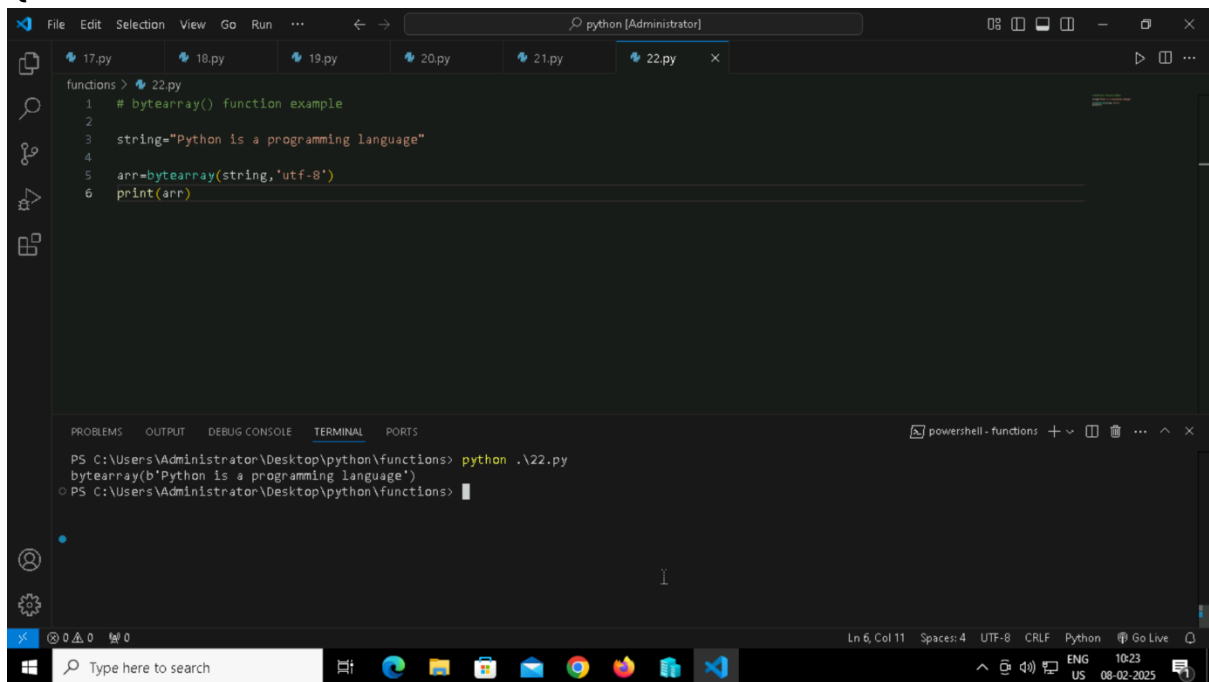
The screenshot shows the Visual Studio Code editor with a file named 21.py open. The code in the editor is as follows:

```
functions > 21.py
1 # ascii() function example
2
3 normalText="Python is interesting"
4 print(ascii(normalText))
5 otherText='Python is interesting'
6 print(ascii(otherText))
7 print('Pyth\xfdn is interesting')
```

The terminal at the bottom shows the command `python .\21.py` being executed, resulting in the following output:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\21.py
'Python is interesting'
'Python is interesting'
Python is interesting
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.



The screenshot shows the Visual Studio Code editor with a file named 22.py open. The code in the editor is as follows:

```
functions > 22.py
1 # bytearray() function example
2
3 string="Python is a programming language"
4
5 arr=bytearray(string,'utf-8')
6 print(arr)
```

The terminal at the bottom shows the command `python .\22.py` being executed, resulting in the following output:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\22.py
bytearray(b'Python is a programming language')
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.

The screenshot shows the Visual Studio Code editor with a file named `23.py` open. The code in the editor is:

```
functions > 23.py
1 # eval() function example()
2 x=8
3 print(eval('x+1'))
```

The terminal at the bottom shows the command `python .\23.py` being executed, resulting in the output `9`.

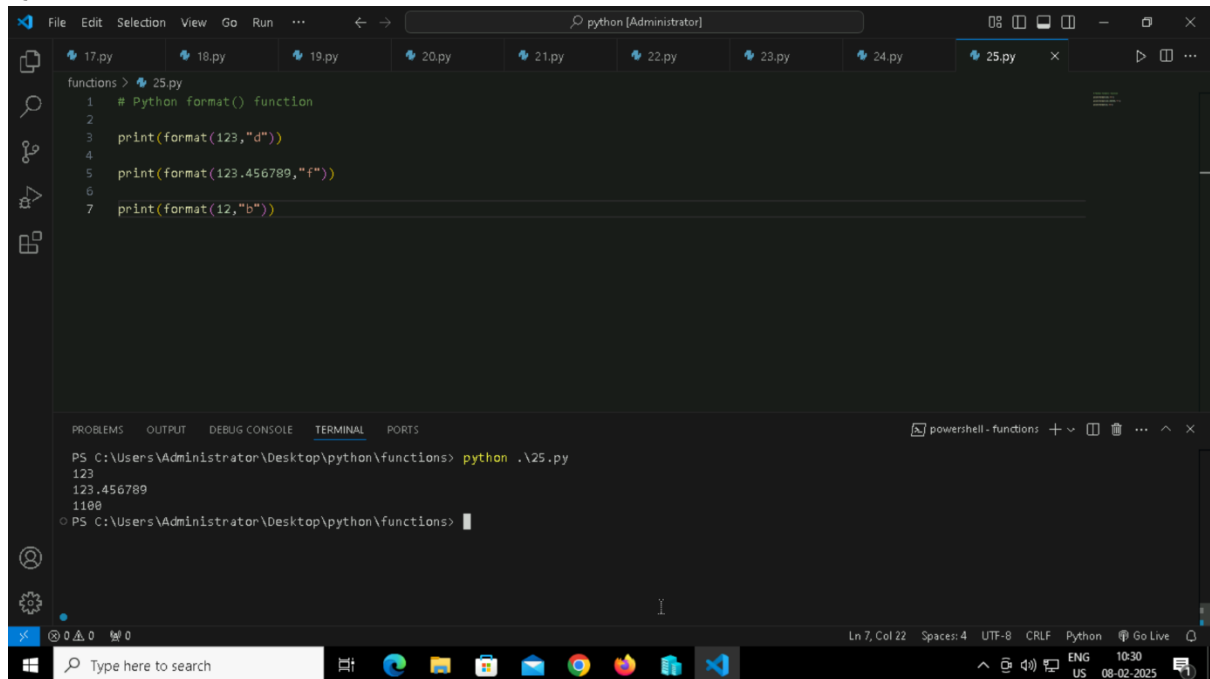
Q.

The screenshot shows the Visual Studio Code editor with a file named `24.py` open. The code in the editor is:

```
functions > 24.py
1 # Python float() example
2 print(float(9))
3
4 print(float(8.19))
5
6 print(float("-24.27"))
7
8 print(float("-17.19\n"))
9
10 print(float("xyz"))
```

The terminal at the bottom shows the command `python .\24.py` being executed. The output shows the first four lines of the script running successfully, but then a `ValueError` is raised at line 10: `ValueError: could not convert string to float: 'xyz'`. The terminal also shows the traceback information.

Q.

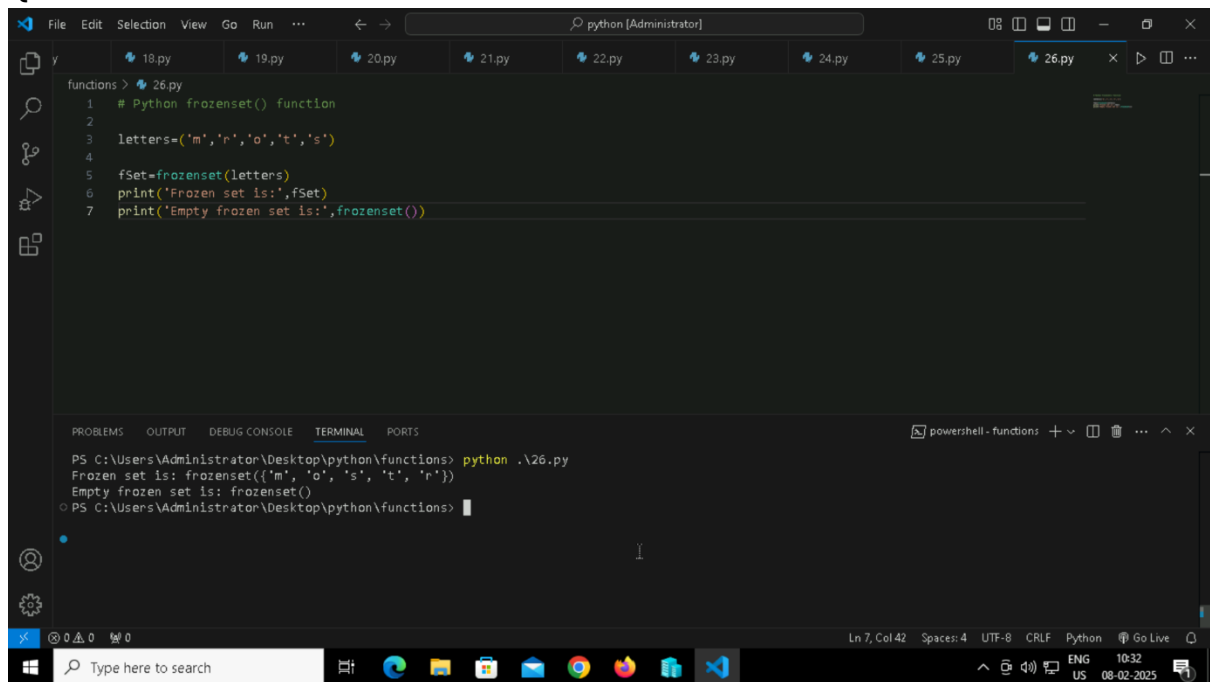


The screenshot shows the Visual Studio Code editor with a Python file named 25.py. The code defines a function to demonstrate the format() method. The terminal window shows the command 'python .\25.py' being executed, resulting in the output: 123, 123.456789, and 1100.

```
functions > 25.py
1 # Python format() function
2
3 print(format(123,"d"))
4
5 print(format(123.456789,"f"))
6
7 print(format(12,"b"))
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Administrator\Desktop\python\functions> python .\25.py
123
123.456789
1100
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.

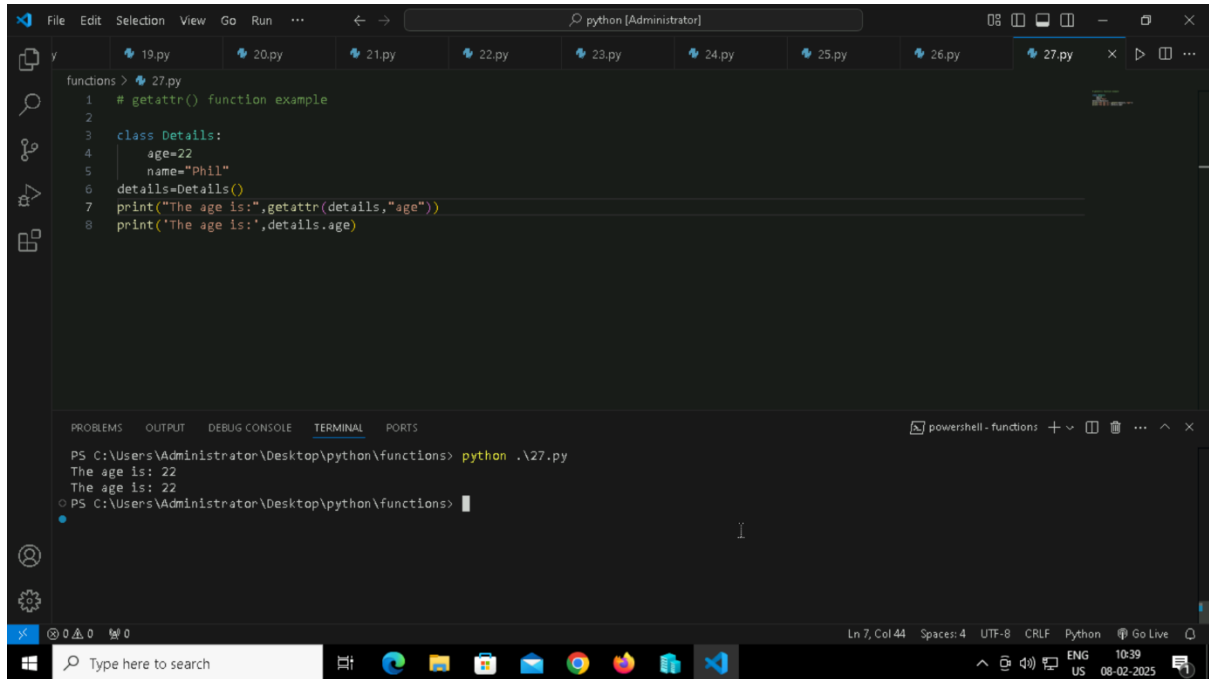


The screenshot shows the Visual Studio Code editor with a Python file named 26.py. The code defines a function to demonstrate the frozenset() method. The terminal window shows the command 'python .\26.py' being executed, resulting in the output: Frozen set is: frozenset({'m', 'o', 's', 't', 'r'}) and Empty frozen set is: frozenset().

```
functions > 26.py
1 # Python frozenset() function
2
3 letters=('m','n','o','t','s')
4
5 fSet=frozenset(letters)
6 print("Frozen set is:",fSet)
7 print("Empty frozen set is:",frozenset())
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Administrator\Desktop\python\functions> python .\26.py
Frozen set is: frozenset({'m', 'o', 's', 't', 'r'})
Empty frozen set is: frozenset()
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.

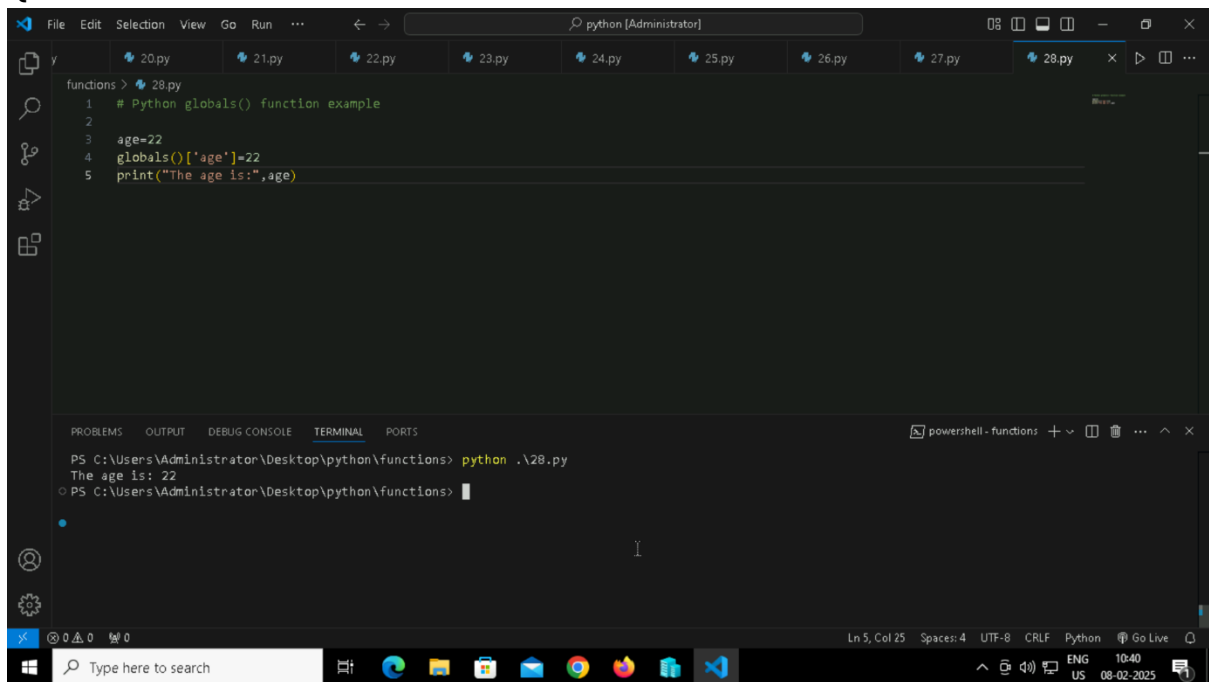


The screenshot shows the Visual Studio Code editor with a Python file named 27.py. The code defines a class Details with an attribute age=22 and a method getattribute(). The terminal output shows the script being executed successfully, printing the age as 22.

```
functions > 27.py
1 # getattribute() function example
2
3 class Details:
4     age=22
5     name="Phil"
6     details=Details()
7     print("The age is:",getattribute(details,"age"))
8     print("The age is:",details.age)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Administrator\Desktop\python\functions> python .\27.py
The age is: 22
The age is: 22
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.



The screenshot shows the Visual Studio Code editor with a Python file named 28.py. The code defines a variable age=22 and a function globals() that returns the current global symbol table. The terminal output shows the script being executed successfully, printing the age as 22.

```
functions > 28.py
1 # Python globals() function example
2
3 age=22
4 globals()['age']=22
5 print("The age is:",age)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Administrator\Desktop\python\functions> python .\28.py
The age is: 22
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.

The screenshot shows the Visual Studio Code editor with a file named 29.py open. The code in the editor is as follows:

```
functions > 29.py
1 # Python iter() function example
2
3 list=[1,2,3,4,5]
4
5 listiter=iter(list)
6
7 print(next(listiter))
8 print(next(listiter))
9 print(next(listiter))
10 print(next(listiter))
11 print(next(listiter))
```

The terminal at the bottom shows the command `python .\29.py` being executed, which results in the output:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\29.py
1
2
3
4
5
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.

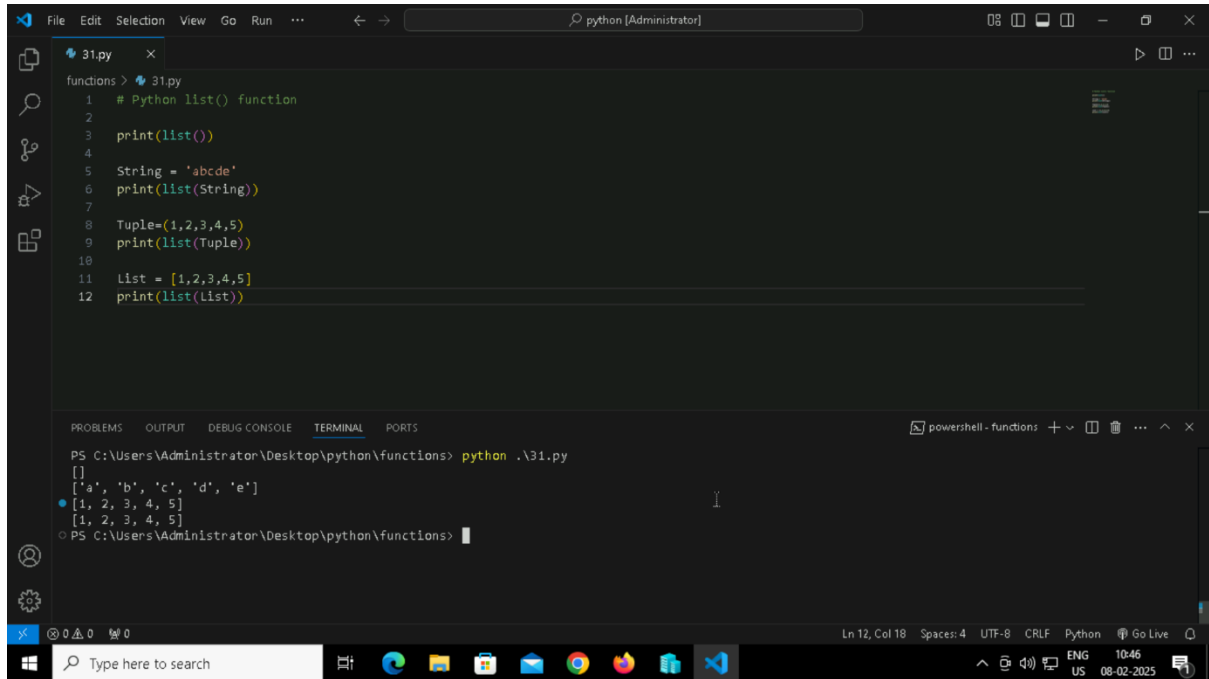
The screenshot shows the Visual Studio Code editor with a file named 30.py open. The code in the editor is as follows:

```
functions > 30.py
1 # Python len() function example
2
3 strA='Python'
4 print(len(strA))
```

The terminal at the bottom shows the command `python .\30.py` being executed, which results in the output:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\30.py
6
PS C:\Users\Administrator\Desktop\python\functions>
```

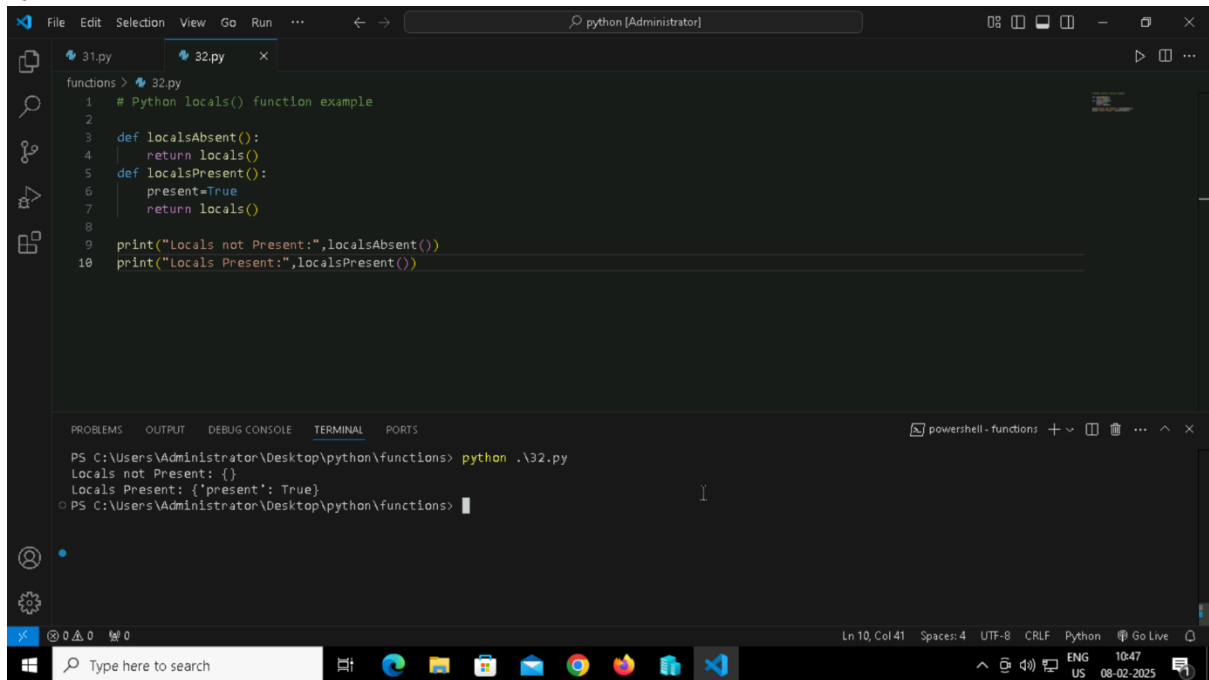
Q.



The screenshot shows the Visual Studio Code editor with a file named `31.py` open. The code defines a function `list()` that prints the contents of various data structures. The terminal output shows the execution of `python .\31.py`, which produces the following output:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\31.py
[]
['a', 'b', 'c', 'd', 'e']
[1, 2, 3, 4, 5]
[1, 2, 3, 4, 5]
```

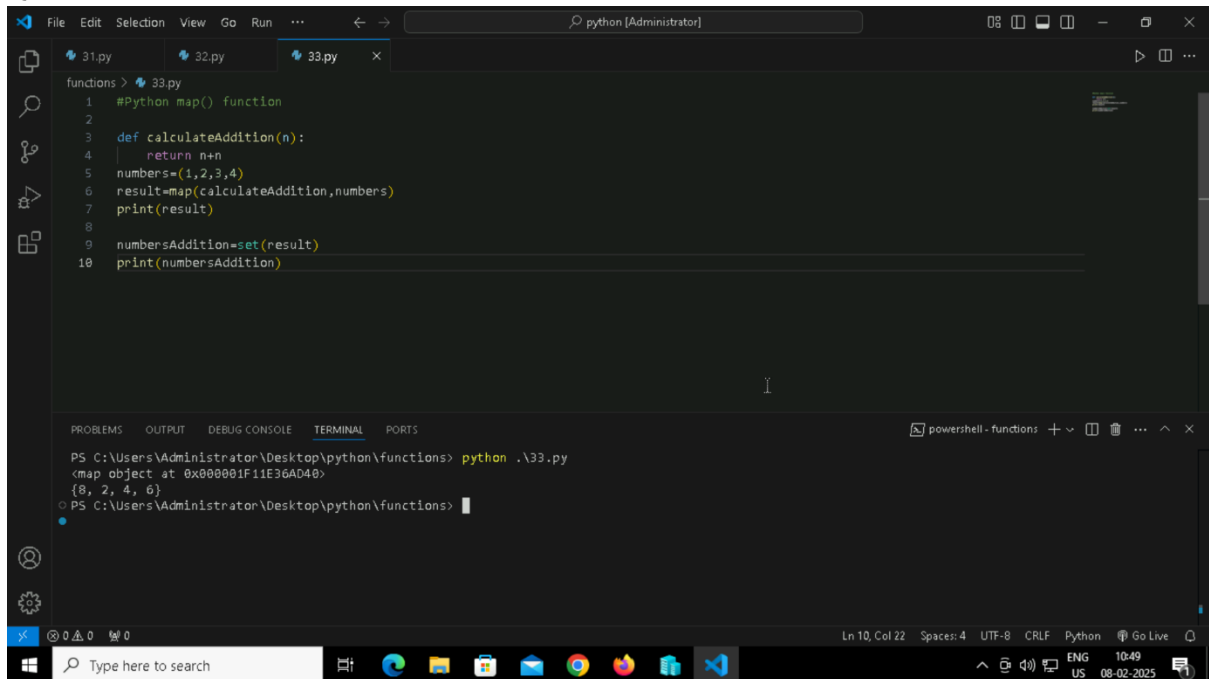
Q.



The screenshot shows the Visual Studio Code editor with a file named `32.py` open. The code defines two functions, `localsAbsent()` and `localsPresent()`, which return the current local namespace. The terminal output shows the execution of `python .\32.py`, which produces the following output:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\32.py
Locals not Present: {}
Locals Present: {'present': True}
```

Q.



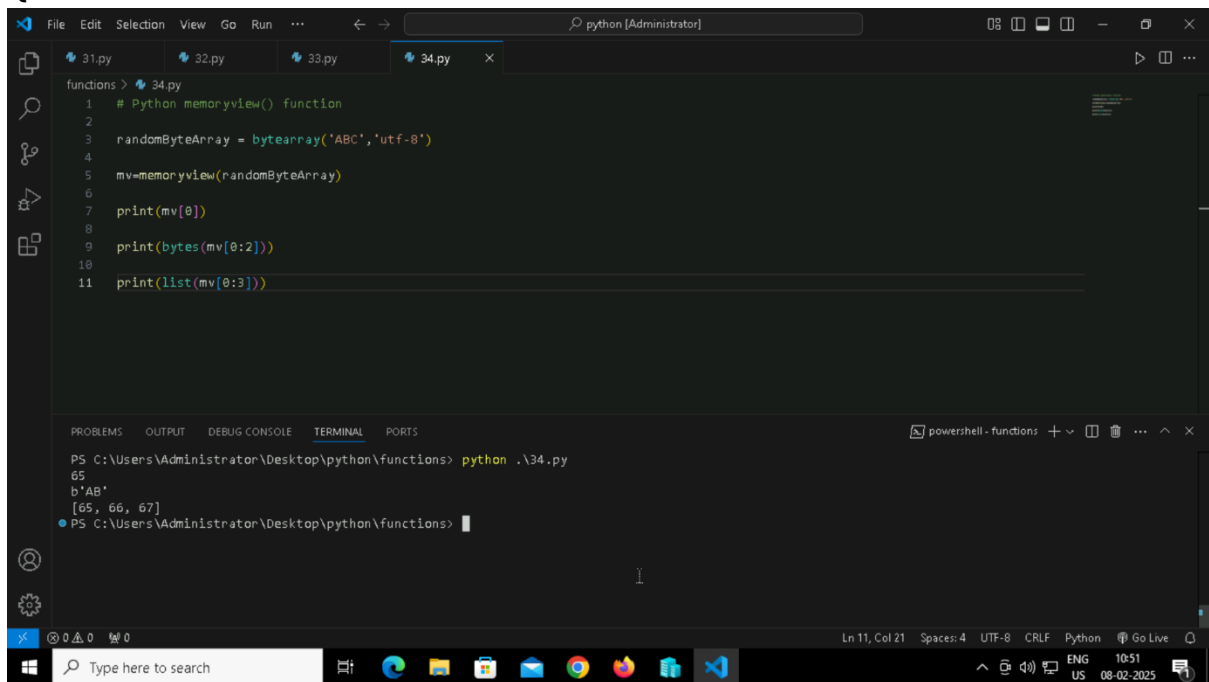
The screenshot shows the Visual Studio Code editor with a file named `33.py` open. The code defines a function `calculateAddition(n)` that returns `n+n`. It then uses `map` to apply this function to the list `(1, 2, 3, 4)`, prints the result, and finally prints the result after it has been converted to a set.

```
1 #Python map() function
2
3 def calculateAddition(n):
4     return n+n
5 numbers=(1,2,3,4)
6 result=map(calculateAddition,numbers)
7 print(result)
8
9 numbersAddition=set(result)
10 print(numbersAddition)
```

The terminal output shows the execution of `python .\33.py`, resulting in a `map` object and its printed values: `{8, 2, 4, 6}`.

```
PS C:\Users\Administrator\Desktop\python\functions> python .\33.py
<map object at 0x000001F1E36AD40>
{8, 2, 4, 6}
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.



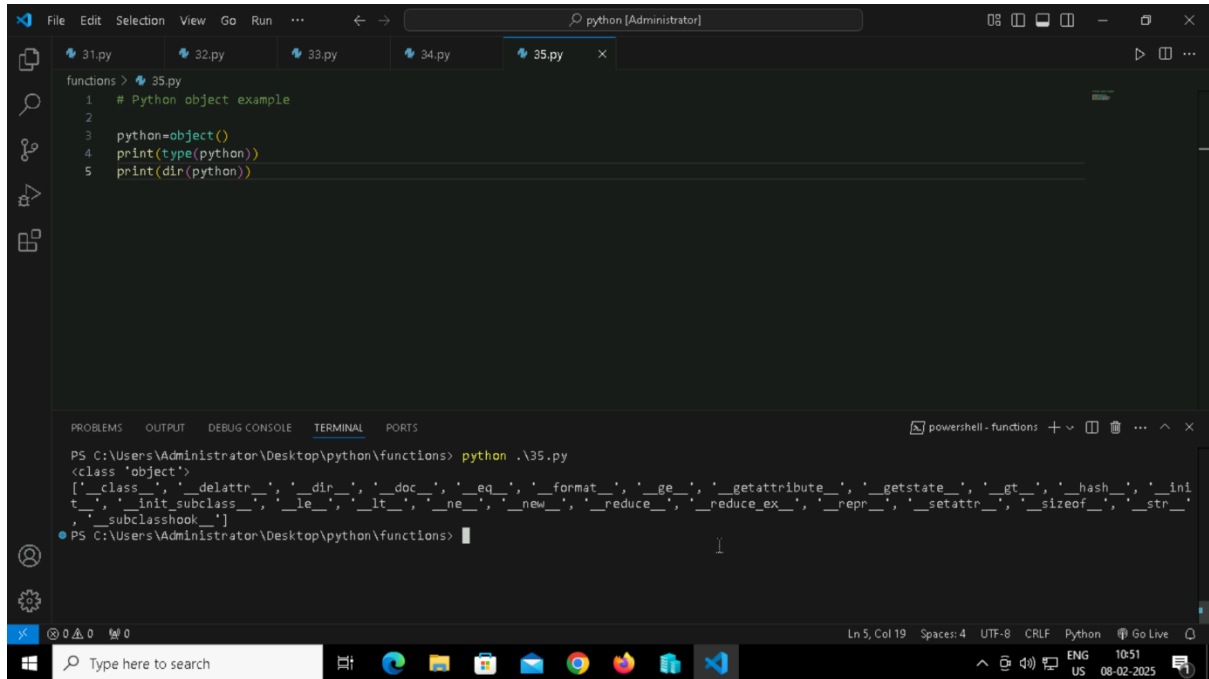
The screenshot shows the Visual Studio Code editor with a file named `34.py` open. The code creates a `bytearray` from the string `'ABC'`, uses `memoryview` to access it, and prints the first byte, a slice of the memoryview, and a list of the slice's values.

```
1 # Python memoryview() function
2
3 randomByteArray = bytearray('ABC','utf-8')
4
5 mv=memoryview(randomByteArray)
6
7 print(mv[0])
8
9 print(bytes(mv[0:2]))
10
11 print(list(mv[0:3]))
```

The terminal output shows the execution of `python .\34.py`, resulting in the following output:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\34.py
65
b'AB'
[65, 66, 67]
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.



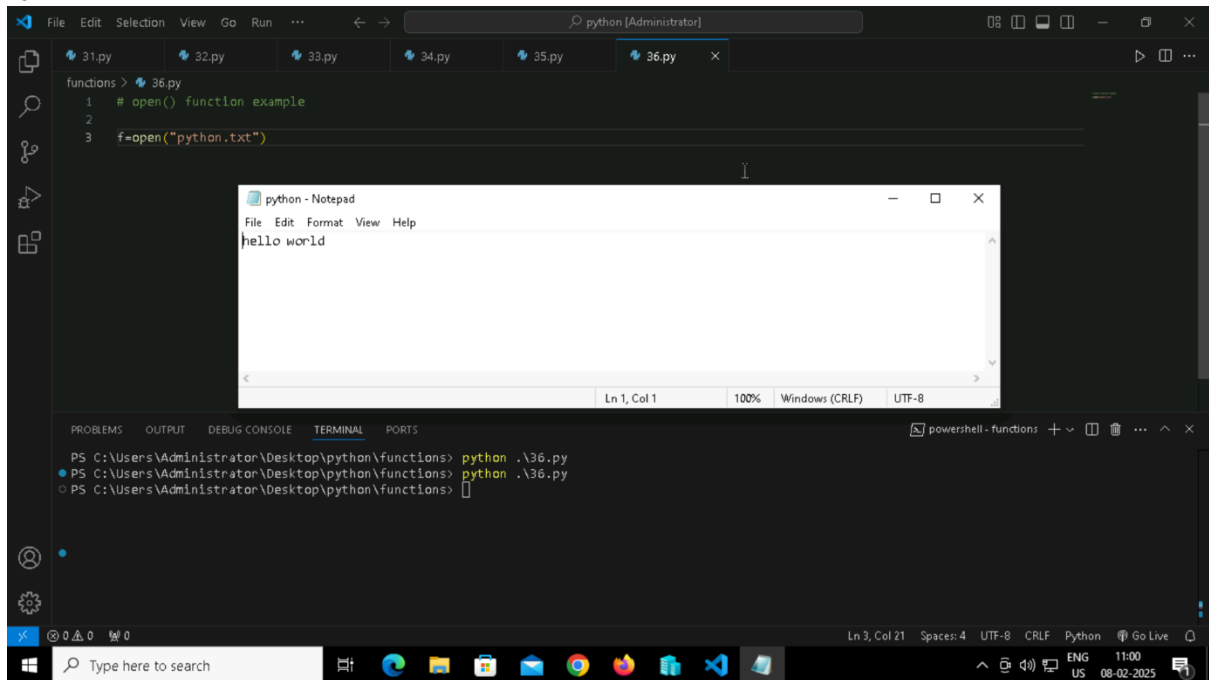
The screenshot shows the Visual Studio Code editor with a file named `35.py` open. The code in the editor is:

```
functions > 35.py
1 # Python object example
2
3 python=object()
4 print(type(python))
5 print(dir(python))
```

The terminal window at the bottom shows the output of running `python .\35.py`:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\35.py
<class 'object'>
['_class_', '_delattr_', '_dir_', '_doc_', '_eq_', '_format_', '_ge_', '_getattr_', '_getstate_', '_gt_', '_hash_', '_init_', '_init_subclass_', '_le_', '_lt_', '_ne_', '_new_', '_reduce_', '_reduce_ex_', '_repr_', '_setattr_', '_sizeof_', '_str_', '_subclasshook_']
```

Q.



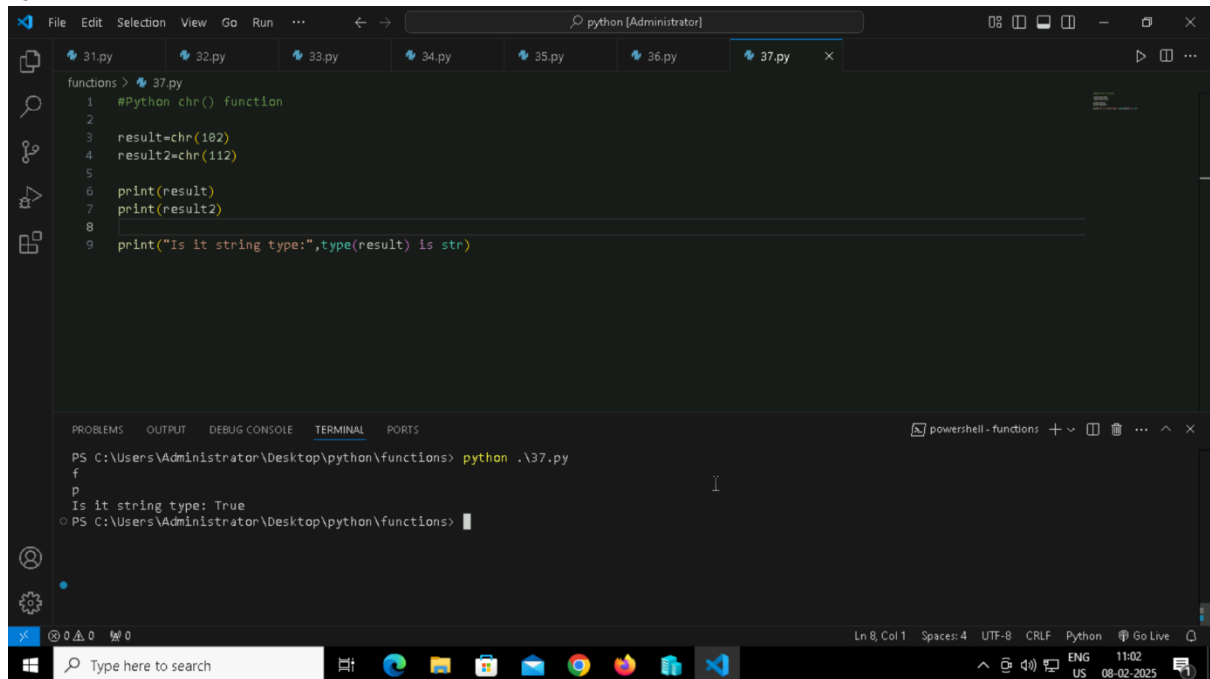
The screenshot shows the Visual Studio Code editor with a file named `36.py` open. The code in the editor is:

```
functions > 36.py
1 # open() function example
2
3 f=open("python.txt")
```

A Notepad window titled "python - Notepad" is open in the foreground, showing the text "hello world". The terminal window at the bottom shows the output of running `python .\36.py`:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\36.py
PS C:\Users\Administrator\Desktop\python\functions> python .\36.py
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.

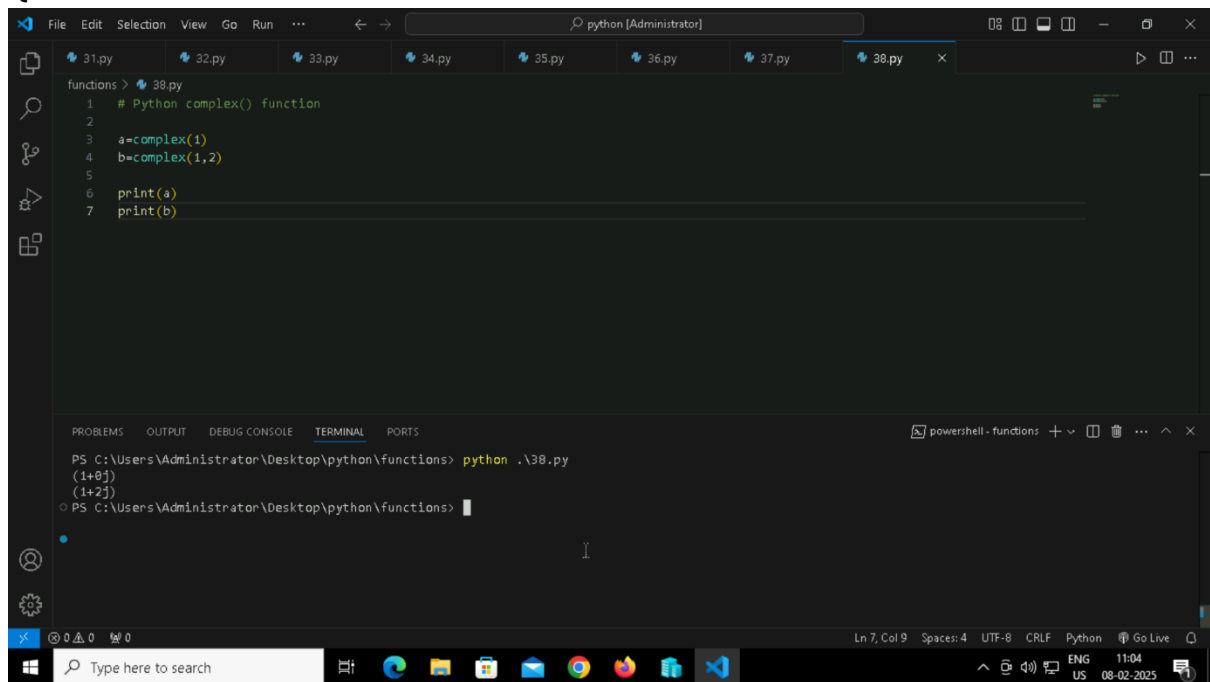


The screenshot shows the Visual Studio Code editor with a Python script named 37.py. The script defines a function that takes two arguments and returns a tuple of their string representations. The terminal shows the output of running the script, which is a tuple of two strings: ('f', 'p').

```
functions > 37.py
1 #Python chr() function
2
3 result=chr(102)
4 result2=chr(112)
5
6 print(result)
7 print(result2)
8
9 print("Is it string type:",type(result) is str)
```

```
PS C:\Users\Administrator\Desktop\python\functions> python .\37.py
f
p
Is it string type: True
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.



The screenshot shows the Visual Studio Code editor with a Python script named 38.py. The script defines a function that takes two arguments and returns a tuple of their complex representations. The terminal shows the output of running the script, which is a tuple of two complex numbers: (1+0j) and (1+2j).

```
functions > 38.py
1 # Python complex() function
2
3 a=complex(1)
4 b=complex(1,2)
5
6 print(a)
7 print(b)
```

```
PS C:\Users\Administrator\Desktop\python\functions> python .\38.py
(1+0j)
(1+2j)
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.

The screenshot shows the Visual Studio Code editor with a Python file named 39.py. The code defines a class `Student` with attributes `id`, `name`, and `email`, and a method `getinfo`. A `delattr` function is called on the `Student` class to remove the `course` attribute. The terminal window shows the execution of the script, which results in an `AttributeError` because the `Student` class does not have a `course` attribute.

```
functions > 39.py
1 # python delattr() function
2 class Student:
3     id=101
4     name="Pranshu"
5     email="pranshu@abc.com"
6     def getinfo(self):
7         print(self.id,self.name,self.email)
8 s=Student()
9 s.getinfo()
10 delattr(Student,'course')
11 s.getinfo()
```

```
PS C:\Users\Administrator\Desktop\python\functions> python .\39.py
101 Pranshu pranshu@abc.com
Traceback (most recent call last):
  File "C:\Users\Administrator\Desktop\python\functions\39.py", line 10, in <module>
    delattr(Student,'course')
AttributeError: type object 'Student' has no attribute 'course'
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.

The screenshot shows the Visual Studio Code editor with a Python file named 40.py. The code uses the `dir()` function to list the attributes of the `__main__` module. The terminal window shows the execution of the script, which outputs the list of attributes: `['__annotations__', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__']`.

```
functions > 40.py
1 # Python dir() function
2
3 att=dir()
4 print(att)
```

```
PS C:\Users\Administrator\Desktop\python\functions> python .\40.py
['__annotations__', '__builtins__', '__cached__', '__doc__', '__file__', '__loader__', '__name__', '__package__', '__spec__']
PS C:\Users\Administrator\Desktop\python\functions>
```


Q.

The screenshot shows the Visual Studio Code editor interface. The top menu bar includes File, Edit, Selection, View, Go, Run, and a search bar. The editor window displays a file named 41.py with the following code:

```
functions > 41.py
1 # Python divmod() function
2
3 result=divmod(10,2)
4 print(result)
```

The bottom panel shows the TERMINAL tab with the following output:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\41.py
(5, 0)
PS C:\Users\Administrator\Desktop\python\functions>
```

The status bar at the bottom indicates the current line and column (Ln 4, Col 13), the file encoding (UTF-8), the line ending (CRLF), the language (Python), and the Go Live status. The Windows taskbar is visible at the bottom of the screen.

Q.

The screenshot shows the Visual Studio Code editor interface. The top menu bar includes File, Edit, Selection, View, Go, Run, and a search bar. The editor window displays a file named 41.py with the following code:

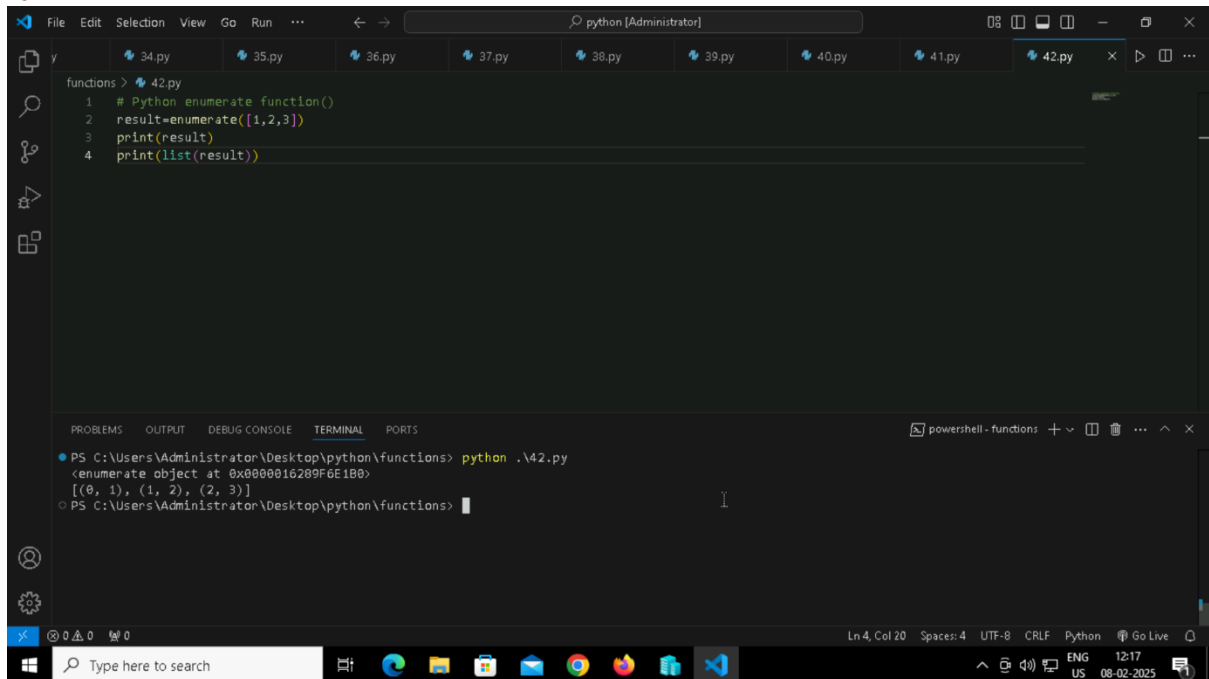
```
functions > 41.py
1 # Python divmod() function
2
3 result=divmod(10,2)
4 print(result)
```

The bottom panel shows the TERMINAL tab with the following output:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\41.py
(5, 0)
PS C:\Users\Administrator\Desktop\python\functions>
```

The status bar at the bottom indicates the current line and column (Ln 4, Col 13), the file encoding (UTF-8), the line ending (CRLF), the language (Python), and the Go Live status. The Windows taskbar is visible at the bottom of the screen.

Q.



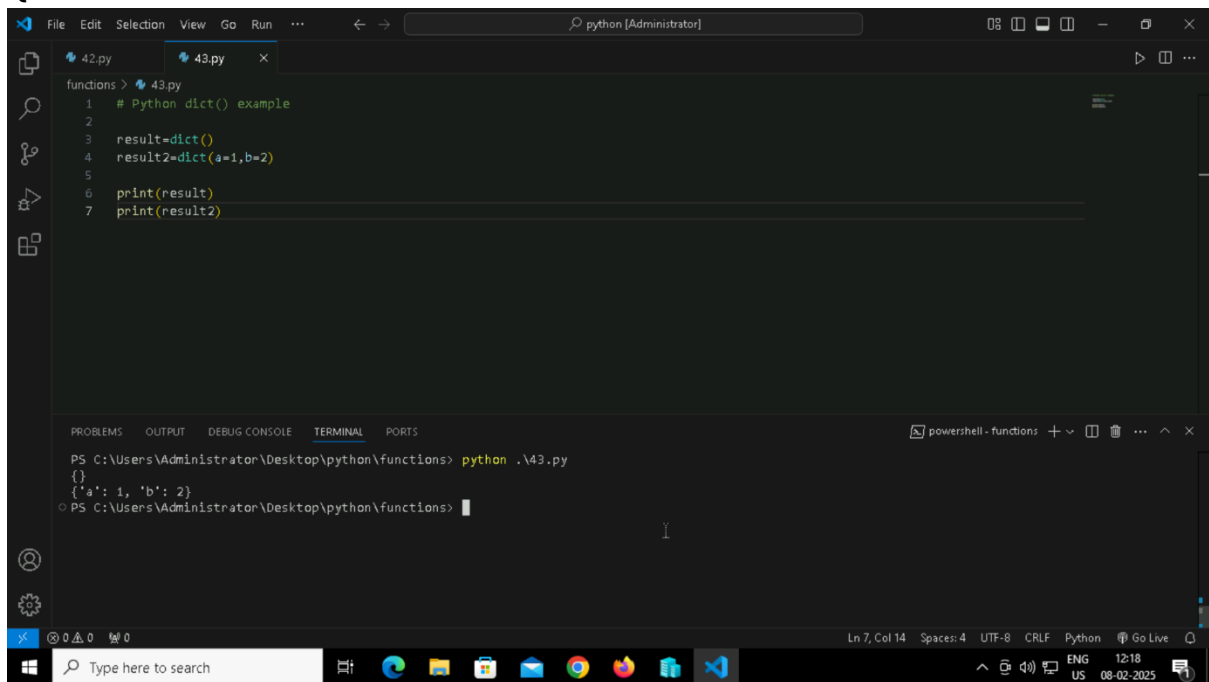
The screenshot shows the Visual Studio Code editor with a file named `42.py` open. The code in the editor is as follows:

```
functions > 42.py
1 # Python enumerate function()
2 result=enumerate([1,2,3])
3 print(result)
4 print(list(result))
```

The terminal at the bottom shows the execution of the script:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\42.py
<enumerate object at 0x0000016289F6E1B0>
[(0, 1), (1, 2), (2, 3)]
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.



The screenshot shows the Visual Studio Code editor with a file named `43.py` open. The code in the editor is as follows:

```
functions > 43.py
1 # Python dict() example
2
3 result=dict()
4 result2=dict(a=1,b=2)
5
6 print(result)
7 print(result2)
```

The terminal at the bottom shows the execution of the script:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\43.py
{}
{'a': 1, 'b': 2}
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.

The screenshot shows the Visual Studio Code editor with a file named `44.py` open. The code defines a function `filterdata(x)` that returns `x` if `x > 5`, and then calls `filter` on a list `[1, 2, 6]` using the function. The terminal shows the command `python .\44.py` being executed, resulting in the output `[6]`.

```
functions > 44.py
1 # Python filter() example
2
3 def filterdata(x):
4     if x > 5:
5         return x
6 result=filter(filterdata,(1,2,6))
7 print(list(result))

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Administrator\Desktop\python\functions> python .\44.py
[6]
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.

The screenshot shows the Visual Studio Code editor with a file named `45.py` open. The code uses the `hash` function to generate hashes for the values `21` and `22.2`, and prints the results. The terminal shows the command `python .\45.py` being executed, resulting in the output `21` and `461168601842737174`.

```
functions > 45.py
1 # Python hash() function example
2
3 result=hash(21)
4 result2=hash(22.2)
5 print(result)
6 print(result2)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Administrator\Desktop\python\functions> python .\45.py
21
461168601842737174
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.

The screenshot shows the Visual Studio Code editor with a Python file named 46.py. The code in the editor is:

```
functions > 46.py
1 # Python help() function
2
3 info=help()
4 print(info)
```

The terminal window at the bottom shows the execution of the script. The command `python .\42.py` was run, resulting in an `enumerate` object. Then, the command `python .\46.py` was run, displaying the Python help utility's welcome message and instructions on how to use the `help()` function.

```
PS C:\Users\Administrator\Desktop\python\functions> python .\42.py
<enumerate object at 0x0000016289F6E1B8>
PS C:\Users\Administrator\Desktop\python\functions> python .\46.py
Welcome to Python 3.13's help utility! If this is your first time using
Python, you should definitely check out the tutorial at
https://docs.python.org/3.13/tutorial/.

Enter the name of any module, keyword, or topic to get help on writing
Python programs and using Python modules. To get a list of available
modules, keywords, symbols, or topics, enter "modules", "keywords",
"symbols", or "topics".
```

Q.

The screenshot shows the Visual Studio Code editor with a Python file named 47.py. The code in the editor is:

```
functions > 47.py
1 # Python min() function
2 small=min(2225,325,2825)
3 small2=min(1808.25,2025.35,5625.36,10052.50)
4 print(small)
5 print(small2)
```

The terminal window at the bottom shows the execution of the script. The command `python .\47.py` was run, resulting in the output of the `min()` function calls: `325` and `1808.25`.

```
PS C:\Users\Administrator\Desktop\python\functions> python .\47.py
325
1808.25
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.

The screenshot shows the Visual Studio Code editor with a file named `48.py` open. The code in the editor is as follows:

```
functions > 48.py
1 # Python set() function
2
3 result=set()
4 result2=set('12')
5 result3=set('javatpoint')
6
7 print(result)
8 print(result2)
9 print(result3)
```

The terminal window at the bottom shows the execution of the script:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\48.py
set()
{'1', '2'}
{'o', 'j', 'i', 'p', 't', 'n', 'a', 'v'}
```

The status bar at the bottom indicates the file is at line 9, column 14, using UTF-8 encoding with CRLF line endings. The system tray shows the date as 08-02-2025 and time as 12:24.

Q.

The screenshot shows the Visual Studio Code editor with a file named `49.py` open. The code in the editor is as follows:

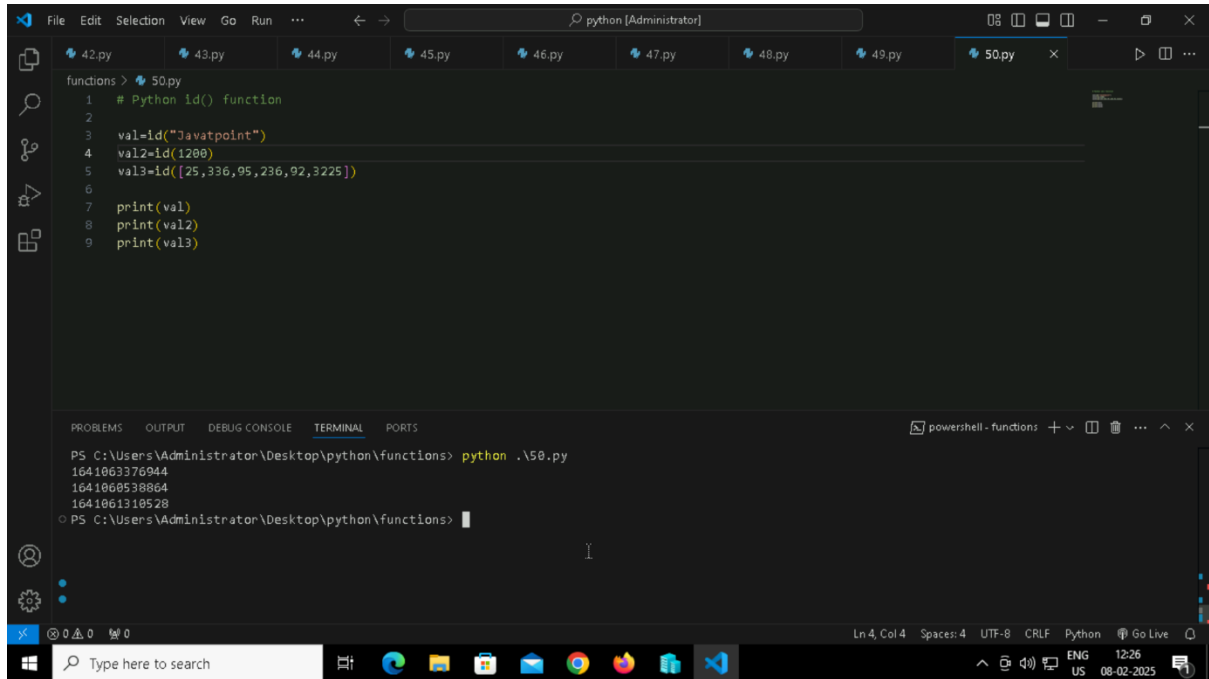
```
functions > 49.py
1 # Python hex() functions
2
3 result=hex(1)
4 result2=hex(342)
5
6 print(result)
7 print(result2)
```

The terminal window at the bottom shows the execution of the script:

```
PS C:\Users\Administrator\Desktop\python\functions> python .\49.py
0x1
0x156
```

The status bar at the bottom indicates the file is at line 7, column 14, using UTF-8 encoding with CRLF line endings. The system tray shows the date as 08-02-2025 and time as 12:25.

Q.

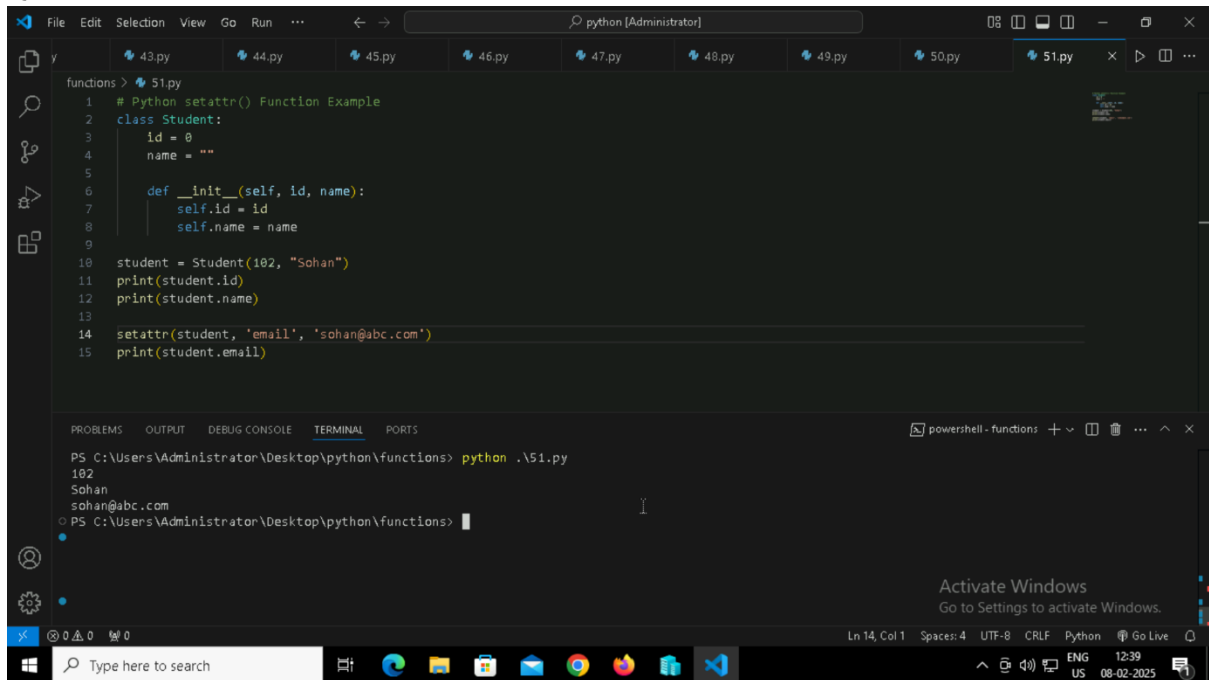


The screenshot shows the Visual Studio Code editor with a file named 50.py open. The code defines a function to demonstrate the id() function. The terminal window shows the execution of the script, displaying the memory addresses for the variables val, val2, and val3.

```
functions > 50.py
1 # Python id() function
2
3 val=id("Javatpoint")
4 val2=id(1200)
5 val3=id([25,336,95,236,92,3225])
6
7 print(val)
8 print(val2)
9 print(val3)
```

```
PS C:\Users\Administrator\Desktop\python\functions> python .\50.py
1641063376944
1641060538864
1641061310528
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.



The screenshot shows the Visual Studio Code editor with a file named 51.py open. The code defines a class Student and creates an instance of it. The terminal window shows the execution of the script, displaying the attributes of the student object.

```
functions > 51.py
1 # Python setattr() Function Example
2 class Student:
3     id = 0
4     name = ""
5
6     def __init__(self, id, name):
7         self.id = id
8         self.name = name
9
10 student = Student(102, "Sohan")
11 print(student.id)
12 print(student.name)
13
14 setattr(student, 'email', 'sohan@abc.com')
15 print(student.email)
```

```
PS C:\Users\Administrator\Desktop\python\functions> python .\51.py
102
Sohan
sohan@abc.com
PS C:\Users\Administrator\Desktop\python\functions>
```

Q.

The screenshot shows the Visual Studio Code editor with a Python file named 52.py. The code defines a function slice() that returns a slice object. The output window shows the execution results.

```
functions > 52.py
1 # Python slice() Function Example
2 result = slice(5) # Returns slice object
3 result2 = slice(0, 5, 3) # Returns slice object
4 print(result)
5 print(result2)
```

OUTPUT

```
[Done] exited with code=0 in 0.101 seconds

[Running] python -u "c:\Users\Administrator\Desktop\python\functions\52.py"
slice(None, 5, None)
slice(0, 5, 3)

[Done] exited with code=0 in 0.172 seconds
```

Q.

The screenshot shows the Visual Studio Code editor with a Python file named 53.py. The code defines a function sorted() that returns a sorted list. The output window shows the execution results.

```
EXPLORER
PYTHON
  functions
    42.py
    43.py
    44.py
    45.py
    46.py
    47.py
    48.py
    49.py
    50.py
    51.py
    52.py
    53.py
  python.txt
  if_else
  loops
  reverse_string
  string
  eight.py
  five.py
  four.py
  one.py
  seven.py
  OUTLINE
  TIMELINE
```

```
functions > 53.py
1 # Python sorted() Function Example
2 str_val = "javatpoint"
3 sorted1 = sorted(str_val)
4 print(sorted1)
```

OUTPUT

```
slice(0, 5, 3)

[Done] exited with code=0 in 0.172 seconds

[Running] python -u "c:\Users\Administrator\Desktop\python\functions\53.py"
['a', 'a', 'i', 'j', 'n', 'o', 'p', 't', 't', 'v']

[Done] exited with code=0 in 0.09 seconds
```

Q.

The screenshot shows the Visual Studio Code editor with a file named 54.py open. The code in the editor is as follows:

```
1 # Python next() Function Example
2 number = iter([256, 32, 82])
3 print(next(number))
4 print(next(number))
5 print(next(number))
```

The bottom panel shows the OUTPUT window with the following text:

```
[Done] exited with code=0 in 0.09 seconds

[Running] python -u "c:\Users\Administrator\Desktop\python\functions\54.py"
256
32
82

[Done] exited with code=0 in 0.096 seconds
```

The status bar at the bottom indicates the cursor is at Line 5, Column 20, with 4 spaces, UTF-8 encoding, CRLF line endings, and the Python interpreter is selected.

Q.

The screenshot shows the Visual Studio Code editor with a file named 55.py open. The code in the editor is as follows:

```
1 # Python int() Function Example
2 val = int(10)
3 val2 = int(10.52)
4 val3 = int('10')
5 print("integer values:", val, val2, val3)
```

The bottom panel shows the OUTPUT window with the following text:

```
82

[Done] exited with code=0 in 0.096 seconds

[Running] python -u "c:\Users\Administrator\Desktop\python\functions\55.py"
integer values: 10 10 10

[Done] exited with code=0 in 0.09 seconds
```

The status bar at the bottom indicates the cursor is at Line 5, Column 42, with 4 spaces, UTF-8 encoding, CRLF line endings, and the Python interpreter is selected.

Q.

The screenshot shows the Visual Studio Code editor with a Python file named 56.py. The code defines a class Student and uses isinstance() to check if a variable is an instance of the class. The terminal output shows the execution results.

```
1 # Python isinstance() Function Example
2 class Student:
3     id = 101
4     name = "John"
5
6     def __init__(self, id, name):
7         self.id = id
8         self.name = name
9
10 student = Student(1010, "John")
11 lst = [12, 34, 5, 6, 767]
12 print(isinstance(student, Student))
13 print(isinstance(lst, Student))
```

Terminal Output:

```
[Done] exited with code=0 in 0.09 seconds

[Running] python -u "c:\Users\Administrator\Desktop\python\functions\56.py"
True
False

[Done] exited with code=0 in 0.092 seconds
```

Q.

The screenshot shows the Visual Studio Code editor with a Python file named 57.py. The code uses the oct() function to convert a decimal number to its octal representation. The terminal output shows the execution results.

```
1 # Python oct() Function Example
2 val = oct(10)
3 print("Octal value of 10:", val)
```

Terminal Output:

```
False

[Done] exited with code=0 in 0.092 seconds

[Running] python -u "c:\Users\Administrator\Desktop\python\functions\57.py"
Octal value of 10: 0o12

[Done] exited with code=0 in 0.122 seconds
```

Q.

The screenshot shows the Visual Studio Code editor with a Python file named 58.py. The code defines a function `and()` that takes two arguments and returns their logical AND. The file explorer on the left shows a directory structure with files 47.py through 58.py. The output console at the bottom shows the execution of the script, which prints the values 56, 82, and 38.

```
1 # Python and() Function Example
2 print(and('8'))
3 print(and('R'))
4 print(and('&'))
```

[Done] exited with code=0 in 0.122 seconds

[Running] python -u "c:\Users\Administrator\Desktop\python\functions\58.py"

56
82
38

[Done] exited with code=0 in 0.252 seconds

Q.

The screenshot shows the Visual Studio Code editor with a Python file named 59.py. The code defines a function `pow()` that takes two arguments and returns their power. The file explorer on the left shows a directory structure with files 48.py through 59.py. The output console at the bottom shows the execution of the script, which prints the values 16, 16, 0.0625, and 0.0625.

```
1 # Python pow() Function Example
2 print(pow(4, 2))
3 print(pow(-4, 2))
4 print(pow(4, -2))
5 print(pow(-4, -2))
```

[Running] python -u "c:\Users\Administrator\Desktop\python\functions\59.py"

16
16
0.0625
0.0625

[Done] exited with code=0 in 0.12 seconds

Q.

```
File Edit Selection View Go Run ... python [Administrator]
```

EXPLORER

- PYTHON
 - functions
 - 49.py
 - 50.py
 - 51.py
 - 52.py
 - 53.py
 - 54.py
 - 55.py
 - 56.py
 - 57.py
 - 58.py
 - 59.py
 - 60.py
 - python.bt
 - if_else
 - loops
 - reverse_string
 - string
 - eight.py
 - five.py
 - four.py
 - one.py
 - seven.py
 - OUTLINE
 - TIMELINE

functions > 60.py

```
1 # Python print() Function Example
2 print("Python is a programming language.")
3 x = 7
4 print("x =", x)
5 y = x
6 print('x =', x, 'y =', y)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

[Done] exited with code=0 in 0.12 seconds

[Running] python -u "c:\Users\Administrator\Desktop\python\functions\60.py"

Python is a programming language.
x = 7
x = 7 = y

[Done] exited with code=0 in 0.127 seconds

Activate Windows
Go to Settings to activate Windows.

Ln 6, Col 23 Spaces: 4 UTF-8 CRLF Python Go Live

Type here to search

Q.

```
File Edit Selection View Go Run ... python [Administrator]
```

EXPLORER

- PYTHON
 - functions
 - 50.py
 - 51.py
 - 52.py
 - 53.py
 - 54.py
 - 55.py
 - 56.py
 - 57.py
 - 58.py
 - 59.py
 - 60.py
 - 61.py
 - python.bt
 - if_else
 - loops
 - reverse_string
 - string
 - eight.py
 - five.py
 - four.py
 - one.py
 - seven.py
 - six.py
 - OUTLINE
 - TIMELINE

functions > 61.py

```
1 # Python range() Function Example
2 print(list(range(0)))
3 print(list(range(4)))
4 print(list(range(1, 7)))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

[Done] exited with code=0 in 0.127 seconds

[Running] python -u "c:\Users\Administrator\Desktop\python\functions\61.py"

[]
[0, 1, 2, 3]
[1, 2, 3, 4, 5, 6]

[Done] exited with code=0 in 0.251 seconds

Activate Windows
Go to Settings to activate Windows.

Ln 4, Col 25 Spaces: 4 UTF-8 CRLF Python Go Live

Type here to search

Q.

The screenshot shows the Visual Studio Code editor with a Python file named 62.py. The code defines a function `reversed()` and uses it on a string, a tuple, a range, and a list. The output window shows the results of these operations.

```
functions > 62.py
1 # Python reversed() Function Example
2 String = 'Java'
3 print(list(reversed(String)))
4 Tuple = ('J', 'a', 'v', 'a')
5 print(list(reversed(Tuple)))
6 Range = range(8, 12)
7 print(list(reversed(Range)))
8 List = [1, 2, 7, 5]
9 print(list(reversed(List)))
```

OUTPUT

```
[Running] python -u "c:\Users\Administrator\Desktop\python\functions\62.py"
['a', 'v', 'a', 'J']
['a', 'v', 'a', 'J']
[11, 10, 9, 8]
[5, 7, 2, 1]
[Done] exited with code=0 in 0.114 seconds
```

Q.

The screenshot shows the Visual Studio Code editor with a Python file named 63.py. The code uses the `round()` function on various numbers. The output window shows the results of these operations.

```
functions > 63.py
1 # Python round() Function Example
2 print(round(10))
3 print(round(10.8))
4 print(round(6.6))
```

OUTPUT

```
[Done] exited with code=0 in 0.114 seconds

[Running] python -u "c:\Users\Administrator\Desktop\python\functions\63.py"
10
11
7
[Done] exited with code=0 in 0.161 seconds
```

Q.

The screenshot shows the Visual Studio Code editor with a Python script named 64.py. The script defines a Rectangle class and a Square subclass, then tests issubclass() with various arguments. The output window shows the results of the execution.

```
1 # Python issubclass() Function Example
2 class Rectangle:
3     def __init__(self, rectangleType):
4         print('Rectangle is a', rectangleType)
5
6 class Square(Rectangle):
7     def __init__(self):
8         Rectangle.__init__(self, 'square')
9
10 print(issubclass(Square, Rectangle))
11 print(issubclass(Square, list))
12 print(issubclass(Square, (list, Rectangle)))
13 print(issubclass(Rectangle, (list, Rectangle)))
```

OUTPUT:

```
[Running] python -u "c:\Users\Administrator\Desktop\python\functions\64.py"
True
False
True
True

[Done] exited with code=0 in 0.126 seconds
```

Q.

The screenshot shows the Visual Studio Code editor with a Python script named 65.py. The script tests the str() function with the argument 4. The output window shows the result of the execution.

```
1 # Python str() Function Example
2 print(str(4))
```

OUTPUT:

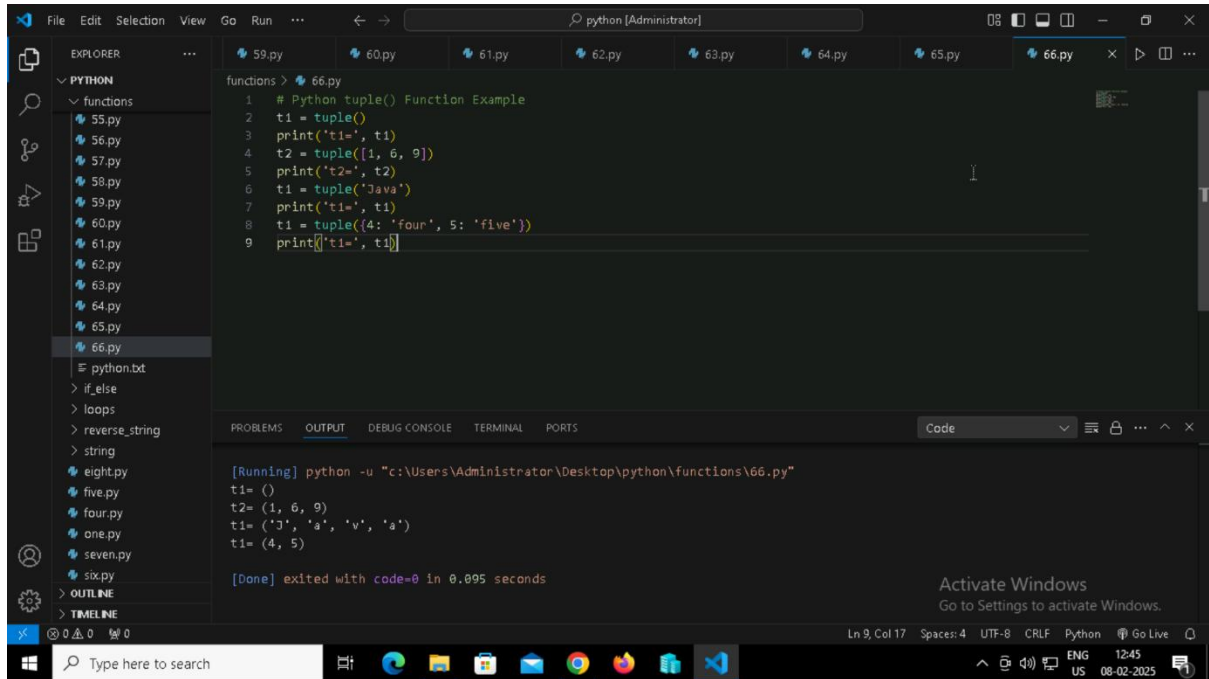
```
True

[Done] exited with code=0 in 0.126 seconds

[Running] python -u "c:\Users\Administrator\Desktop\python\functions\65.py"
4

[Done] exited with code=0 in 0.116 seconds
```

Q.



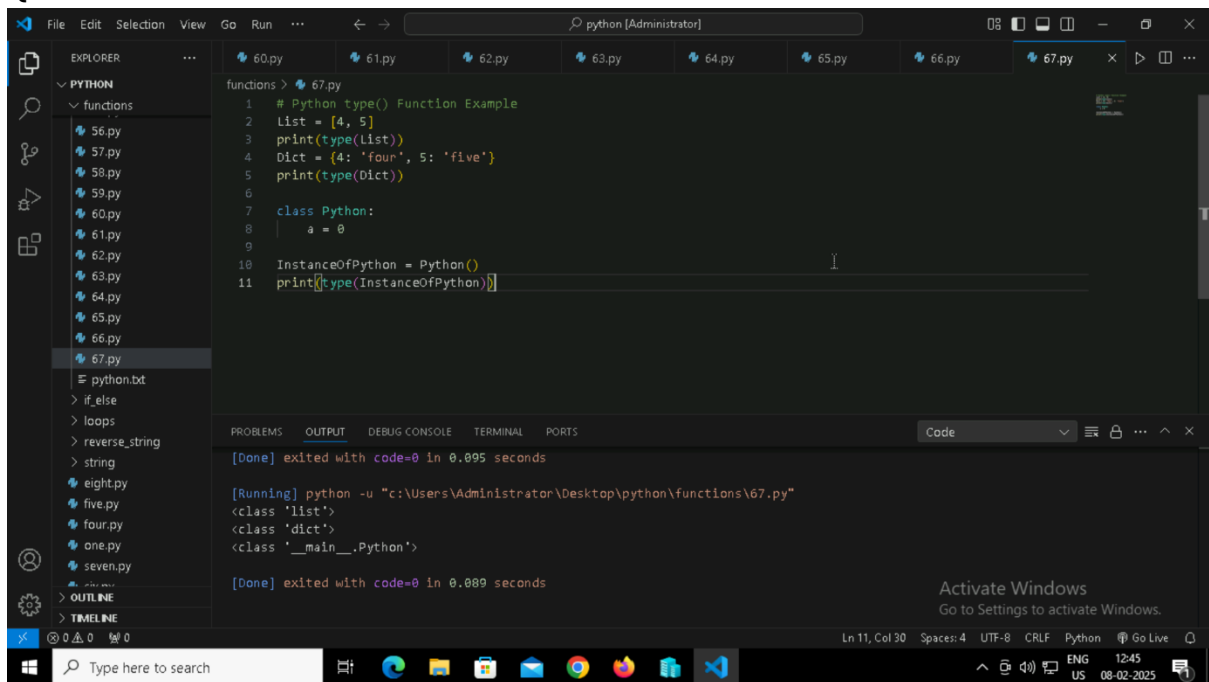
The screenshot shows the Visual Studio Code editor with a Python file named 66.py. The code defines a function `tuple()` and uses it to create two tuples, `t1` and `t2`. The output window shows the execution results.

```
1 # Python tuple() Function Example
2 t1 = tuple()
3 print('t1=', t1)
4 t2 = tuple([1, 6, 9])
5 print('t2=', t2)
6 t1 = tuple('Java')
7 print('t1=', t1)
8 t1 = tuple({4: 'four', 5: 'five'})
9 print('t1=', t1)
```

Output:

```
[Running] python -u "c:\Users\Administrator\Desktop\python\functions\66.py"
t1= ()
t2= (1, 6, 9)
t1= ('J', 'a', 'v', 'a')
t1= (4, 5)
[Done] exited with code=0 in 0.095 seconds
```

Q.



The screenshot shows the Visual Studio Code editor with a Python file named 67.py. The code creates a list, a dict, and a class, and prints their types. The output window shows the execution results.

```
1 # Python type() Function Example
2 List = [4, 5]
3 print(type(List))
4 Dict = {4: 'four', 5: 'five'}
5 print(type(Dict))
6
7 class Python:
8     a = 0
9
10 InstanceOfPython = Python()
11 print(type(InstanceOfPython))
```

Output:

```
[Done] exited with code=0 in 0.095 seconds

[Running] python -u "c:\Users\Administrator\Desktop\python\functions\67.py"
<class 'list'>
<class 'dict'>
<class '__main__.Python'>
[Done] exited with code=0 in 0.089 seconds
```

Q.

The screenshot shows the Visual Studio Code editor with a Python file named 68.py. The code defines a class Python with an __init__ method that takes x and y parameters. An instance of the class is created and its variables are printed. The output window shows the execution results.

```
functions > 68.py
1 # Python vars() Function Example
2 class Python:
3     def __init__(self, x=7, y=9):
4         self.x = x
5         self.y = y
6
7 InstanceOfPython = Python()
8 print(vars(InstanceOfPython))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Code

```
<class '__main__.Python'>

[Done] exited with code=0 in 0.089 seconds

[Running] python -u "c:\Users\Administrator\Desktop\python\functions\68.py"
{'x': 7, 'y': 9}

[Done] exited with code=0 in 0.097 seconds
```

Activate Windows
Go to Settings to activate Windows.

Q.

The screenshot shows the Visual Studio Code editor with a Python file named 69.py. The code demonstrates the use of the zip() function to combine a list of numbers and a list of strings into a list of tuples. The output window shows the execution results.

```
functions > 69.py
1 # Python zip() Function Example
2 numList = [4, 5, 6]
3 strList = ['four', 'five', 'six']
4
5 result = zip()
6 resultList = list(result)
7 print(resultList)
8
9 result = zip(numList, strList)
10 resultSet = set(result)
11 print(resultSet)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Code

```
[Done] exited with code=0 in 0.087 seconds

[Running] python -u "c:\Users\Administrator\Desktop\python\functions\69.py"
[]
{(6, 'six'), (5, 'five'), (4, 'four')}
```

[Done] exited with code=0 in 0.084 seconds

Activate Windows
Go to Settings to activate Windows.

Q.

The screenshot shows the Visual Studio Code editor with a Python file named 70.py. The code defines a lambda function 'add' and a function 'a' that uses the lambda function. The output window shows the execution results.

```
functions > 70.py
1 # Python Lambda Functions Example
2 add = lambda num: num + 4
3 print(add(6))
4
5 a = lambda x, y: (x * y)
6 print(a(4, 5))
7
8 a = lambda x, y, z: (x + y + z)
9 print(a(4, 5, 5))
```

OUTPUT

```
[Done] exited with code=0 in 0.084 seconds

[Running] python -u "c:\Users\Administrator\Desktop\python\functions\70.py"
10
20
14

[Done] exited with code=0 in 0.117 seconds
```

Activate Windows
Go to Settings to activate Windows.

Q.

The screenshot shows the Visual Studio Code editor with a Python file named 71.py. The code defines a regular function 'reciprocal' and a lambda function 'lambda_reciprocal'. The output window shows the execution results.

```
functions > 71.py
1 # Python lambda vs def Example
2 def reciprocal(num):
3     return 1 / num
4
5 lambda_reciprocal = lambda num: 1 / num
6
7 print("Def keyword:", reciprocal(6))
8 print("Lambda keyword:", lambda_reciprocal(6))
```

OUTPUT

```
[Done] exited with code=0 in 0.117 seconds

[Running] python -u "c:\Users\Administrator\Desktop\python\functions\71.py"
Def keyword: 0.16666666666666666
Lambda keyword: 0.16666666666666666

[Done] exited with code=0 in 0.091 seconds
```

Activate Windows
Go to Settings to activate Windows.

Q.

The screenshot shows the Visual Studio Code interface with a Python file named 72.py open. The code in the editor is as follows:

```
1 # Using Lambda Function with filter()
2 list_ = [35, 12, 69, 55, 75, 14, 73]
3 odd_list = list(filter(lambda num: (num % 2 != 0), list_))
4 print("The list of odd numbers is:", odd_list)
```

The Explorer sidebar on the left shows a project structure with a 'PYTHON' folder containing several files, including 72.py which is currently selected. The Output window at the bottom displays the execution results:

```
Lambda keyword: 0.16666666666666666
[Done] exited with code=0 in 0.091 seconds
[Running] python -u "c:\Users\Administrator\Desktop\python\functions\72.py"
The list of odd numbers is: [35, 69, 55, 75, 73]
[Done] exited with code=0 in 0.124 seconds
```

The status bar at the bottom indicates the current cursor position is at Line 4, Column 47, with 4 spaces, in UTF-8 encoding, CRLF line endings, and Python language mode.

Q.

The screenshot shows the Visual Studio Code interface with a Python file named 73.py open. The code in the editor is as follows:

```
1 # Using Lambda Function with map()
2 num_list = [1, 2, 3, 4, 5]
3 squared_list = list(map(lambda x: x ** 2, num_list))
4 print("Squared List:", squared_list)
```

The Explorer sidebar on the left shows the same project structure as the previous screenshot, with 73.py now selected. The Output window at the bottom displays the execution results:

```
The list of odd numbers is: [35, 69, 55, 75, 73]
[Done] exited with code=0 in 0.124 seconds
[Running] python -u "c:\Users\Administrator\Desktop\python\functions\73.py"
Squared List: [1, 4, 9, 16, 25]
[Done] exited with code=0 in 0.121 seconds
```

The status bar at the bottom indicates the current cursor position is at Line 4, Column 37, with 4 spaces, in UTF-8 encoding, CRLF line endings, and Python language mode.

Q.

The screenshot shows the Visual Studio Code editor with a Python file named 74.py. The code defines a lambda function for squaring numbers and prints the results for numbers 0 to 10. The terminal output shows the execution of the script, displaying the square values for each number from 0 to 10.

```
functions > 74.py
1 # Lambda function with list comprehension
2 squares=[lambda num:num**2 for num in range(0,11)]
3
4 for square in squares:
5     print("The square value of all numbers from 0 to 10:",square(),end=" ")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Code

```
[Done] exited with code=0 in 0.113 seconds
```

```
[Running] python -u "c:\Users\Administrator\Desktop\python\functions\74.py"
The square value of all numbers from 0 to 10: 0 The square value of all numbers from 0 to 10: 1 The square value of all numbers from 0 to 10: 4 The
square value of all numbers from 0 to 10: 9 The square value of all numbers from 0 to 10: 16 The square value of all numbers from 0 to 10: 25 The
square value of all numbers from 0 to 10: 36 The square value of all numbers from 0 to 10: 49 The square value of all numbers from 0 to 10: 64 The
square value of all numbers from 0 to 10: 81 The square value of all numbers from 0 to 10: 100
[Done] exited with code=0 in 0.094 seconds
```

Activate Windows
Go to Settings to activate Windows.

Ln 5, Col 74 Spaces: 4 UTF-8 CRLF Python Go Live

Q.

The screenshot shows the Visual Studio Code editor with a Python file named 75.py. The code defines a lambda function for finding the minimum of two numbers and prints the result for the numbers 35 and 74. The terminal output shows the execution of the script, displaying the greater number (35).

```
functions > 75.py
1 # Lambda function with if-else
2
3 Minimum= lambda x,y:x if(x<y) else y
4 print("The greater number is:",Minimum(35,74))
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS Code

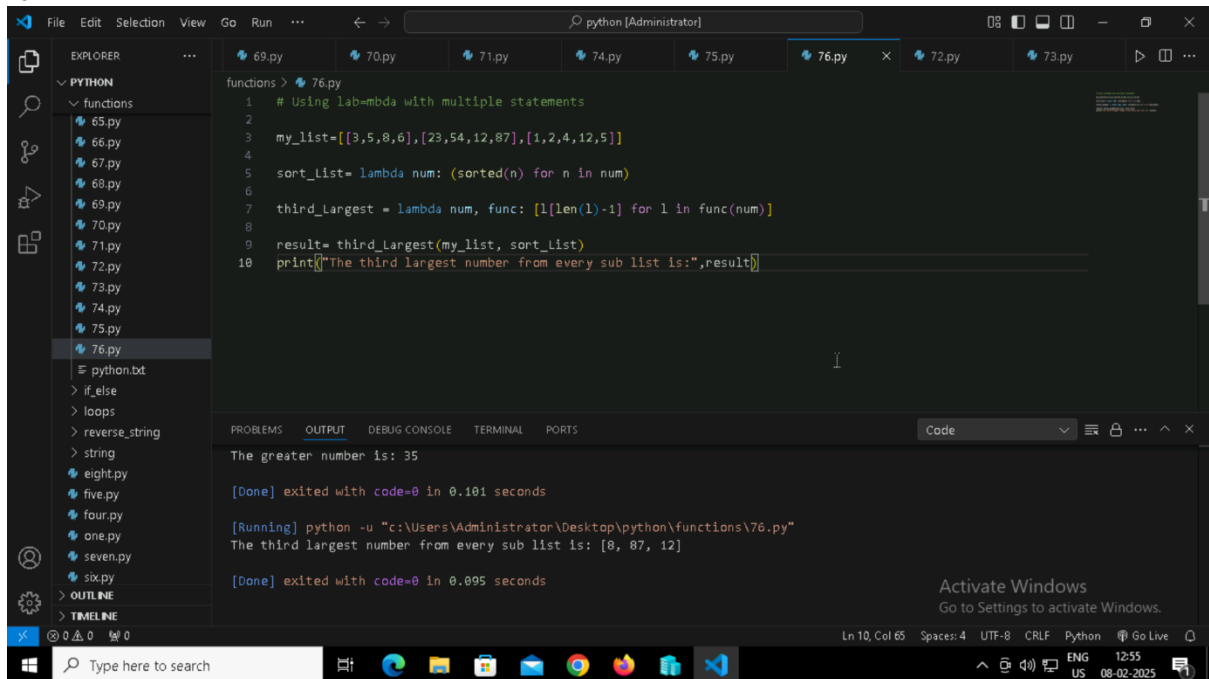
```
to 10: 49 The square value of all numbers from 0 to 10: 64 The square value of all numbers from 0 to 10: 81 The square value of
all numbers from 0 to 10: 100
[Done] exited with code=0 in 0.094 seconds
```

```
[Running] python -u "c:\Users\Administrator\Desktop\python\functions\75.py"
The greater number is: 35
[Done] exited with code=0 in 0.101 seconds
```

Activate Windows
Go to Settings to activate Windows.

Ln 4, Col 45 Spaces: 4 UTF-8 CRLF Python Go Live

Q.



The screenshot shows the Visual Studio Code editor with a Python file named 76.py. The code defines a list of lists, a lambda function to sort each sub-list, another lambda function to find the third largest element in a sorted list, and a function to apply this to all sub-lists. The output shows the third largest number from every sub-list is [8, 87, 12].

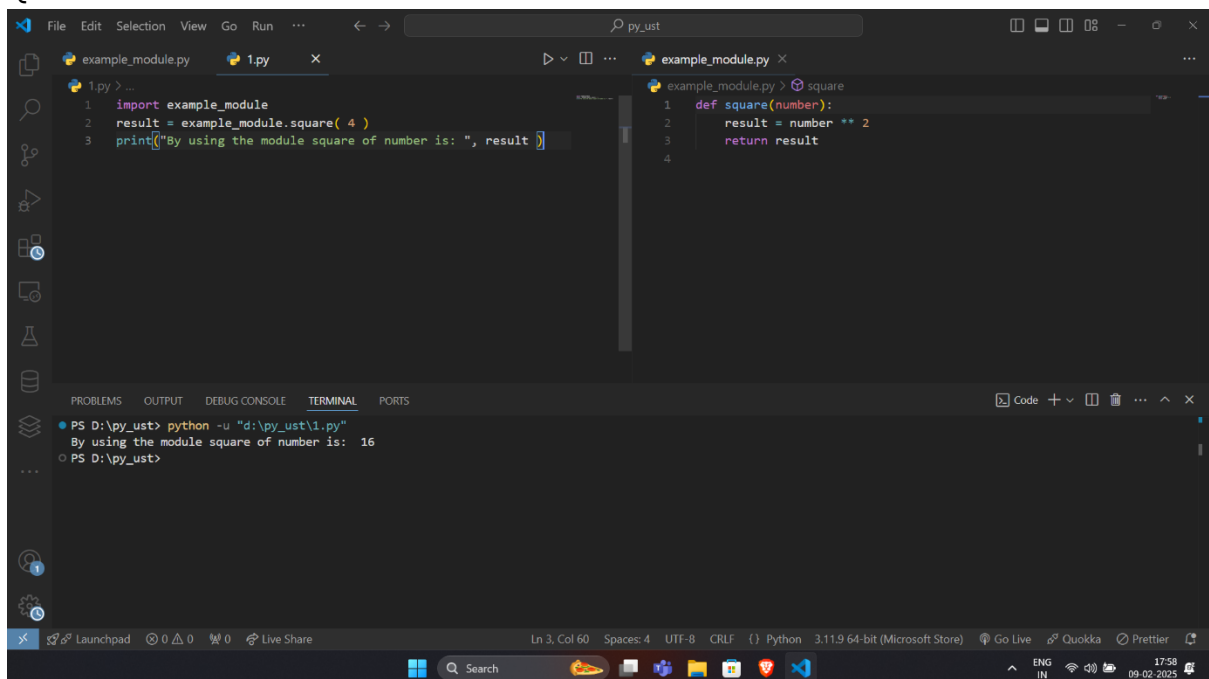
```
1 # Using lab=mbda with multiple statements
2
3 my_list=[[3,5,8,6],[23,54,12,87],[1,2,4,12,5]]
4
5 sort_list= lambda num: (sorted(n) for n in num)
6
7 third_Largest = lambda num, func: [l[len(l)-1] for l in func(num)]
8
9 result= third_Largest(my_list, sort_list)
10 print(f"The third largest number from every sub list is:",result)
```

The output pane shows the execution results:

```
The greater number is: 35
[Done] exited with code=0 in 0.101 seconds
[Running] python -u "c:\Users\Administrator\Desktop\python\functions\76.py"
The third largest number from every sub list is: [8, 87, 12]
[Done] exited with code=0 in 0.095 seconds
```

Python Modules

Q.



The screenshot shows the Visual Studio Code editor with two files: 1.py and example_module.py. 1.py imports the example_module and calls the square function. example_module.py defines the square function. The terminal shows the execution of 1.py, which outputs 'By using the module square of number is: 16'.

```
1.py > ...
1 import example_module
2 result = example_module.square( 4 )
3 print(f"By using the module square of number is: ", result )
```

```
example_module.py > square
1 def square(number):
2     result = number ** 2
3     return result
4
```

The terminal output is:

```
PS D:\py_ust> python -u "d:\py_ust\1.py"
By using the module square of number is: 16
PS D:\py_ust>
```

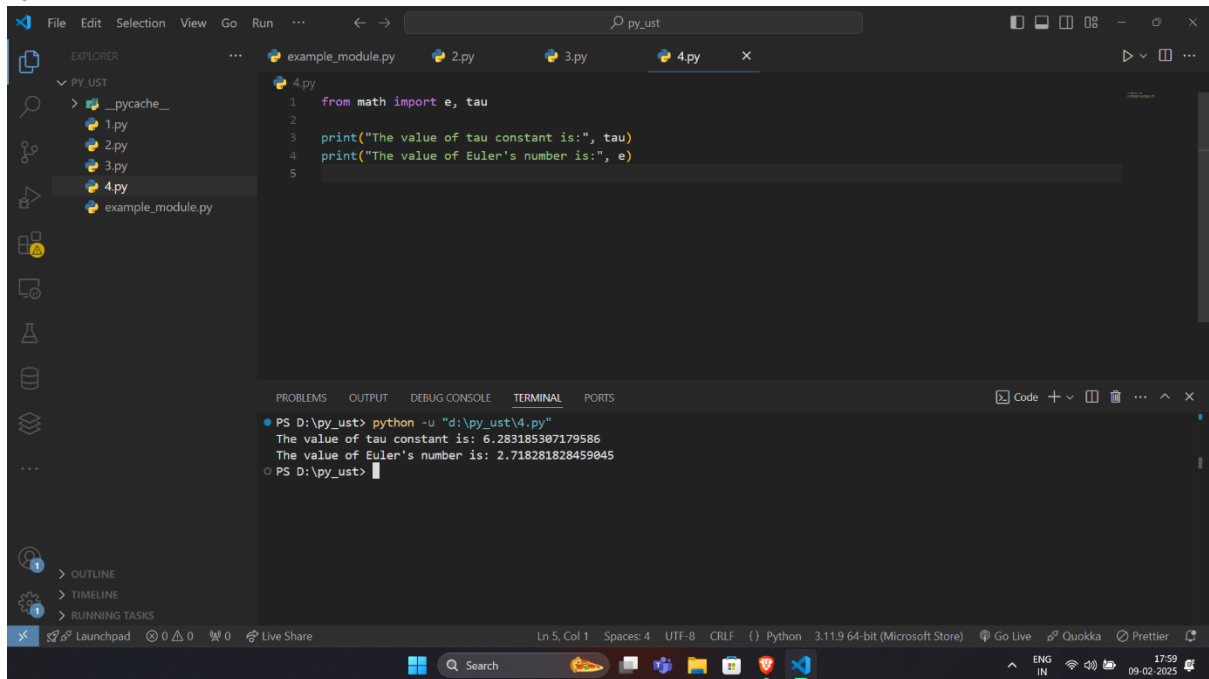
Q.

```
File Edit Selection View Go Run ...  
py_ust  
EXPLORER  
py_ust  
  __pycache__  
  1.py  
  2.py  
  example_module.py  
2.py  
1 import math  
2  
3 print("The value of Euler's number is", math.e)  
4  
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
PS D:\py_ust> python -u "d:\py_ust\2.py"  
The value of Euler's number is 2.718281828459045  
PS D:\py_ust>  
Ln 4, Col 1 Spaces: 4 UTF-8 CRLF Python 3.11.9 64-bit (Microsoft Store) Go Live Quokka Prettier  
Launchpad 0 0 0 Live Share
```

Q.

```
File Edit Selection View Go Run ...  
py_ust  
EXPLORER  
py_ust  
  __pycache__  
  1.py  
  2.py  
  3.py  
  example_module.py  
3.py  
1 import math as mt  
2  
3 print("The value of Euler's number is", mt.e)  
4  
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS  
PS D:\py_ust> python -u "d:\py_ust\3.py"  
The value of Euler's number is 2.718281828459045  
PS D:\py_ust>  
Ln 4, Col 1 Spaces: 4 UTF-8 CRLF Python 3.11.9 64-bit (Microsoft Store) Go Live Quokka Prettier  
Launchpad 0 0 0 Live Share
```

Q.



The screenshot shows the Visual Studio Code interface with a file explorer on the left displaying a project named 'py_ust' containing files '1.py', '2.py', '3.py', '4.py', and 'example_module.py'. The main editor window displays the content of '4.py':

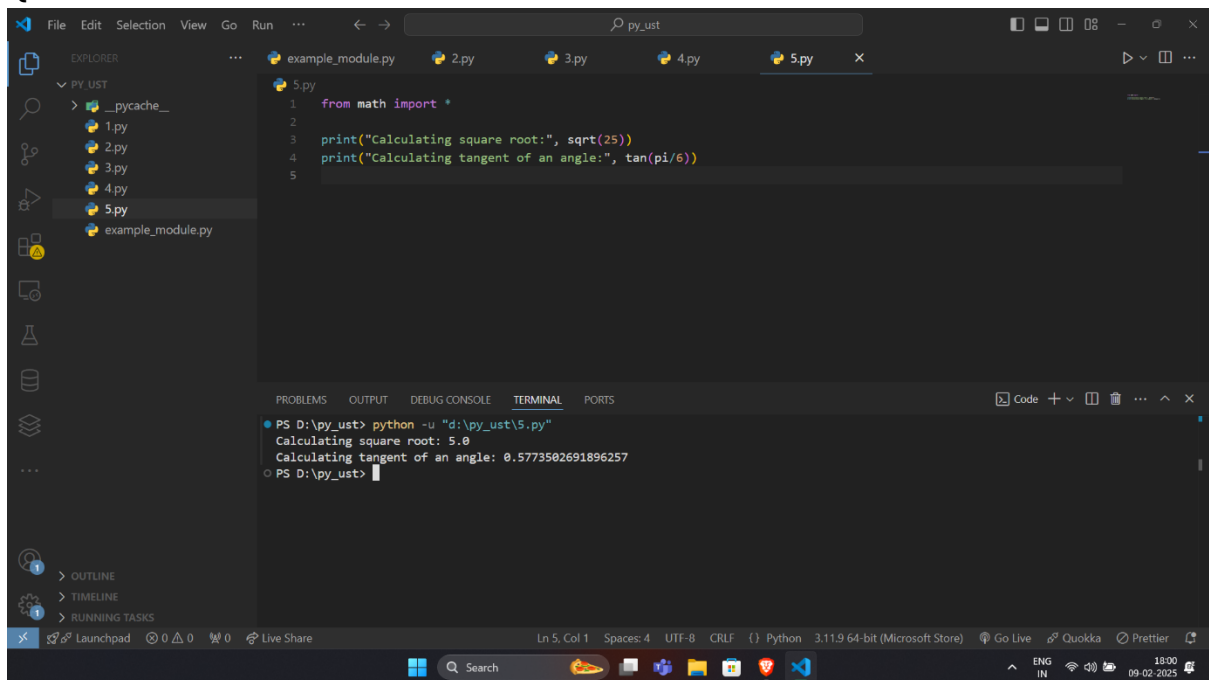
```
1 from math import e, tau
2
3 print("The value of tau constant is:", tau)
4 print("The value of Euler's number is:", e)
5
```

Below the editor, the 'TERMINAL' panel shows the command prompt output:

```
PS D:\py_ust> python -u "d:\py_ust\4.py"
The value of tau constant is: 6.283185307179586
The value of Euler's number is: 2.718281828459045
PS D:\py_ust>
```

The status bar at the bottom indicates the file is 'Ln 5, Col 1' with 'Spaces: 4' in 'UTF-8' encoding, using 'CRLF' line endings, and is a 'Python 3.11.9 64-bit (Microsoft Store)' file.

Q.



The screenshot shows the Visual Studio Code interface with a file explorer on the left displaying a project named 'py_ust' containing files '1.py', '2.py', '3.py', '4.py', '5.py', and 'example_module.py'. The main editor window displays the content of '5.py':

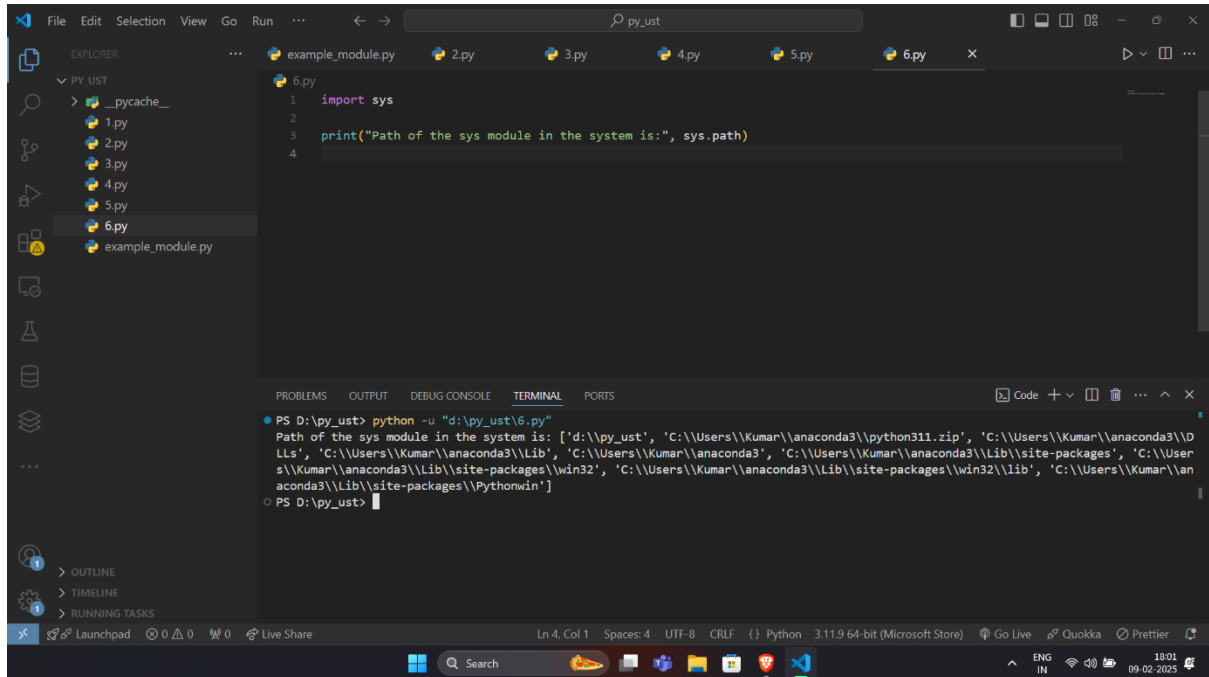
```
1 from math import *
2
3 print("Calculating square root:", sqrt(25))
4 print("Calculating tangent of an angle:", tan(pi/6))
5
```

Below the editor, the 'TERMINAL' panel shows the command prompt output:

```
PS D:\py_ust> python -u "d:\py_ust\5.py"
Calculating square root: 5.0
Calculating tangent of an angle: 0.5773502691896257
PS D:\py_ust>
```

The status bar at the bottom indicates the file is 'Ln 5, Col 1' with 'Spaces: 4' in 'UTF-8' encoding, using 'CRLF' line endings, and is a 'Python 3.11.9 64-bit (Microsoft Store)' file.

Q.



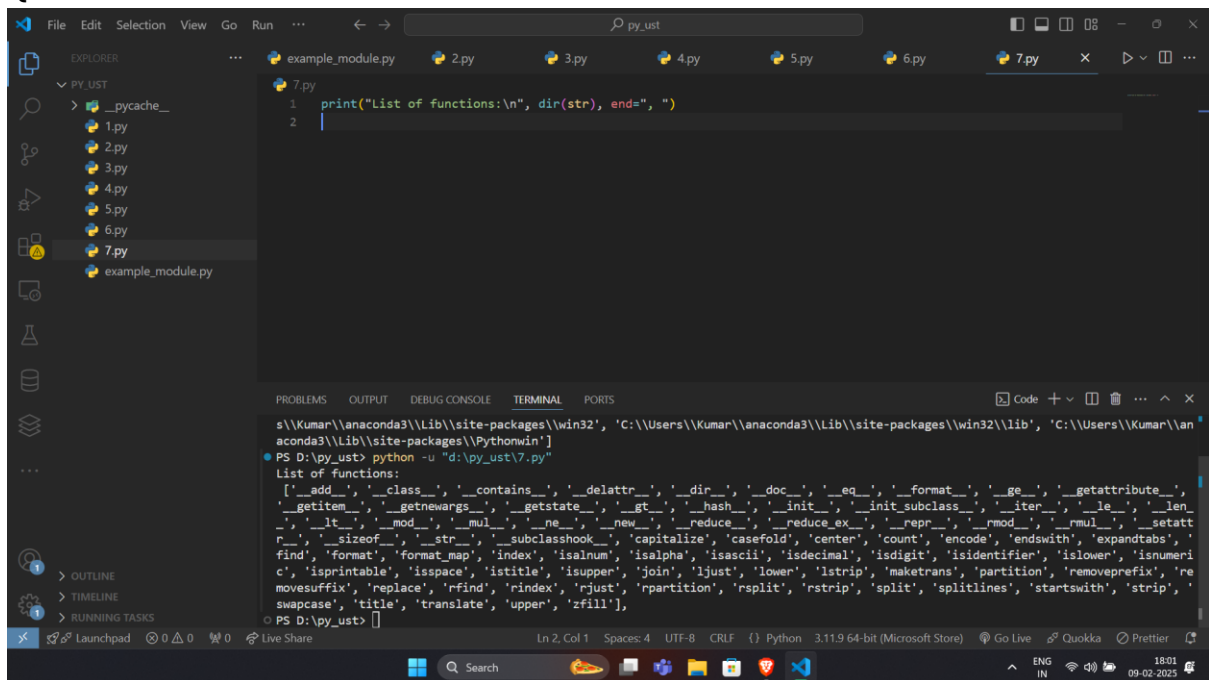
The screenshot shows a Visual Studio Code editor with a file explorer on the left displaying a project named 'PY_UST' containing several Python files (1.py to 6.py) and 'example_module.py'. The main editor window shows '6.py' with the following code:

```
1 import sys
2
3 print("Path of the sys module in the system is:", sys.path)
4
```

The terminal at the bottom shows the command `python -u "d:\py_ust\6.py"` being executed. The output displays the `sys.path` list, which includes the current directory and various system and user paths.

```
PS D:\py_ust> python -u "d:\py_ust\6.py"
Path of the sys module in the system is: ['d:\py_ust', 'C:\\Users\\Kumar\\anaconda3\\python311.zip', 'C:\\Users\\Kumar\\anaconda3\\DLLs', 'C:\\Users\\Kumar\\anaconda3\\Lib', 'C:\\Users\\Kumar\\anaconda3', 'C:\\Users\\Kumar\\anaconda3\\Lib\\site-packages', 'C:\\Users\\Kumar\\anaconda3\\Lib\\site-packages\\win32', 'C:\\Users\\Kumar\\anaconda3\\Lib\\site-packages\\win32\\lib', 'C:\\Users\\Kumar\\anaconda3\\Lib\\site-packages\\Pythonwin']
PS D:\py_ust>
```

Q.



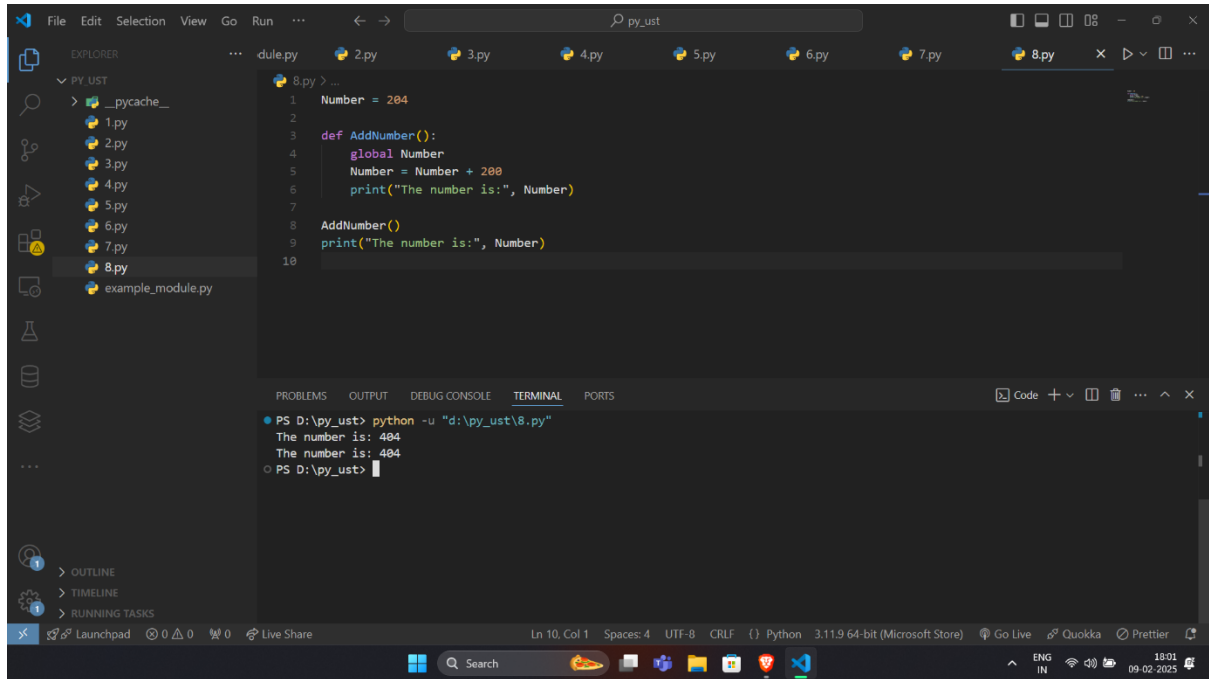
The screenshot shows the same Visual Studio Code editor with '7.py' open. The code in '7.py' is:

```
1 print("List of functions:\n", dir(str), end=" ")
2
```

The terminal shows the command `python -u "d:\py_ust\7.py"` being executed. The output lists all the methods and attributes available for the `str` object in Python.

```
s\\Kumar\\anaconda3\\Lib\\site-packages\\win32', 'C:\\Users\\Kumar\\anaconda3\\Lib\\site-packages\\win32\\lib', 'C:\\Users\\Kumar\\anaconda3\\Lib\\site-packages\\Pythonwin']
PS D:\py_ust> python -u "d:\py_ust\7.py"
List of functions:
['_add_', '_class_', '_contains_', '_delattr_', '_dir_', '_doc_', '_eq_', '_format_', '_ge_', '_getattribute_', '_getitem_', '_getnewargs_', '_getstate_', '_gt_', '_hash_', '_init_', '_init_subclass_', '_iter_', '_le_', '_len_', '_lt_', '_mod_', '_mul_', '_ne_', '_new_', '_reduce_', '_reduce_ex_', '_repr_', '_rmod_', '_rmul_', '_setattr_', '_sizeof_', '_str_', '_subclasshook_', '_capitalize_', '_casefold_', '_center_', '_count_', '_encode_', '_endswith_', '_expandtabs_', '_find_', '_format_', '_format_map_', '_index_', '_isalnum_', '_isalpha_', '_isascii_', '_isdecimal_', '_isdigit_', '_isidentifier_', '_islower_', '_isnumeric_', '_isprintable_', '_isspace_', '_istitle_', '_isupper_', '_join_', '_ljust_', '_lower_', '_lstrip_', '_maketrans_', '_partition_', '_removeprefix_', '_movesuffix_', '_replace_', '_rfind_', '_rindex_', '_rjust_', '_rpartition_', '_rsplit_', '_rstrip_', '_split_', '_splitlines_', '_startswith_', '_strip_', '_swapcase_', '_title_', '_translate_', '_upper_', '_zfill_']
PS D:\py_ust>
```

Q.



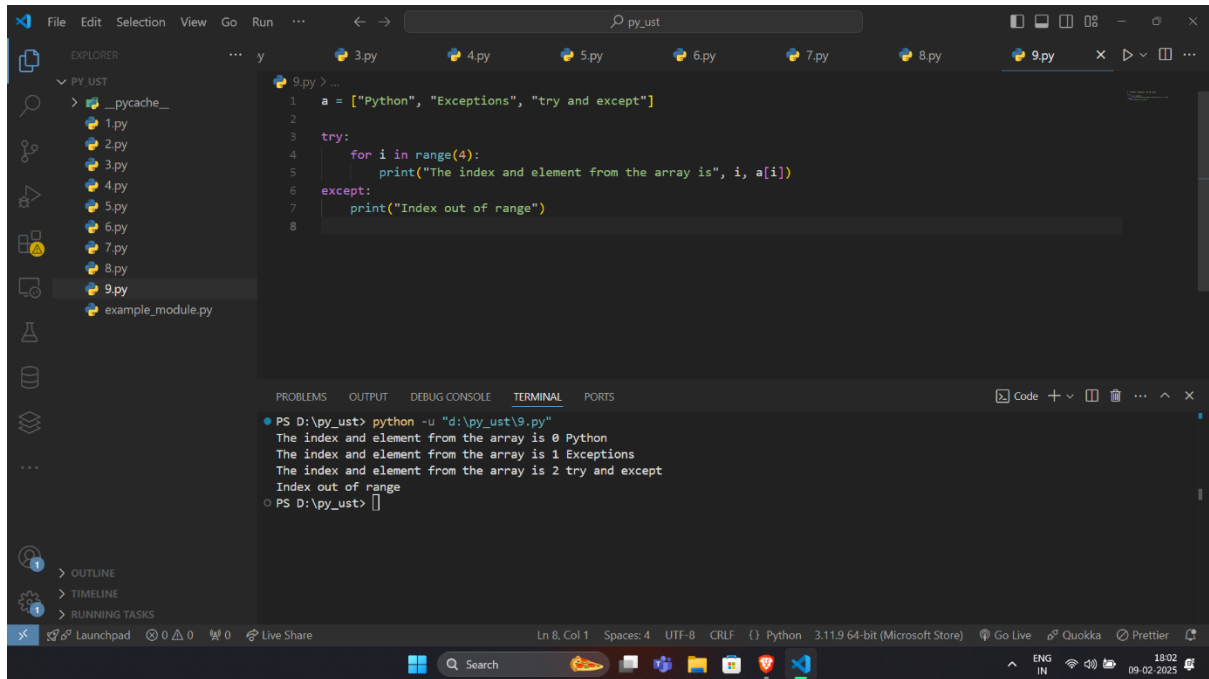
The screenshot shows the Visual Studio Code editor with a file explorer on the left displaying a directory named 'py_ust' containing files 1.py through 8.py and 'example_module.py'. The editor window shows the code for '8.py'.

```
1 Number = 204
2
3 def AddNumber():
4     global Number
5     Number = Number + 200
6     print("The number is:", Number)
7
8 AddNumber()
9 print("The number is:", Number)
10
```

The terminal at the bottom shows the command 'python -u "d:\py_ust\8.py"' being executed, resulting in the output:

```
PS D:\py_ust> python -u "d:\py_ust\8.py"
The number is: 404
The number is: 404
PS D:\py_ust>
```

Q.



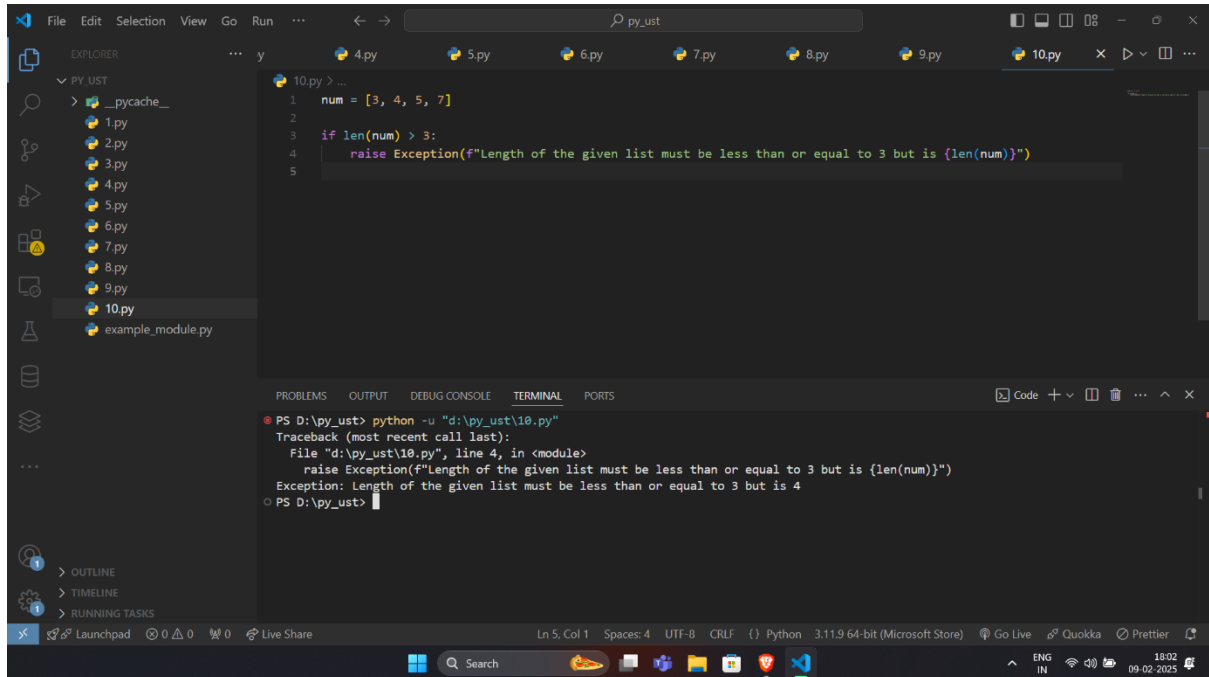
The screenshot shows the Visual Studio Code editor with a file explorer on the left displaying a directory named 'py_ust' containing files 1.py through 9.py and 'example_module.py'. The editor window shows the code for '9.py'.

```
1 a = ["Python", "Exceptions", "try and except"]
2
3
4 try:
5     for i in range(4):
6         print("The index and element from the array is", i, a[i])
7 except:
8     print("Index out of range")
```

The terminal at the bottom shows the command 'python -u "d:\py_ust\9.py"' being executed, resulting in the output:

```
PS D:\py_ust> python -u "d:\py_ust\9.py"
The index and element from the array is 0 Python
The index and element from the array is 1 Exceptions
The index and element from the array is 2 try and except
Index out of range
PS D:\py_ust>
```

Q.



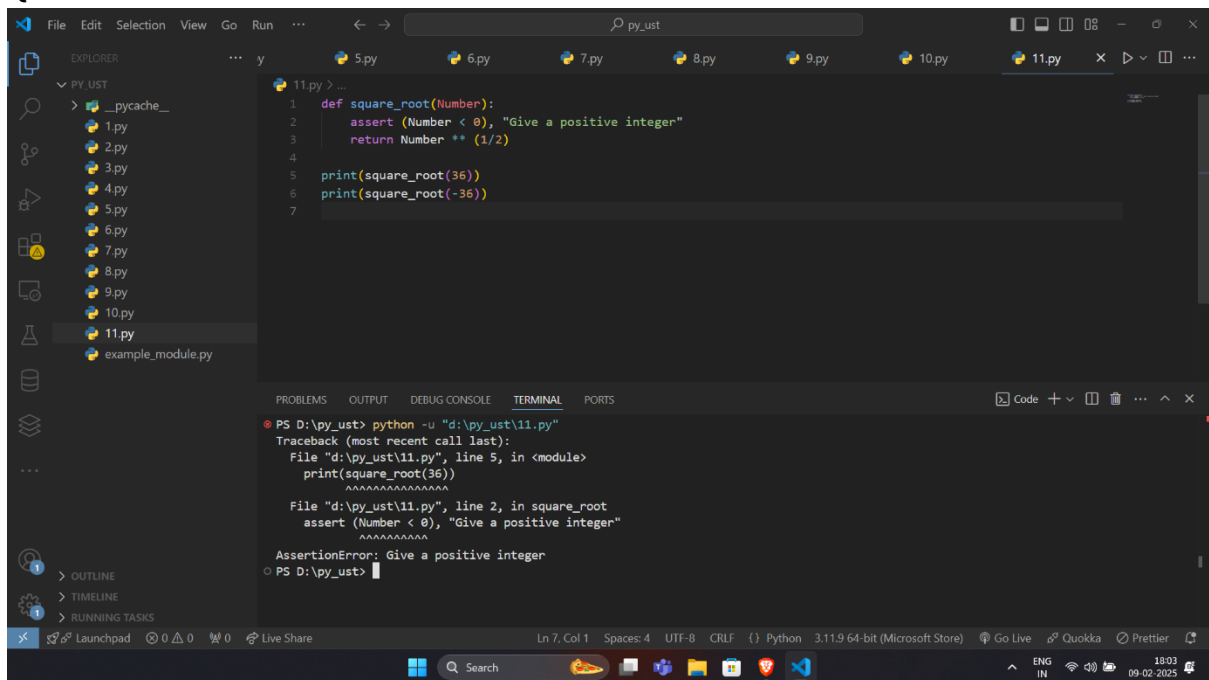
The screenshot shows the Visual Studio Code editor with a file explorer on the left displaying a directory named 'py_ust' containing files 1.py through 10.py and 'example_module.py'. The editor window shows '10.py' with the following code:

```
1 num = [3, 4, 5, 7]
2
3 if len(num) > 3:
4     raise Exception(f"Length of the given list must be less than or equal to 3 but is {len(num)}")
5
```

The terminal at the bottom shows the command `python -u "d:\py_ust\10.py"` being executed. The output is a traceback showing the exception being raised:

```
PS D:\py_ust> python -u "d:\py_ust\10.py"
Traceback (most recent call last):
  File "d:\py_ust\10.py", line 4, in <module>
    raise Exception(f"Length of the given list must be less than or equal to 3 but is {len(num)}")
Exception: Length of the given list must be less than or equal to 3 but is 4
PS D:\py_ust>
```

Q.



The screenshot shows the Visual Studio Code editor with a file explorer on the left displaying a directory named 'py_ust' containing files 1.py through 11.py and 'example_module.py'. The editor window shows '11.py' with the following code:

```
1 def square_root(Number):
2     assert (Number < 0), "Give a positive integer"
3     return Number ** (1/2)
4
5 print(square_root(36))
6 print(square_root(-36))
7
```

The terminal at the bottom shows the command `python -u "d:\py_ust\11.py"` being executed. The output is a traceback showing the `AssertionError` being raised:

```
PS D:\py_ust> python -u "d:\py_ust\11.py"
Traceback (most recent call last):
  File "d:\py_ust\11.py", line 5, in <module>
    print(square_root(36))
    ~~~~~^~~~~~
  File "d:\py_ust\11.py", line 2, in square_root
    assert (Number < 0), "Give a positive integer"
    ~~~~~^~~~~~
AssertionError: Give a positive integer
PS D:\py_ust>
```


Q.

The screenshot shows the Visual Studio Code editor with a file explorer on the left displaying a directory named 'PY_UST' containing files 1.py through 12.py and 'example_module.py'. The main editor window displays the code for '12.py'.

```
1 def reciprocal(num1):  
2     try:  
3         reci = 1 / num1  
4     except ZeroDivisionError:  
5         print("We cannot divide by zero")  
6     else:  
7         print(reci)  
8  
9     reciprocal(4)  
10    reciprocal(0)  
11
```

The bottom panel shows the 'TERMINAL' output for the command `python -u "d:\py_ust\12.py"`:

```
PS D:\py_ust> python -u "d:\py_ust\12.py"  
0.25  
We cannot divide by zero  
PS D:\py_ust>
```

Q.

The screenshot shows the Visual Studio Code editor with a file explorer on the left displaying a directory named 'PY_UST' containing files 1.py through 13.py and 'example_module.py'. The main editor window displays the code for '13.py'.

```
1 try:  
2     div = 4 // 0  
3     print(div)  
4 except ZeroDivisionError:  
5     print("Attempting to divide by zero")  
6 finally:  
7     print('This is code of finally clause')  
8
```

The bottom panel shows the 'TERMINAL' output for the command `python -u "d:\py_ust\13.py"`:

```
PS D:\py_ust> python -u "d:\py_ust\13.py"  
Attempting to divide by zero  
This is code of finally clause  
PS D:\py_ust>
```

Q.

The screenshot shows a Visual Studio Code editor with a file explorer on the left containing a folder named 'py_ust' with files 1.py through 14.py. The main editor displays '14.py' with the following code:

```
1 class EmptyError(RuntimeError):
2     def __init__(self, argument):
3         self.arguments = argument
4
5     var = " "
6
7     try:
8         raise EmptyError("The variable is empty")
9     except (EmptyError, var):
10        print(var.arguments)
```

The terminal at the bottom shows the execution of the script and the resulting exception:

```
PS D:\py_ust> python -u "d:\py_ust\14.py"
Traceback (most recent call last):
  File "d:\py_ust\14.py", line 7, in <module>
    raise EmptyError("The variable is empty")
EmptyError: The variable is empty

During handling of the above exception, another exception occurred:

Traceback (most recent call last):
  File "d:\py_ust\14.py", line 8, in <module>
    except (EmptyError, var):
TypeError: catching classes that do not inherit from BaseException is not allowed
PS D:\py_ust>
```

Python Arrays

Q.

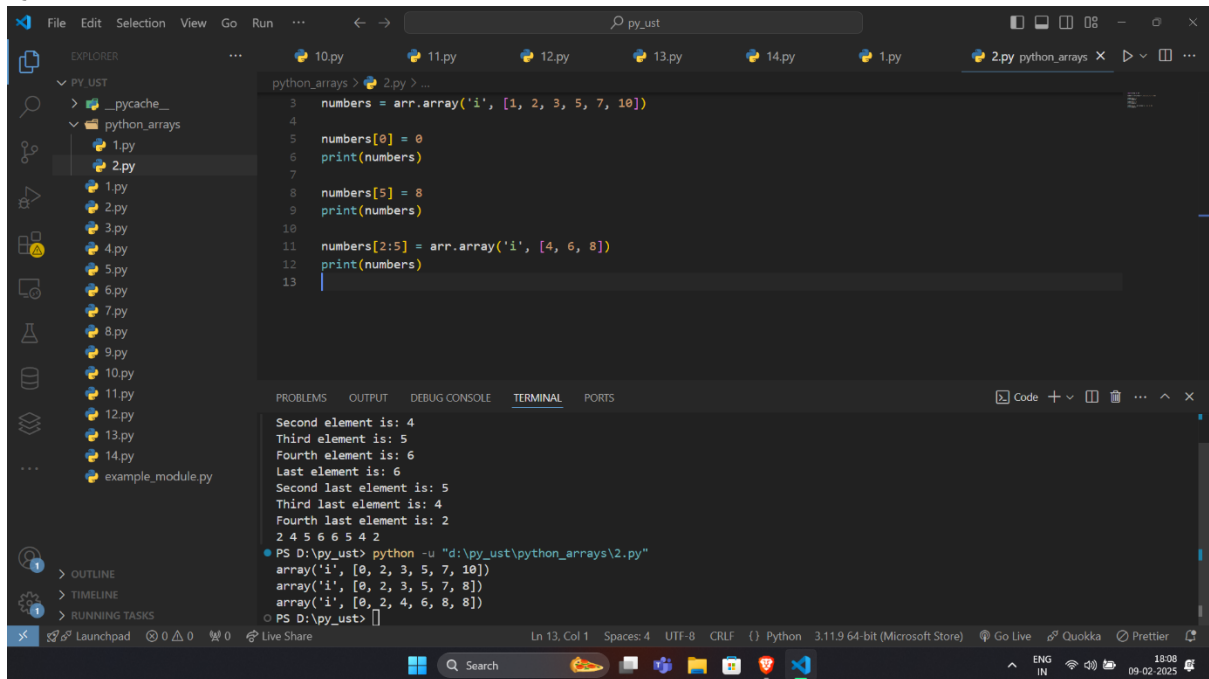
The screenshot shows a Visual Studio Code editor with a file explorer on the left containing a folder named 'python_arrays' with files 1.py through 14.py. The main editor displays '1.py' with the following code:

```
4
5 print("First element is:", a[0])
6 print("Second element is:", a[1])
7 print("Third element is:", a[2])
8 print("Fourth element is:", a[3])
9 print("Last element is:", a[-1])
10 print("Second last element is:", a[-2])
11 print("Third last element is:", a[-3])
12 print("Fourth last element is:", a[-4])
13 print(a[0], a[1], a[2], a[3], a[-1], a[-2], a[-3], a[-4])
14
```

The terminal at the bottom shows the execution of the script and the resulting output:

```
PS D:\py_ust> python -u "d:\py_ust\python_arrays\1.py"
First element is: 2
Second element is: 4
Third element is: 5
Fourth element is: 6
Last element is: 6
Second last element is: 5
Third last element is: 4
Fourth last element is: 2
2 4 5 6 6 5 4 2
PS D:\py_ust>
```

Q.



The screenshot shows the Visual Studio Code editor with a file explorer on the left displaying a project named 'py_ust' containing a subdirectory 'python_arrays' with files 1.py through 14.py and 'example_module.py'. The editor window shows '2.py' with the following code:

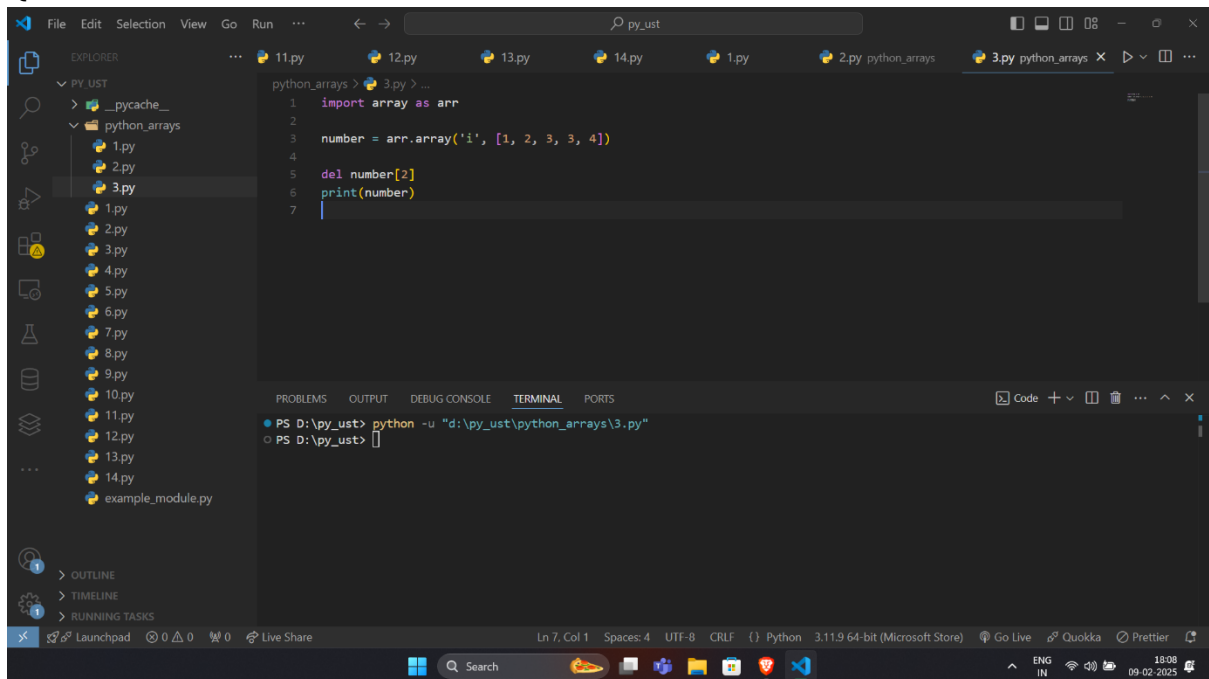
```
python_arrays > 2.py > ...  
3 numbers = arr.array('i', [1, 2, 3, 5, 7, 10])  
4  
5 numbers[0] = 0  
6 print(numbers)  
7  
8 numbers[5] = 8  
9 print(numbers)  
10  
11 numbers[2:5] = arr.array('i', [4, 6, 8])  
12 print(numbers)  
13
```

The terminal at the bottom shows the output of running the script:

```
PS D:\py_ust> python -u "d:\py_ust\python_arrays\2.py"  
array('i', [0, 2, 3, 5, 7, 10])  
array('i', [0, 2, 3, 5, 7, 8])  
array('i', [0, 2, 4, 6, 8, 8])  
PS D:\py_ust>
```

The terminal output also includes a list of elements and their indices: 'Second element is: 4', 'Third element is: 5', 'Fourth element is: 6', 'Last element is: 6', 'Second last element is: 5', 'Third last element is: 4', 'Fourth last element is: 2', and a final line '2 4 5 6 6 5 4 2'.

Q.



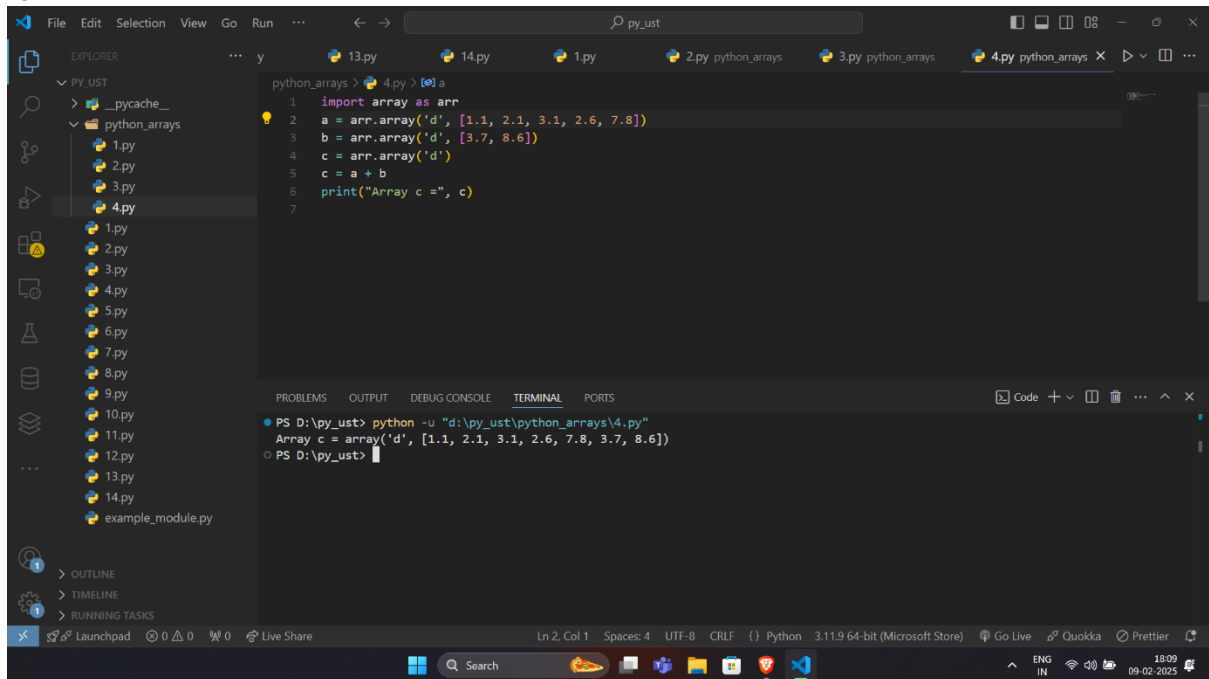
The screenshot shows the Visual Studio Code editor with the same project structure as the first image. The editor window now shows '3.py' with the following code:

```
python_arrays > 3.py > ...  
1 import array as arr  
2  
3 number = arr.array('i', [1, 2, 3, 3, 4])  
4  
5 del number[2]  
6 print(number)  
7
```

The terminal at the bottom shows the output of running the script:

```
PS D:\py_ust> python -u "d:\py_ust\python_arrays\3.py"  
PS D:\py_ust>
```

Q.



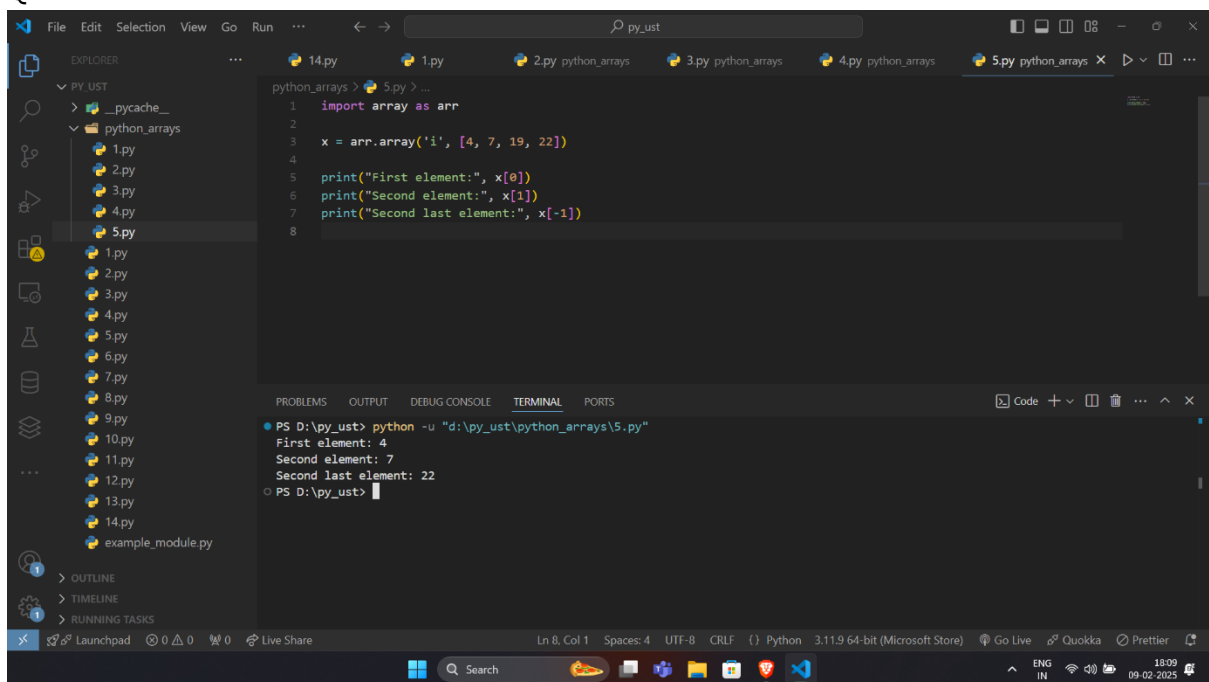
The screenshot shows the Visual Studio Code editor interface. The Explorer panel on the left displays a file tree with a project named 'PY_UST' containing a subdirectory 'python_arrays' with files 1.py through 14.py and an 'example_module.py' file. The main editor window is open to '4.py' in the 'python_arrays' directory. The code in 4.py is as follows:

```
1 import array as arr
2 a = arr.array('d', [1.1, 2.1, 3.1, 2.6, 7.8])
3 b = arr.array('d', [3.7, 8.6])
4 c = arr.array('d')
5 c = a + b
6 print("Array c =", c)
7
```

The TERMINAL panel at the bottom shows the command prompt output for running the script:

```
PS D:\py_ust> python -u "d:\py_ust\python_arrays\4.py"
Array c = array('d', [1.1, 2.1, 3.1, 2.6, 7.8, 3.7, 8.6])
PS D:\py_ust>
```

Q.



The screenshot shows the Visual Studio Code editor interface. The Explorer panel on the left displays the same file tree as the previous screenshot. The main editor window is open to '5.py' in the 'python_arrays' directory. The code in 5.py is as follows:

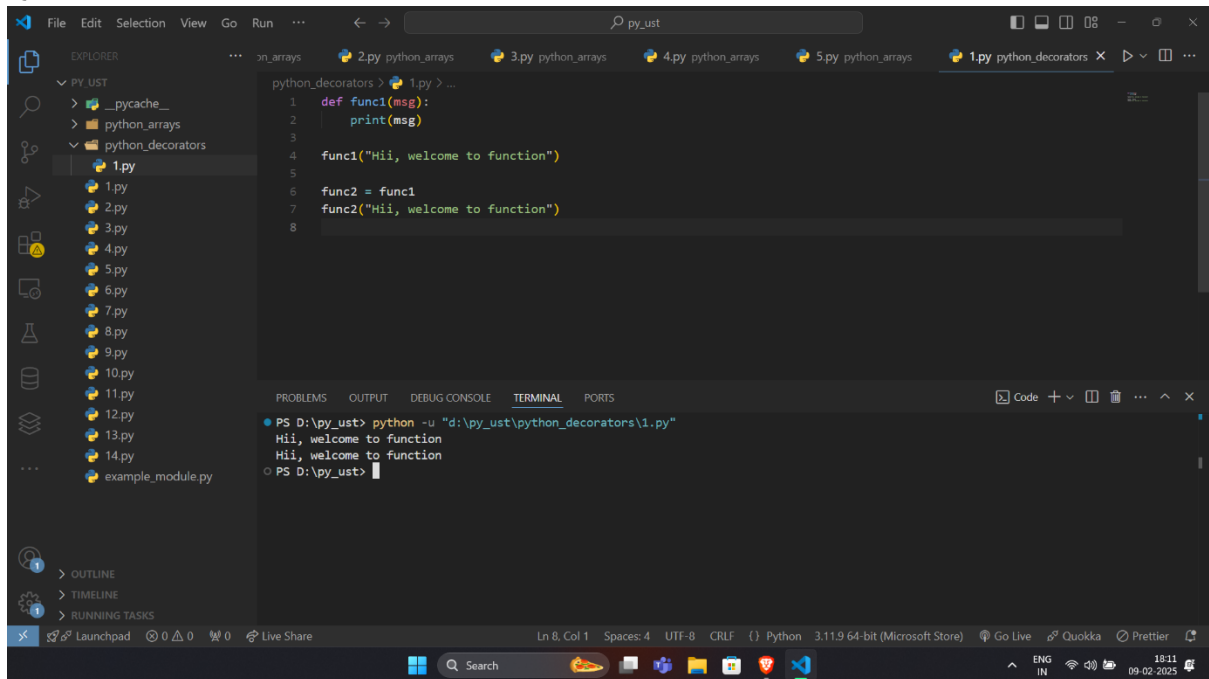
```
1 import array as arr
2
3 x = arr.array('i', [4, 7, 19, 22])
4
5 print("First element:", x[0])
6 print("Second element:", x[1])
7 print("Second last element:", x[-1])
8
```

The TERMINAL panel at the bottom shows the command prompt output for running the script:

```
PS D:\py_ust> python -u "d:\py_ust\python_arrays\5.py"
First element: 4
Second element: 7
Second last element: 22
PS D:\py_ust>
```

Python Decorators

Q.



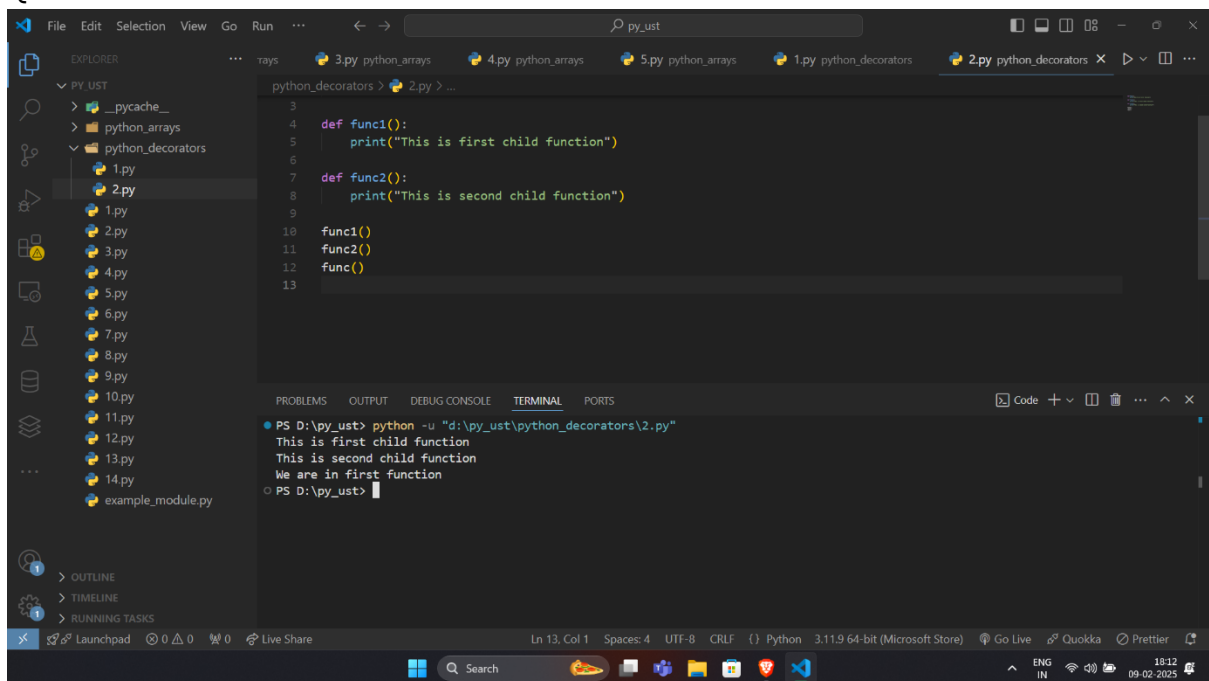
The screenshot shows the Visual Studio Code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'py_ust' with a subdirectory 'python_decorators' containing files '1.py' through '14.py' and 'example_module.py'. The main editor window displays the code in '1.py' from the 'python_decorators' directory. The code defines a decorator 'func1' that prints a message, then calls 'func1' with the string 'Hii, welcome to function'. It then defines 'func2' as 'func1' and calls 'func2' with the same string. The terminal at the bottom shows the command 'python -u "d:\py_ust\python_decorators\1.py"' being executed, resulting in the output 'Hii, welcome to function' printed twice.

```
python_decorators > 1.py > ...
1  def func1(msg):
2      print(msg)
3
4  func1("Hii, welcome to function")
5
6  func2 = func1
7  func2("Hii, welcome to function")
8
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\py_ust> python -u "d:\py_ust\python_decorators\1.py"
Hii, welcome to function
Hii, welcome to function
PS D:\py_ust>
```

Q.



The screenshot shows the Visual Studio Code editor with a file explorer on the left and a terminal at the bottom. The file explorer shows a project named 'py_ust' with a subdirectory 'python_decorators' containing files '1.py' through '14.py' and 'example_module.py'. The main editor window displays the code in '2.py' from the 'python_decorators' directory. The code defines two child functions, 'func1' and 'func2', each printing a message. It then calls 'func1()', 'func2()', and 'func()' (which is not defined). The terminal at the bottom shows the command 'python -u "d:\py_ust\python_decorators\2.py"' being executed, resulting in the output 'This is first child function', 'This is second child function', and 'We are in first function'.

```
python_decorators > 2.py > ...
3
4  def func1():
5      print("This is first child function")
6
7  def func2():
8      print("This is second child function")
9
10 func1()
11 func2()
12 func()
13
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\py_ust> python -u "d:\py_ust\python_decorators\2.py"
This is first child function
This is second child function
We are in first function
PS D:\py_ust>
```

Q.

The screenshot shows the Visual Studio Code editor with a project named 'py_ust'. The Explorer pane on the left shows a file structure with 'python_decorators' containing '1.py', '2.py', and '3.py'. The main editor window displays the code for '3.py' in the 'python_decorators' directory. The code defines a 'sub' function, a decorator 'operator' that takes a function 'func' and returns a wrapper that calls 'func' and then subtracts 1 from the result, and finally calls 'operator(sub, 10)' and 'operator(add, 20)'. The output pane at the bottom shows the execution of 'python -u "d:\py_ust\python_decorators\3.py"' resulting in the output '21'.

```
python_decorators > 3.py > ...
3
4     def sub(x):
5         return x - 1
6
7     def operator(func, x):
8         temp = func(x)
9         return temp
10
11     print(operator(sub, 10))
12     print(operator(add, 20))
13
```

```
PS D:\py_ust> python -u "d:\py_ust\python_decorators\3.py"
21
PS D:\py_ust>
```

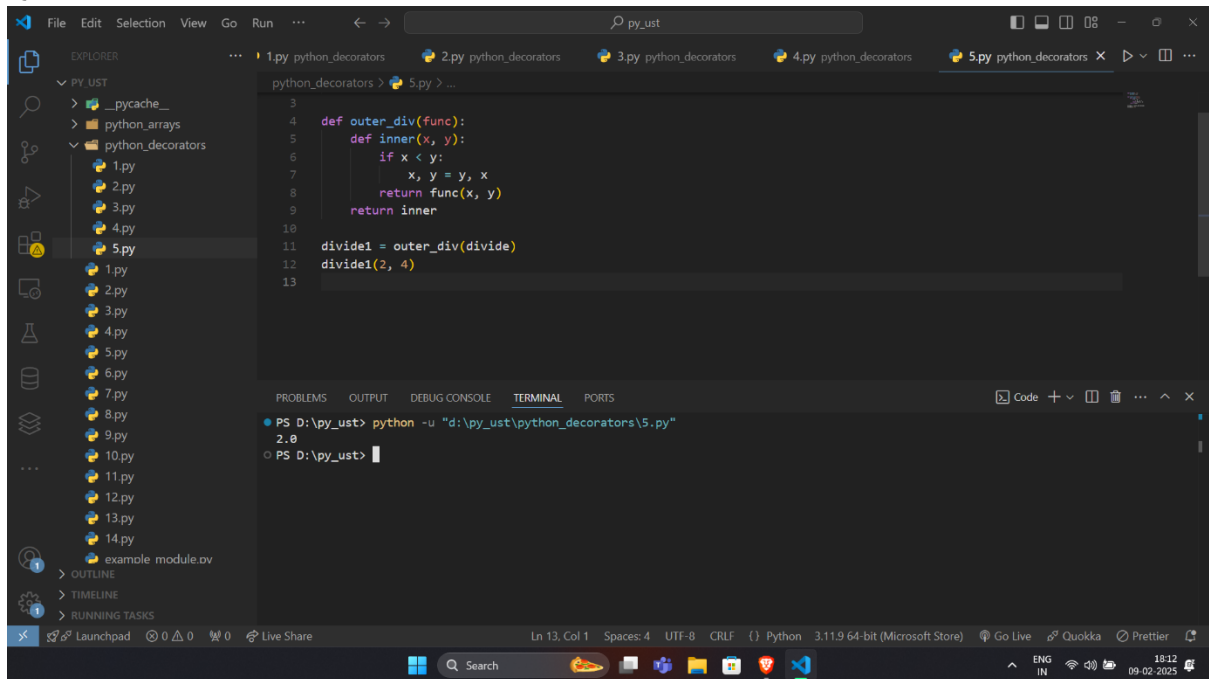
Q.

The screenshot shows the Visual Studio Code editor with a project named 'py_ust'. The Explorer pane on the left shows a file structure with 'python_decorators' containing '1.py', '2.py', '3.py', and '4.py'. The main editor window displays the code for '4.py' in the 'python_decorators' directory. The code defines a 'hello' function, a decorator 'hi' that takes a function 'func' and returns a wrapper that calls 'func' and then prints 'Hello', and finally calls 'new = hello()' and 'new()'. The output pane at the bottom shows the execution of 'python -u "d:\py_ust\python_decorators\4.py"' resulting in the output 'Hello'.

```
python_decorators > 4.py > ...
1     def hello():
2         def hi():
3             print("Hello")
4             return hi
5
6     new = hello()
7     new()
8
```

```
PS D:\py_ust> python -u "d:\py_ust\python_decorators\4.py"
Hello
PS D:\py_ust>
```

Q.

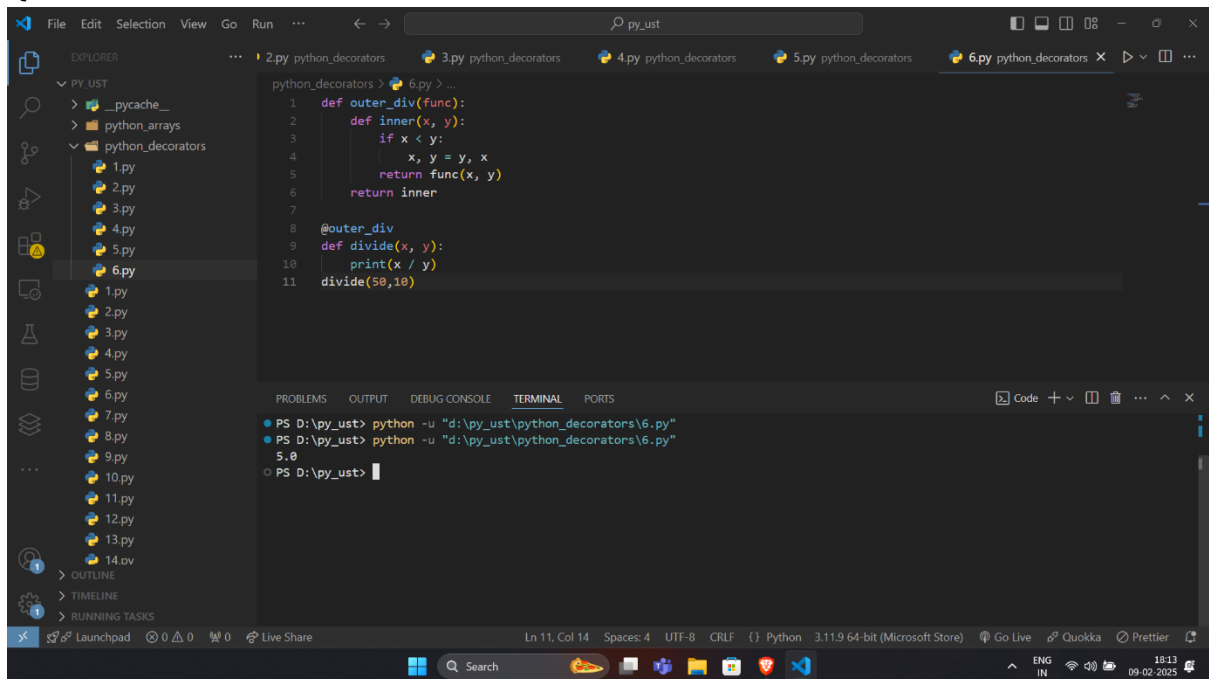


```
python_decorators > 5.py > ...
3
4 def outer_div(func):
5     def inner(x, y):
6         if x < y:
7             x, y = y, x
8         return func(x, y)
9     return inner
10
11 divide = outer_div(divide)
12 divide(2, 4)
13
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\py_ust> python -u "d:\py_ust\python_decorators\5.py"
2.0
PS D:\py_ust>
```

Q.



```
python_decorators > 6.py > ...
1 def outer_div(func):
2     def inner(x, y):
3         if x < y:
4             x, y = y, x
5         return func(x, y)
6     return inner
7
8 @outer_div
9 def divide(x, y):
10     print(x / y)
11 divide(50,10)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\py_ust> python -u "d:\py_ust\python_decorators\6.py"
5.0
PS D:\py_ust>
```

Q.

The screenshot shows the VS Code interface with a file explorer on the left displaying a project structure. The main editor shows a file named `7.py` with the following code:

```
python_decorators > 7.py > ...
1  from decorator import do_twice # type: ignore
2
3  @do_twice
4  def say_hello():
5      print("Hello There")
6
7  say_hello()
8
```

The terminal at the bottom shows the command `python -u "d:\py_ust\python_decorators\7.py"` being executed, which results in a `ImportError: cannot import name 'do_twice' from 'decorator' (C:\Users\Kumar\anaconda3\Lib\site-packages\decorator.py)`. The output of the program shows "Hello There" printed twice.

Q.

The screenshot shows the VS Code interface with a file explorer on the left displaying a project structure. The main editor shows a file named `8.py` with the following code:

```
python_decorators > 8.py > display
1  from mod_decorator import do_twice
2
3  @do_twice
4  def display(name):
5      print(f"Hello {name}")
6
7  display()
8
```

The terminal at the bottom shows the command `python -u "d:\py_ust\python_decorators\8.py"` being executed, which results in a `TypeError: display() missing 1 required positional argument: 'name'`. The output of the program shows "Hello {name}" printed twice.

Q.

```
python_decorators > 9.py > display
1 from mod_decorator import do_twice
2
3 @do_twice
4 def display(name):
5     print(f"Hello {name}")
6
7 display("John")
8
```

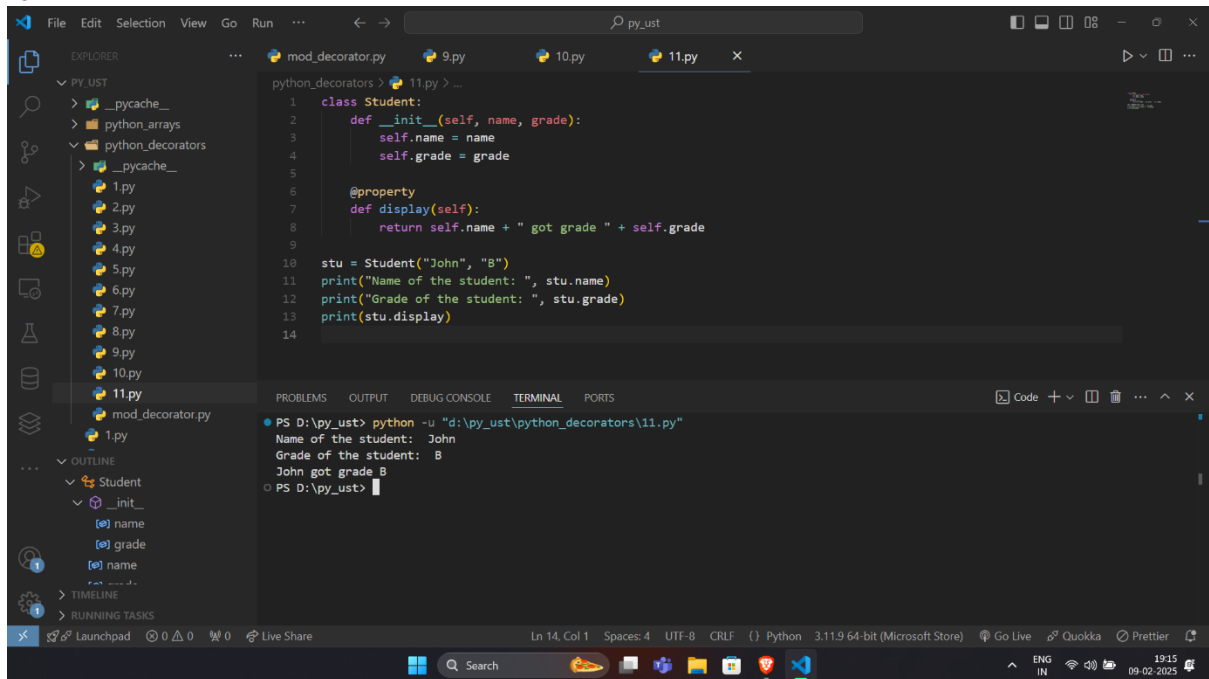
```
PS D:\py_ust> python -u "d:\py_ust\python_decorators\9.py"
Hello John
Hello John
PS D:\py_ust>
```

Q.

```
python_decorators > 10.py > ...
1 from mod_decorator import do_twice
2
3 @do_twice
4 def return_greeting(name):
5     print("We are created greeting")
6     return f"Hi {name}"
7
8 hi_adam = return_greeting("Adam")
9
```

```
PS D:\py_ust> python -u "d:\py_ust\python_decorators\9.py"
Hello John
Hello John
Hello John
PS D:\py_ust> python -u "d:\py_ust\python_decorators\10.py"
We are created greeting
We are created greeting
Hi Adam
PS D:\py_ust>
```

Q.



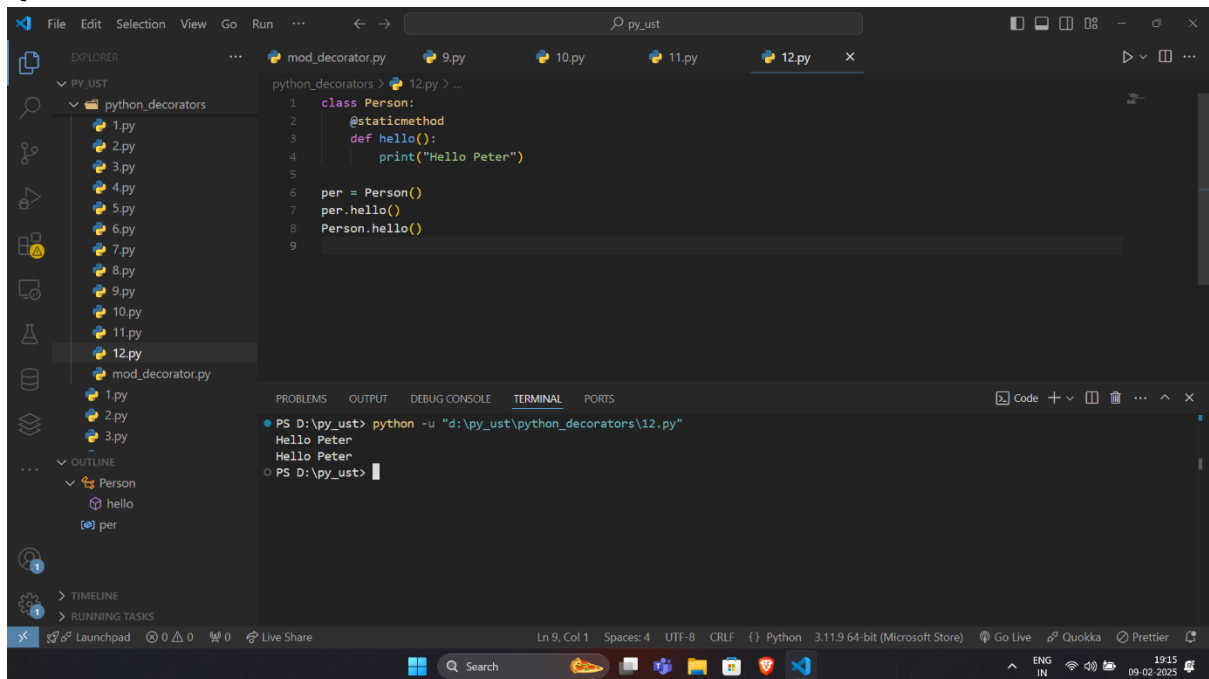
The screenshot shows the Visual Studio Code editor with a Python file named `11.py` open. The file contains a `Student` class with an `__init__` method and a `display` property. The terminal output shows the execution of the script, which prints the student's name and grade, and then the `display` property.

```
python_decorators > 11.py > ...  
1 class Student:  
2     def __init__(self, name, grade):  
3         self.name = name  
4         self.grade = grade  
5  
6     @property  
7     def display(self):  
8         return self.name + " got grade " + self.grade  
9  
10 stu = Student("John", "B")  
11 print("Name of the student: ", stu.name)  
12 print("Grade of the student: ", stu.grade)  
13 print(stu.display)  
14
```

Terminal Output:

```
PS D:\py_ust> python -u "d:\py_ust\python_decorators\11.py"  
Name of the student: John  
Grade of the student: B  
John got grade B  
PS D:\py_ust>
```

Q.



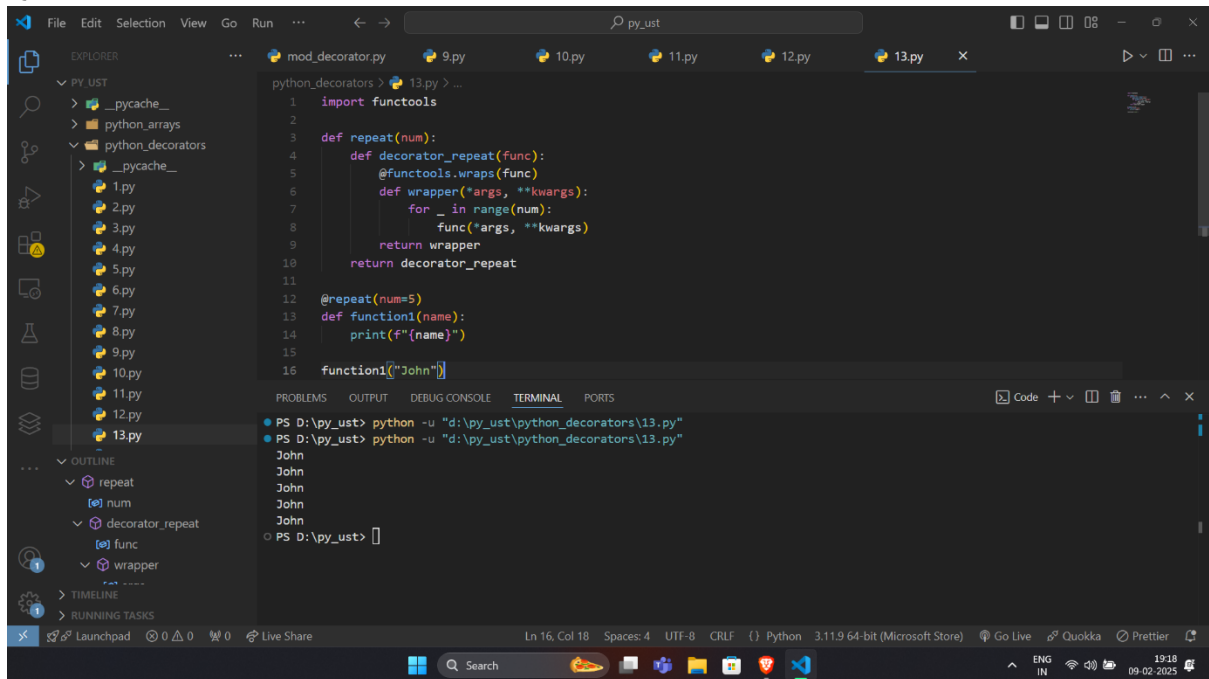
The screenshot shows the Visual Studio Code editor with a Python file named `12.py` open. The file contains a `Person` class with a static method `hello`. The terminal output shows the execution of the script, which prints the output of the static method.

```
python_decorators > 12.py > ...  
1 class Person:  
2     @staticmethod  
3     def hello():  
4         print("Hello Peter")  
5  
6 per = Person()  
7 per.hello()  
8 Person.hello()  
9
```

Terminal Output:

```
PS D:\py_ust> python -u "d:\py_ust\python_decorators\12.py"  
Hello Peter  
Hello Peter  
PS D:\py_ust>
```

Q.

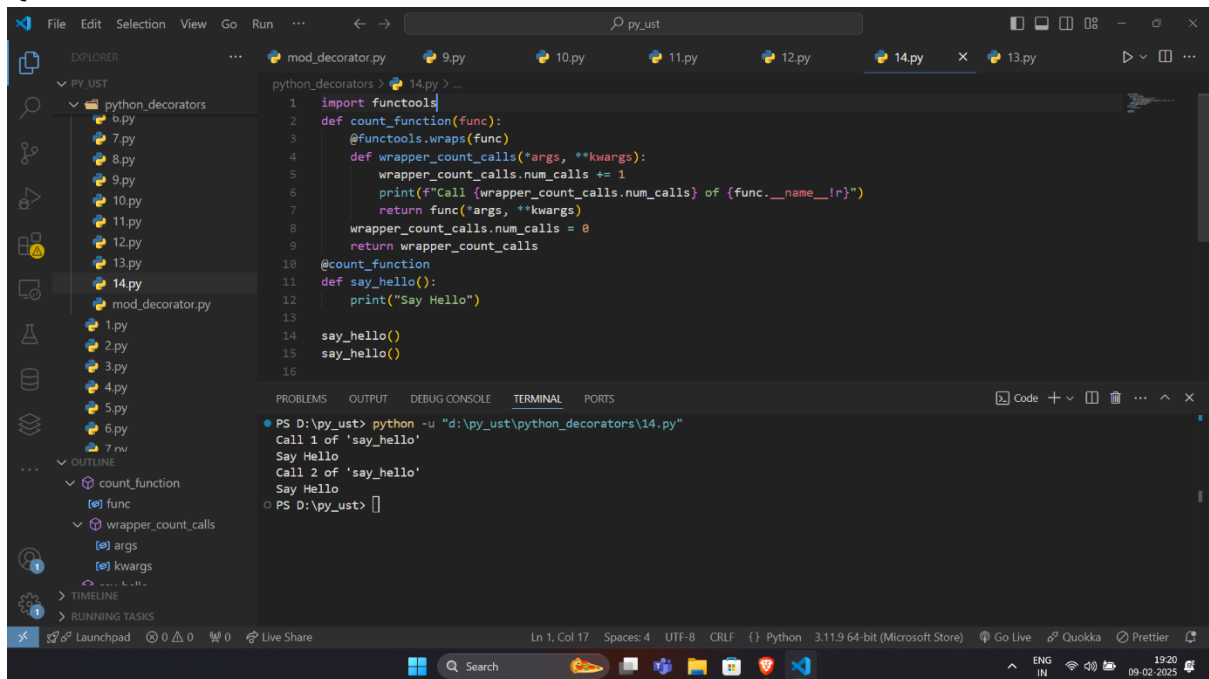


```
python_decorators > 13.py > ...
1 import functools
2
3 def repeat(num):
4     def decorator_repeat(func):
5         @functools.wraps(func)
6         def wrapper(*args, **kwargs):
7             for _ in range(num):
8                 func(*args, **kwargs)
9             return wrapper
10        return decorator_repeat
11
12 @repeat(num=5)
13 def function1(name):
14     print(f'{name}')
15
16 function1("John")
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\py_ust> python -u "d:\py_ust\python_decorators\13.py"
John
John
John
John
John
PS D:\py_ust> python -u "d:\py_ust\python_decorators\13.py"
John
John
John
John
John
PS D:\py_ust>
```

Q.



```
python_decorators > 14.py > ...
1 import functools
2 def count_function(func):
3     @functools.wraps(func)
4     def wrapper_count_calls(*args, **kwargs):
5         wrapper_count_calls.num_calls += 1
6         print(f'Call {wrapper_count_calls.num_calls} of {func.__name__!r}')
7         return func(*args, **kwargs)
8     wrapper_count_calls.num_calls = 0
9     return wrapper_count_calls
10 @count_function
11 def say_hello():
12     print("Say Hello")
13
14 say_hello()
15 say_hello()
16
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS D:\py_ust> python -u "d:\py_ust\python_decorators\14.py"
Call 1 of 'say_hello'
Say Hello
Call 2 of 'say_hello'
Say Hello
PS D:\py_ust>
```

Q.

The screenshot shows the Visual Studio Code interface with a Python file named `15.py` in the `python_decorators` folder. The code defines a `Count_Calls` decorator class that uses `functools.update_wrapper` to track the number of calls to a wrapped function. The `say_hello` function is decorated with `@Count_Calls`. The terminal output shows the function being called three times, with the decorator printing the call count for each invocation.

```
python_decorators > 15.py > Count_Calls > __call__
1 import functools
2
3 class Count_Calls:
4     def __init__(self, func):
5         functools.update_wrapper(self, func)
6         self.func = func
7         self.num_calls = 0
8     def __call__(self, *args, **kwargs):
9         self.num_calls += 1
10        print(f"Call {self.num_calls} of {self.func.__name__}")
11        return self.func(*args, **kwargs)
12
13 @Count_Calls
14 def say_hello():
15     print("Say Hello")
16 say_hello()
17 say_hello()
18 say_hello()
```

Terminal Output:

```
PS D:\py_ust> python -u "d:\py_ust\python_decorators\15.py"
Call 1 of 'say_hello'
Say Hello
Call 2 of 'say_hello'
Say Hello
Call 3 of 'say_hello'
Say Hello
PS D:\py_ust>
```

Python Generators

Q.

The screenshot shows the Visual Studio Code interface with a Python file named `1.py` in the `generators` folder. The code defines a `simple` generator function that yields values from 0 to 9. The `simple` function is called, and its values are printed. The terminal output shows the values 0, 2, 4, 6, and 8 being printed, indicating that the generator is being iterated over.

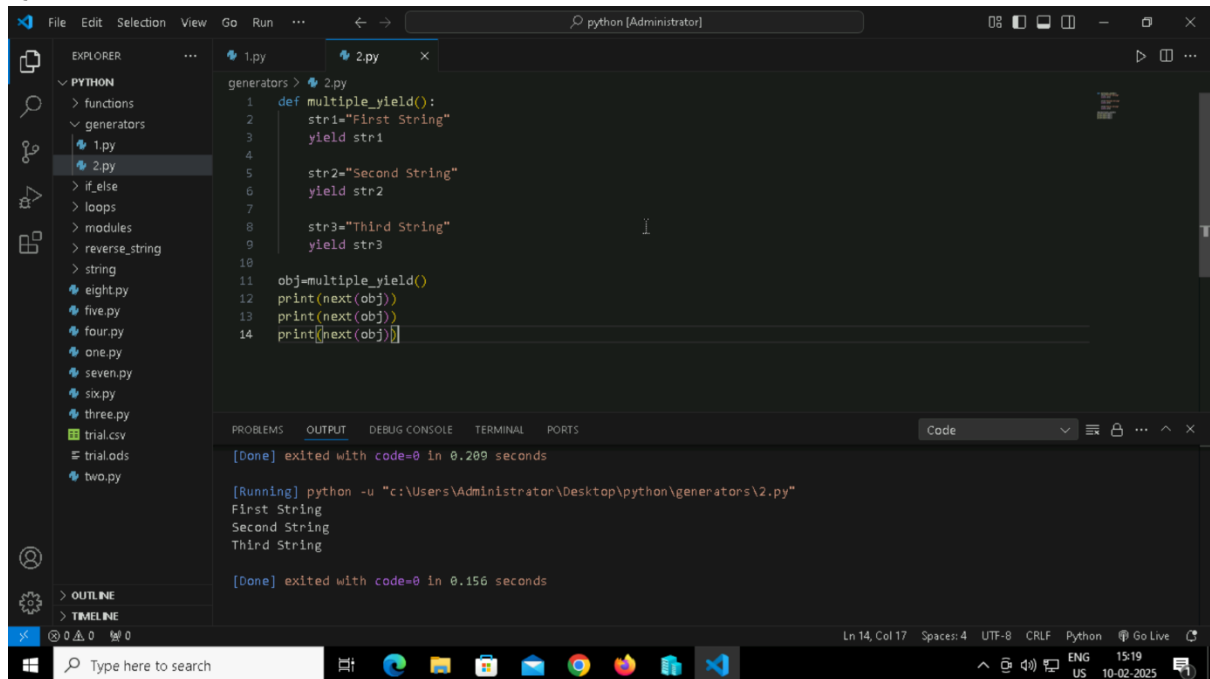
```
generators > 1.py
1 def simple():
2     for i in range(10):
3         if i%2==0:
4             yield i
5
6 for i in simple():
7     print(i)
```

Terminal Output:

```
[Running] python -u "c:\Users\Administrator\Desktop\python\generators\1.py"
0
2
4
6
8

[Done] exited with code=0 in 0.209 seconds
```

Q.



The screenshot shows the Visual Studio Code editor with a file named `2.py` open. The file contains a generator function `multiple_yield()` that yields three strings: "First String", "Second String", and "Third String". The function is called, and the results are printed using `next()`. The output in the terminal shows the three strings being printed sequentially.

```
def multiple_yield():
    str1="First String"
    yield str1
    str2="Second String"
    yield str2
    str3="Third String"
    yield str3

obj=multiple_yield()
print(next(obj))
print(next(obj))
print(next(obj))
```

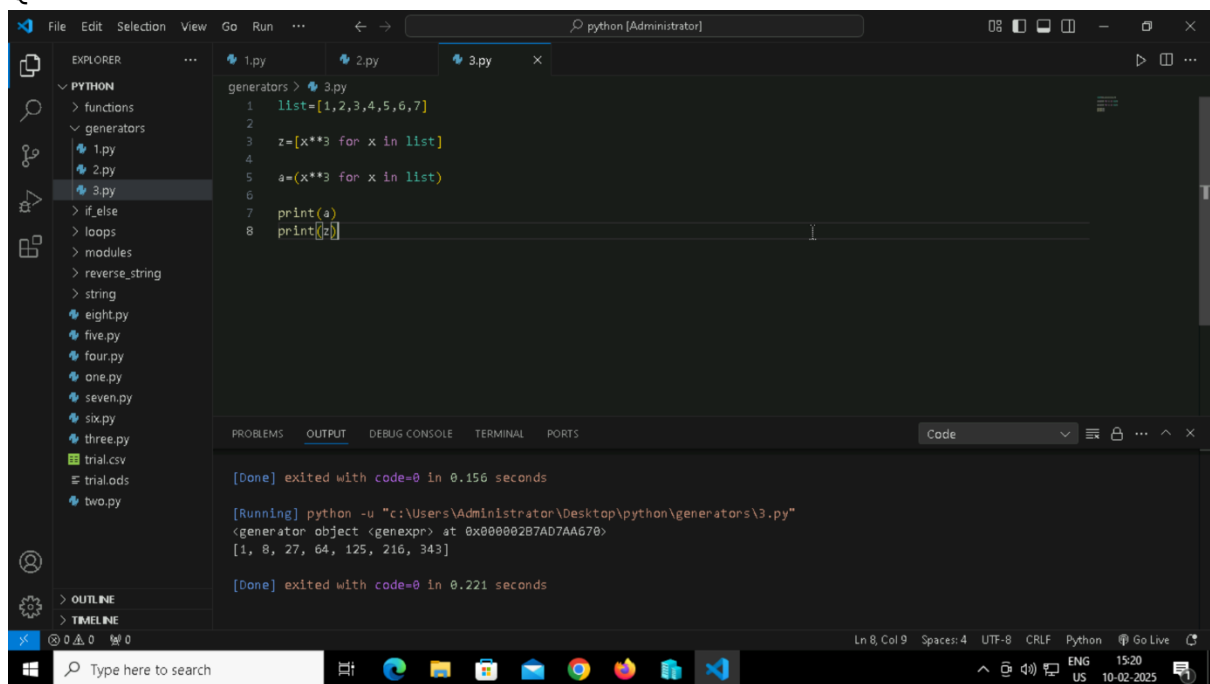
Output:

```
[Done] exited with code=0 in 0.209 seconds

[Running] python -u "c:\Users\Administrator\Desktop\python\generators\2.py"
First String
Second String
Third String

[Done] exited with code=0 in 0.156 seconds
```

Q.



The screenshot shows the Visual Studio Code editor with a file named `3.py` open. The file contains a list `list=[1,2,3,4,5,6,7]` and a generator expression `z=[x**3 for x in list]`. The generator expression is converted to a list `a=[x**3 for x in list]` and printed. The output in the terminal shows the generator object and the resulting list of cubes.

```
list=[1,2,3,4,5,6,7]
z=[x**3 for x in list]
a=[x**3 for x in list]
print(a)
print(z)
```

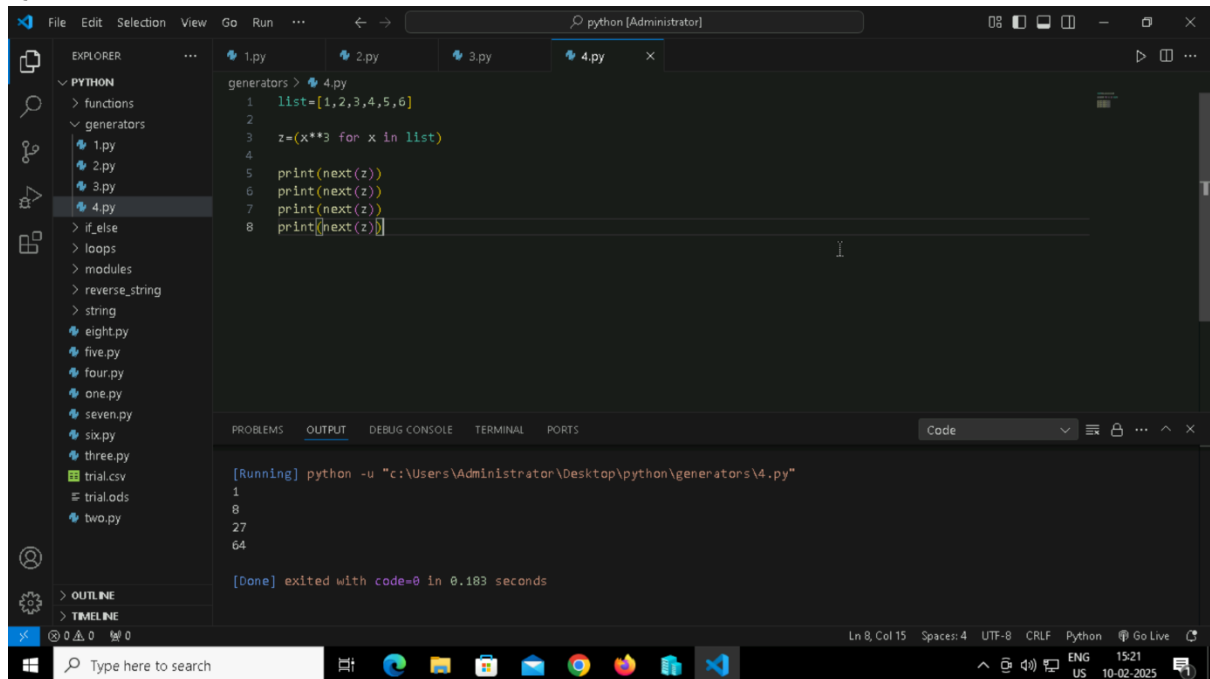
Output:

```
[Done] exited with code=0 in 0.156 seconds

[Running] python -u "c:\Users\Administrator\Desktop\python\generators\3.py"
(generator object <genexpr> at 0x000002B7AD7AA670)
[1, 8, 27, 64, 125, 216, 343]

[Done] exited with code=0 in 0.221 seconds
```

Q.



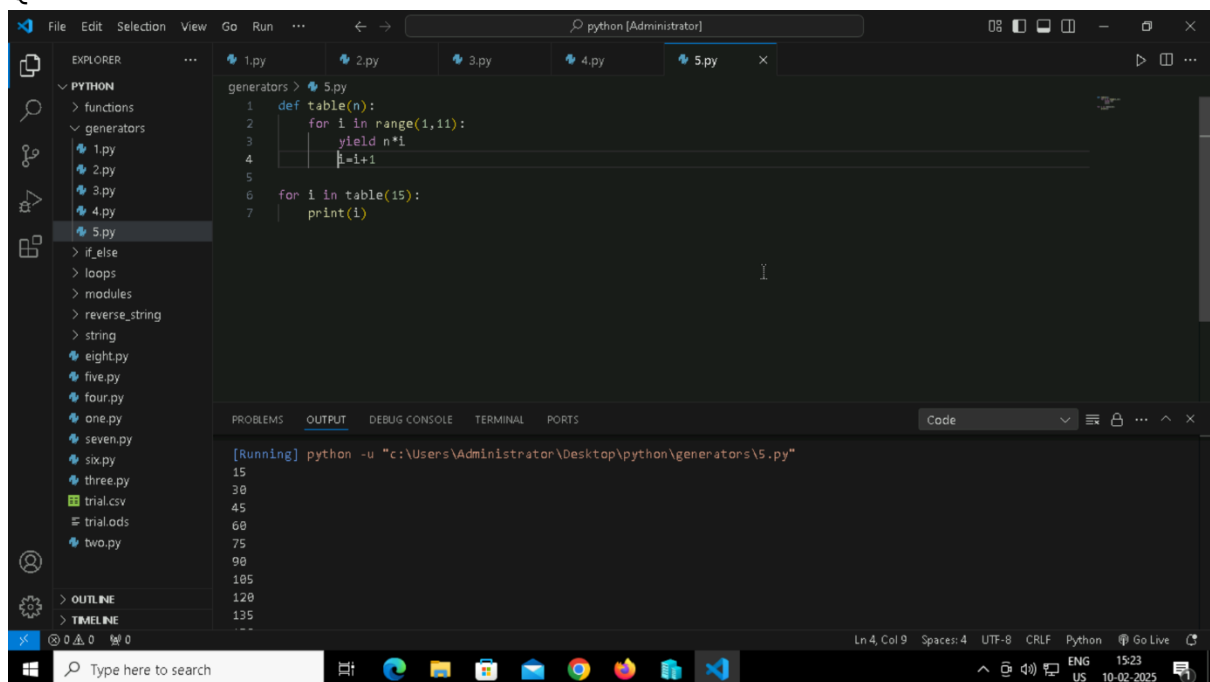
```
generators > 4.py
1 list=[1,2,3,4,5,6]
2
3 z=(x**3 for x in list)
4
5 print(next(z))
6 print(next(z))
7 print(next(z))
8 print(next(z))
```

[Running] python -u "c:\Users\Administrator\Desktop\python\generators\4.py"

1
8
27
64

[Done] exited with code=0 in 0.183 seconds

Q.

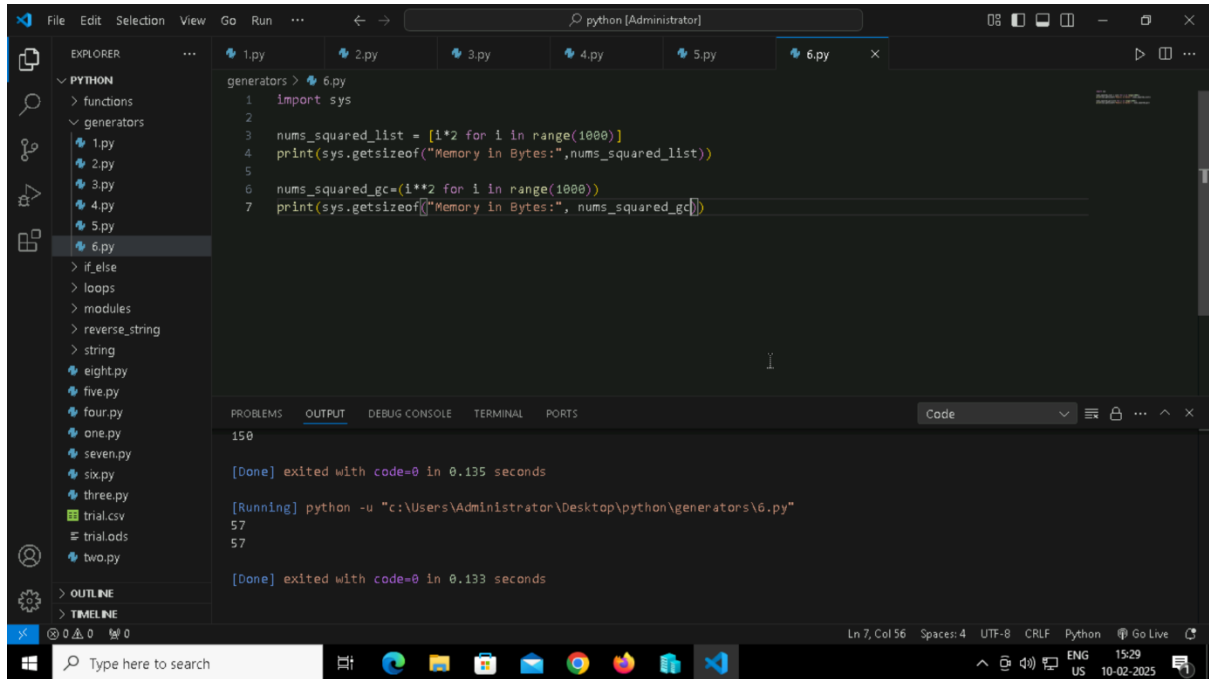


```
generators > 5.py
1 def table(n):
2     for i in range(1,11):
3         yield n*i
4         i=i+1
5
6 for i in table(15):
7     print(i)
```

[Running] python -u "c:\Users\Administrator\Desktop\python\generators\5.py"

15
30
45
60
75
90
105
120
135

Q.



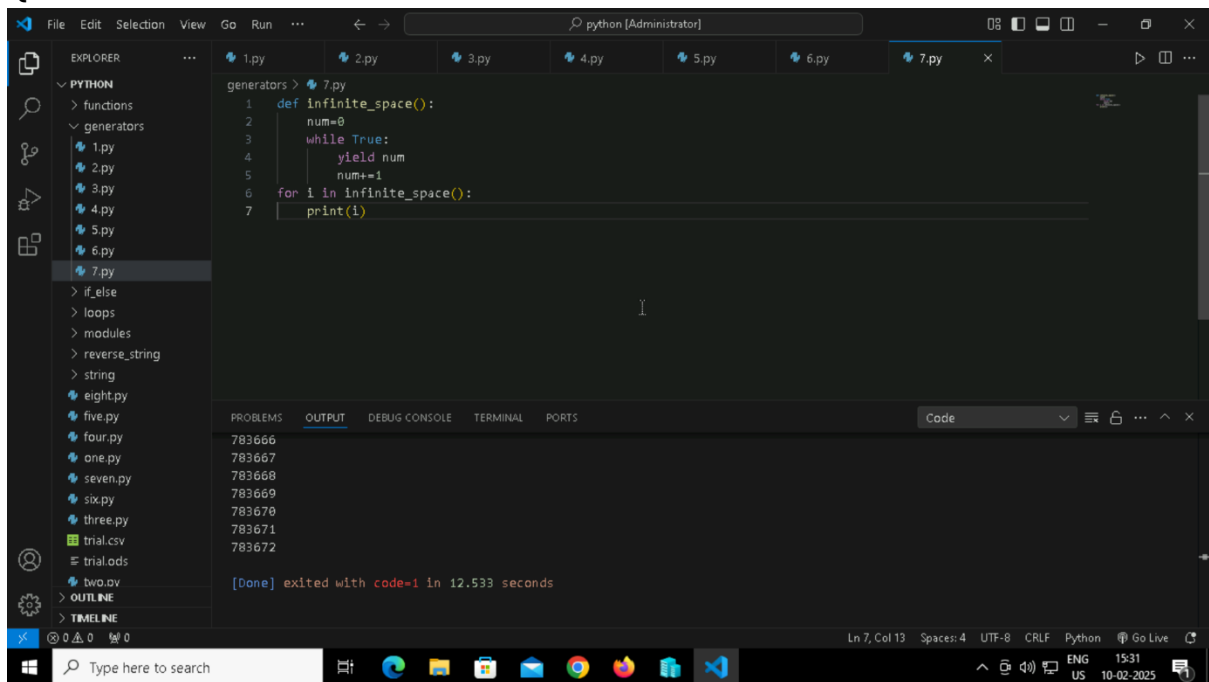
The screenshot shows the Visual Studio Code editor with a Python file named 6.py. The code in the file is as follows:

```
1 import sys
2
3 nums_squared_list = [i**2 for i in range(1000)]
4 print(sys.getsizeof("Memory in Bytes:", nums_squared_list))
5
6 nums_squared_gc=(i**2 for i in range(1000))
7 print(sys.getsizeof("Memory in Bytes:", nums_squared_gc))
```

The output window shows the following results:

```
150
[Done] exited with code=0 in 0.135 seconds
[Running] python -u "c:\Users\Administrator\Desktop\python\generators\6.py"
57
[Done] exited with code=0 in 0.133 seconds
```

Q.



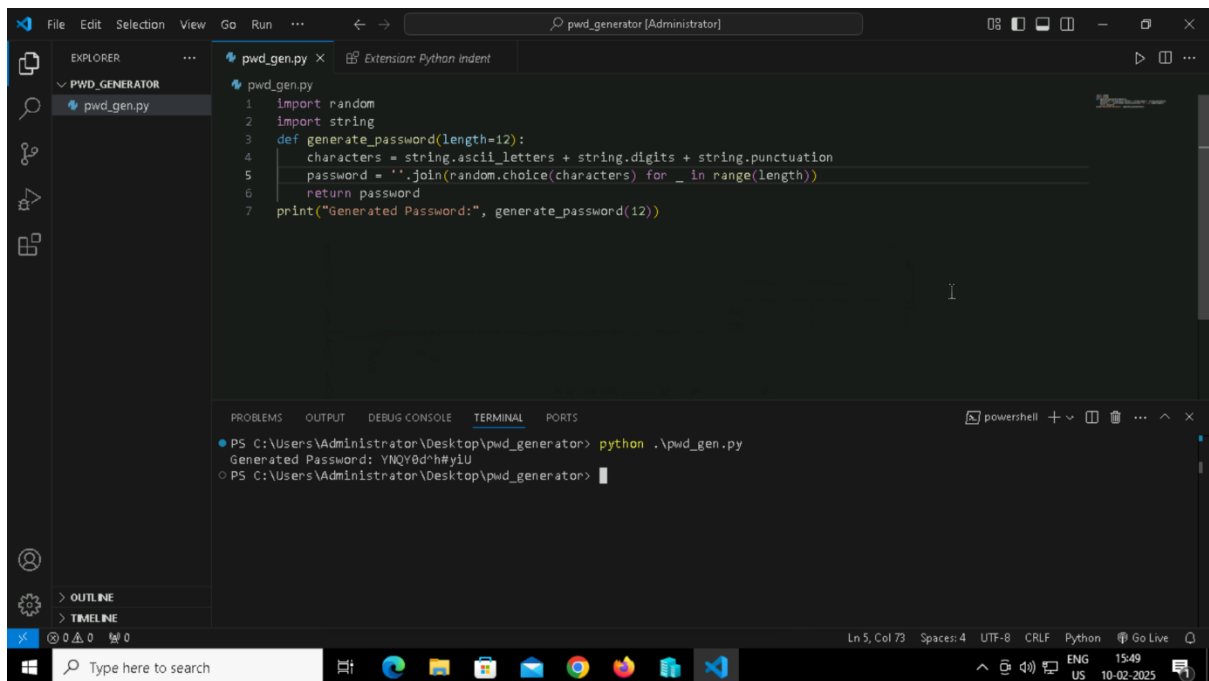
The screenshot shows the Visual Studio Code editor with a Python file named 7.py. The code in the file is as follows:

```
1 def infinite_space():
2     num=0
3     while True:
4         yield num
5         num+=1
6 for i in infinite_space():
7     print(i)
```

The output window shows the following results:

```
783666
783667
783668
783669
783670
783671
783672
[Done] exited with code=1 in 12.533 seconds
```

Q. Password Generator



The image shows a Visual Studio Code editor window with a Python file named `pwd_gen.py` open. The file contains a function `generate_password` that generates a random password of a specified length. The terminal at the bottom shows the command `python .\pwd_gen.py` being executed, which outputs `Generated Password: YNQY8d'h#yIU`.

```
1 import random
2 import string
3 def generate_password(length=12):
4     characters = string.ascii_letters + string.digits + string.punctuation
5     password = ''.join(random.choice(characters) for _ in range(length))
6     return password
7 print("Generated Password:", generate_password(12))
```

Terminal Output:

```
PS C:\Users\Administrator\Desktop\pwd_generator> python .\pwd_gen.py
Generated Password: YNQY8d'h#yIU
PS C:\Users\Administrator\Desktop\pwd_generator>
```


Q. To Do List

The image shows a Visual Studio Code editor window with a Python file named `main.py` and a terminal window below it. The code implements a To-Do List application with the following logic:

```
1 tasks = []
2
3 while True:
4     print("\n1. Add Task\n2. View Tasks\n3. Remove Task\n4. Exit")
5     choice = input("Enter choice: ")
6
7     if choice == "1":
8         task = input("Enter task: ")
9         tasks.append(task)
10        print("Task added!")
11
12    elif choice == "2":
13        print("\nTo-Do List:")
14        for idx, task in enumerate(tasks, 1):
15            print(f"{idx}. {task}")
16
17    elif choice == "3":
18        task_num = int(input("Enter task number to remove: "))
19        if 0 < task_num <= len(tasks):
20            tasks.pop(task_num - 1)
21            print("Task removed!")
22        else:
23            print("Invalid task number.")
24
25    elif choice == "4":
26        break
27
28    else:
29        print("Invalid choice. Try again.")
30
```

The terminal window shows the execution of the program. The user enters choice 1, then "Finish Project", and the task is added. Then the user enters choice 2, and the list shows "1. Finish Project". Next, the user enters choice 3, then "1", and the task is removed. Finally, the user enters choice 4, and the program exits.

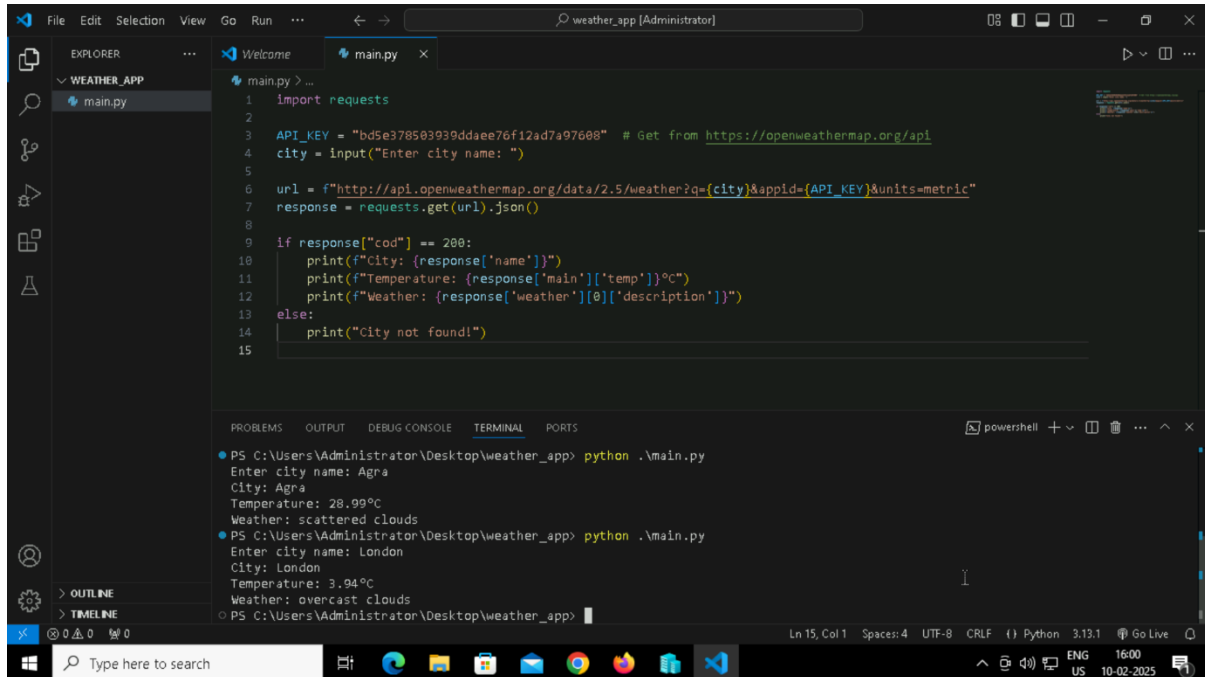
```
PS C:\Users\Administrator\Desktop\to-do-list> python .\main.py
1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter choice: 1
Enter task: Finish Project
Task added!

1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter choice: 2
To-Do List:
1. Finish Project

1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter choice: 3
Enter task number to remove: 1
Task removed!

1. Add Task
2. View Tasks
3. Remove Task
4. Exit
Enter choice: 4
PS C:\Users\Administrator\Desktop\to-do-list>
```

Q. Weather Application



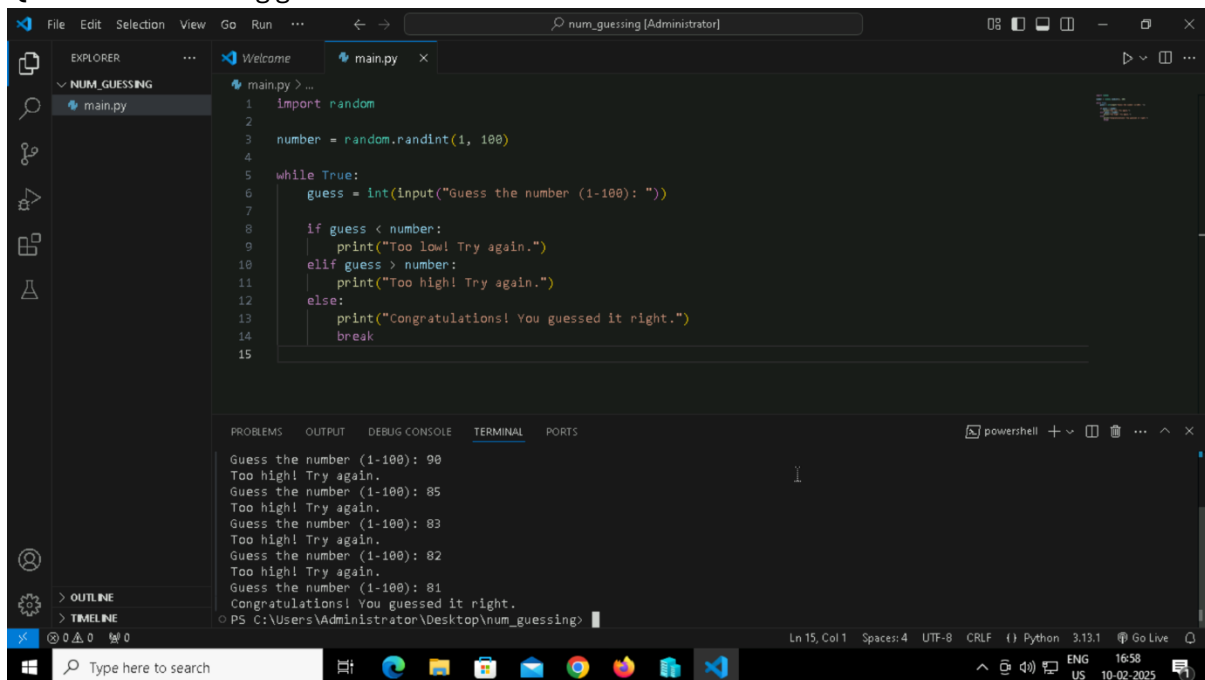
The screenshot shows a Visual Studio Code editor window titled 'weather_app [Administrator]'. The Explorer pane on the left shows a project named 'WEATHER_APP' with a file 'main.py'. The main editor displays the following Python code:

```
1 import requests
2
3 API_KEY = "bd5e378503939ddae76f12ad7a97608" # Get from https://openweathermap.org/api
4 city = input("Enter city name: ")
5
6 url = f"http://api.openweathermap.org/data/2.5/weather?q={city}&appid={API_KEY}&units=metric"
7 response = requests.get(url).json()
8
9 if response["cod"] == 200:
10     print(f"City: {response['name']}")
11     print(f"Temperature: {response['main']['temp']}°C")
12     print(f"Weather: {response['weather'][0]['description']}")
13 else:
14     print("City not found!")
15
```

The TERMINAL pane at the bottom shows the execution of the script:

```
PS C:\Users\Administrator\Desktop\weather_app> python .\main.py
Enter city name: Agra
City: Agra
Temperature: 28.99°C
Weather: scattered clouds
PS C:\Users\Administrator\Desktop\weather_app> python .\main.py
Enter city name: London
City: London
Temperature: 3.94°C
Weather: overcast clouds
PS C:\Users\Administrator\Desktop\weather_app>
```

Q. Number Guessing game



The screenshot shows a Visual Studio Code editor window titled 'num_guessing [Administrator]'. The Explorer pane on the left shows a project named 'NUM_GUESSING' with a file 'main.py'. The main editor displays the following Python code:

```
1 import random
2
3 number = random.randint(1, 100)
4
5 while True:
6     guess = int(input("Guess the number (1-100): "))
7
8     if guess < number:
9         print("Too low! Try again.")
10    elif guess > number:
11        print("Too high! Try again.")
12    else:
13        print("Congratulations! You guessed it right.")
14        break
15
```

The TERMINAL pane at the bottom shows the execution of the script:

```
Guess the number (1-100): 90
Too high! Try again.
Guess the number (1-100): 85
Too high! Try again.
Guess the number (1-100): 83
Too high! Try again.
Guess the number (1-100): 82
Too high! Try again.
Guess the number (1-100): 81
Congratulations! You guessed it right.
PS C:\Users\Administrator\Desktop\num_guessing>
```

Q. QR Code generator

